Гл в 4
Java Persistence API

Приложения включают бизнес-логику, взаимодействие с другими системами, интерфейсы пользователя... и данные. Большую часть данных, которыми манипулируют наши приложения, приходится хранить в базах данных. Оттуда их требуется извлекать, а также анализировать их. Базы данных имеют важное значение: в них хранятся бизнес-данные, они выступают в качестве центральной точки между приложениями и обрабатывают информацию посредством триггеров или хранимых процедур. Постоянные данные встречаются повсюду, и большую часть времени они используют реляционные базы данных как основной механизм обеспечения постоянства (а не бессхемные базы данных). Информация в реляционных базах данных располагается в таблицах, состоящих из строк и столбцов. Данные идентифицируются с помощью первичных ключей, которые представляют собой особые столбцы с ограничениями уникальности и иногда индексами. Связи между таблицами предполагают использование внешних ключей и таблиц соединения с ограничениями целостности.

В таком объектно-ориентированном языке программирования, как Java, вся эта терминология неактуальна. При использовании Java мы манипулируем объектами, являющимися экземплярами классов. Объекты наследуют от других объектов, располагают ссылками на коллекции прочих объектов, а также иногда указывают на себя рекурсивным образом. У нас есть конкретные классы, абстрактные классы, интерфейсы, перечисления, аннотации, методы, атрибуты и т. д. Объекты хорошо инкапсулируют состояние и поведение, однако это состояние доступно только при работающей виртуальной машине Java (Java Virtual Machine — JVM): если виртуальная машина Java останавливается или сборщик мусора удаляет содержимое ее памяти, объекты исчезают вместе со своим состоянием. Некоторые объекты нуждаются в том, чтобы быть постоянными. Под постоянными данными я имею в виду данные, которые намеренно сохранены на перманентной основе на магнитном носителе, флеш-накопителе и т. п. Объект, который может сохранить свое состояние для его повторного использования позднее, называется постоянным.

Основная идея объектно-реляционного отображения (Object-Relational Mapping — ORM) заключается в объединении миров баз данных и объектов. Это подразумевает делегирование доступа к реляционным базам данных внешним инструментам или фреймворкам, которые, в свою очередь, обеспечивают объектно-ориентированное представление реляционных данных, и наоборот. Инструменты отображения предусматривают двунаправленное соответствие между базой данных и объектами. Несколько фреймворков обеспечивают это, например Hibernate, TopLink и Java Data Objects (JDO), а Java Persistence API (JPA) является предпочтительной технологией и частью Java EE 7.

Эта глава представляет собой введение в JPA, а в двух последующих главах я сосредоточусь на объектно-реляционном отображении, а также на выполнении запросов к объектам и управлении объектами.

Понятие сущностей

При разговоре об отображении объектов в реляционной базе данных, обеспечении постоянства объектов или выполнении запросов к ним вместо слова «объект» следует использовать термин «сущность». Объекты — это экземпляры, которые располагаются в памяти. Сущности представляют собой объекты, которые недолго располагаются в памяти, но постоянно — в базе данных. Все они могут быть отображены в базе данных. Они также могут быть конкретными или абстрактными; кроме того, они поддерживают наследование, связи и т. д. Произведя отображение этих сущностей, ими можно управлять посредством JPA. Вы можете обеспечить постоянство сущности в базе данных, удалить ее, а также выполнять запросы к этой сущности с использованием языка запросов Java Persistence Query Language, или JPQL. Объектно-реляционное отображение позволяет вам манипулировать сущностями при доступе к базе данных «за кадром». И, как вы увидите, у сущности имеется определенный жизненный цикл. JPA позволяет вам привязывать бизнес-код к событиям жизненного цикла с применением методов обратного вызова и слушателей.

В качестве первого примера начнем с простой сущности, которая только может быть у нас. В модели постоянства JPA сущность — это простой Java-объект в старом стиле (Plain Old Java Object — POJO). Это означает, что объявление сущности, создание ее экземпляра и использование осуществляются точно так же, как и в случае с любым другим Java-классом. У сущности имеются атрибуты (ее состояние), которыми можно манипулировать с помощью геттеров и сеттеров. Пример простой сущности приведен в листинге 4.1.

Листинг 4.1. Простой пример сущности Book

```
@Entity
public class Book {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
  public Book() {
}
// Геттеры, сеттеры
}
```

В примере в листинге 4.1 представлена сущность `Book`, из которой я для ясности убрал геттеры и сеттеры. Как вы можете видеть, за исключением некоторых анно-

таций, эта сущность выглядит точно так же, как любой другой Java-класс: у нее есть атрибуты (`id`, `title`, `price` и т. д.) разных типов (`Long`, `String`, `Float`, `Integer` и `Boolean`), конструктор по умолчанию, при этом имеются геттеры и сеттеры для каждого атрибута. Как все это отображается в таблицу? Ответ можно получить благодаря аннотациям.

Ан томия сущности

Чтобы класс был сущностью, его нужно снабдить аннотацией `@javax.persistence.Entity`, которая позволит поставщику постоянства признать его постоянным классом, а не простым POJO. Кроме того, аннотация `@javax.persistence.Id` будет определять уникальный идентификатор этого объекта. Поскольку JPA используется для отображения объектов в реляционные таблицы, объектам необходим идентификатор, который будет отображен в первичный ключ. Остальные атрибуты в листинге 4.1 (`title`, `price`, `description` и т. д.) не снабжены аннотациями, поэтому они будут сделаны постоянными путем применения отображения по умолчанию.

Этот пример кода включает атрибуты, однако, как вы увидите позднее, сущность также может располагать бизнес-методами. Обратите внимание, что эта сущность `Book` является Java-классом, который не реализует никакого интерфейса и не расширяет какого-либо класса. Фактически, чтобы быть сущностью, класс должен придерживаться таких правил.

- Класс-сущность должен быть снабжен аннотацией `@javax.persistence.Entity` (или обозначен в XML-дескрипторе как сущность).
- Для обозначения простого первичного ключа должна быть использована аннотация `@javax.persistence.Id`.
- Класс-сущность должен располагать конструктором без аргументов, который должен быть `public` или `protected`. У класса-сущности также могут иметься другие конструкторы.
- Класс-сущность должен быть классом верхнего уровня. Перечисление или интерфейс не могут быть обозначены как сущность.
- Класс-сущность не должен быть `final`. Ни один из методов или постоянные переменные экземпляра класса-сущности тоже не могут быть `final`.
- Если экземпляр сущности надлежит передать с использованием значения как обособленный объект (например, с помощью удаленного интерфейса), то класс-сущность должен реализовывать интерфейс `Serializable`.

ПРИМЕЧАНИЕ

В предыдущих версиях Java EE постоянная компонентная модель называлась Entity Bean (Компонент-сущность EJB) или Entity Bean CMP (Container-Managed Persistence) (Сохраняемость компонентов-сущностей EJB, управляемая контейнером) и была связана с Enterprise JavaBeans. Эта модель постоянства использовалась, начиная с J2EE 1.3 и до появления версии 1.4, однако была тяжеловесной и в конце концов оказалась заменена JPA с выходом версии Java EE 5. В JPA вместо словосочетания «компонент-сущность EJB» используется термин «сущность».

Объектно-реляционное отобр жение

Принцип объектно-реляционного отображения заключается в возложении на внешние инструменты или фреймворки (в нашем случае JPA) задачи по обеспечению соответствия между объектами и таблицами. Тогда мир классов, объектов и атрибутов можно будет отобразить в реляционные базы данных, состоящие из таблиц, которые содержат строки и столбцы. Отображение обеспечивает объектно-ориентированное представление для разработчиков, которые смогут прозрачно использовать сущности вместо таблиц. Как именно JPA отображает объекты в базе данных? Ответ: с помощью *метаданных*.

С каждой сущностью ассоциированы метаданные, которые описывают отображение. Они позволяют поставщику постоянства распознать сущность и интерпретировать отображение. Метаданные могут быть записаны в двух разных форматах.

- *Аннотации* — код сущности непосредственно снабжается всевозможными аннотациями, описанными в пакете `javax.persistence`.
- *XML-дескрипторы* — вместо аннотаций (или в дополнение к ним) вы можете использовать XML-дескрипторы. Отображение определяется во внешнем XML-файле, который будет развернут вместе с сущностями. Это может оказаться очень полезным, если, к примеру, конфигурация базы данных будет изменяться в зависимости от среды.

В случае с сущностью `Book` (показанной в листинге 4.1) используются аннотации JPA, чтобы поставщик постоянства смог синхронизировать данные между атрибутами сущности `Book` и столбцами таблицы `BOOK`. Следовательно, если атрибут `isbn` окажется модифицирован приложением, то будет синхронизирован столбец `ISBN` (при управлении сущностью, задании контекста транзакций и т. д.).

Как показано на рис. 4.1, сущность `Book` отображается в таблице `BOOK`, а каждому столбцу присваивается имя в соответствии с именем атрибута класса (например, атрибут `isbn`, имеющий тип `String`, отображается в столбец, который имеет имя `ISBN` и тип `VARCHAR`). Эти правила отображения по умолчанию являются важной частью принципа, известного как конфигурация в порядке исключения.

В версии Java EE 5 была представлена идея конфигурации в порядке исключения (иногда называемая программированием в порядке исключения или *соглашением прежде конфигурации*), которая по-прежнему активно используется сегодня в Java EE 7. Это означает, что, если не указано иное, контейнер или поставщик должен применять правила по умолчанию. Другими словами, необходимость обеспечения конфигурации является исключением из правила. Это позволяет вам написать минимальное количество кода для того, чтобы ваше приложение работало, положившись на правила, применяемые контейнером и поставщиком по умолчанию. Если вы не хотите, чтобы поставщик применял правила по умолчанию, то можете настроить отображение в соответствии со своими нуждами. Иначе говоря, необходимость обеспечения конфигурации является исключением из правила.

Без аннотаций сущность `Book` в листинге 4.1 рассматривалась бы как простой POJO, а ее постоянство не обеспечивалось бы. Это закон: если не предусмотрено

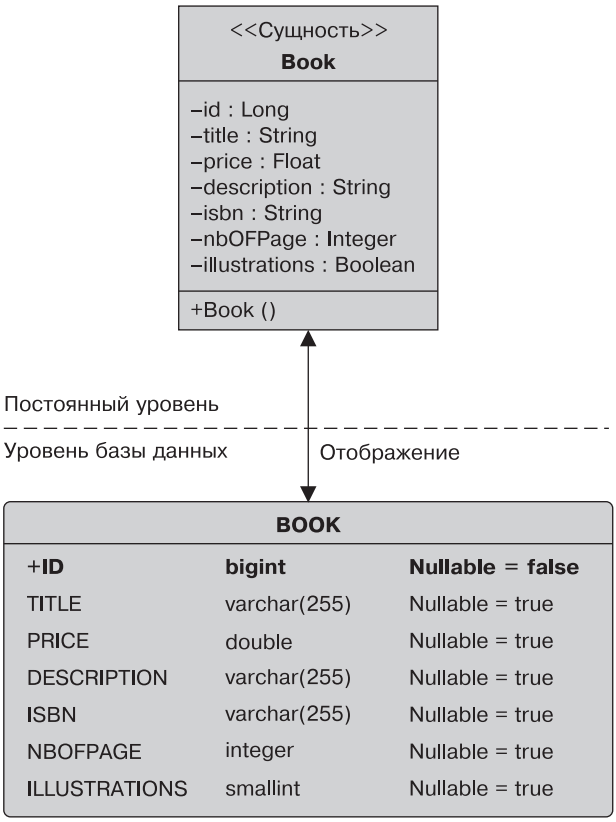


Рис. 4.1. Синхрониз ция д нных между сущностью и т блицей

никакой специальной конфигурации, то должны применяться правила по умолчанию, а поставщик постоянства по умолчанию исходит из того, что у класса `Book` нет представления базы данных. Но, поскольку вам необходимо изменить это поведение по умолчанию, вы снабжаете класс аннотацией `@Entity`. То же самое и в случае с идентификатором. Вам нужен способ сообщить поставщику постоянства о том, что атрибут `id` требуется отобразить в первичный ключ, поэтому вы снабжаете его аннотацией `@Id`, а значение соответствующего идентификатора автоматически генерируется поставщиком постоянства с использованием опциональной аннотации `@GeneratedValue`. Решение такого рода характеризует подход «конфигурация в порядке исключения», при котором аннотации не требуются в более общих случаях, а используются, только когда необходимо переопределение. Это означает, что по отношению ко всем прочим атрибутам будут применяться следующие правила отображения по умолчанию.

- Имя сущности будет отображаться в имя реляционной таблицы (например, сущность `Book` отобразится в таблицу `BOOK`). Если вы захотите отобразить сущность в другую таблицу, то вам потребуется прибегнуть к аннотации `@Table`, как вы увидите в разделе «Элементарное отображение» в следующей главе.

- Имена атрибутов будут отображаться в имени столбца (например, атрибут `id` или метод `getId()` отобразится в столбце `ID`). Если вы захотите изменить это отображение по умолчанию, то вам придется воспользоваться аннотацией `@Column`.
- Правила JDBC применяются при отображении Java-примитивов в типах реляционных данных. `String` отобразится в `VARCHAR`, `Long` — в `BIGINT`, `Boolean` — в `SMALLINT` и т. д. Размер по умолчанию столбца, отображаемого из `String`, будет равен `255` (`String` отобразится в `VARCHAR(255)`). Однако имейте в виду, что правила отображения по умолчанию разнятся от одной базы данных к другой. Например, `String` отображается в `VARCHAR` при использовании базы данных Derby и в `VARCHAR2`, если применяется Oracle. `Integer` отображается в `INTEGER` в случае с базой данных Derby и в `NUMBER` при использовании Oracle. Информация, касающаяся основной базы данных, будет содержаться в файле `persistence.xml`, который вы увидите позднее.

Придерживаясь этих правил, сущность `Book` будет отображена в Derby-таблицу со структурой, описанной в листинге 4.2.

Листинг 4.2. Сценарий для создания структуры таблицы BOOK

```
CREATE TABLE BOOK (
  ID BIGINT NOT NULL,
  TITLE VARCHAR(255),
  PRICE FLOAT,
  DESCRIPTION VARCHAR(255),
  ISBN VARCHAR(255),
  NBOFPAGE INTEGER,
  ILLUSTRATIONS SMALLINT DEFAULT 0,
  PRIMARY KEY (ID)
)
```

JPA2.1 предусматривает API-интерфейс и стандартный механизм для автоматического генерирования баз данных из сущностей и создания сценариев вроде того, что показан в листинге 4.2. Это очень удобно, когда вы находитесь в режиме разработки. Однако бо́льшую часть времени вам понадобится подключение к унаследованной базе данных, которая уже существует.

В листинге 4.1 приведен пример очень простого отображения. Как вы увидите в следующей главе, отображение может быть намного более обширным, включая всевозможные вещи, начиная с объектов и заканчивая связями. Мир объектно-ориентированного программирования изобилует классами и ассоциациями между классами (и коллекциями классов). В случае с базами данных также моделируются связи, но по-другому — с использованием внешних ключей или таблиц соединения. В JPA имеется набор метаданных для управления отображением связей. Даже наследование может быть отображено. Оно широко применяется разработчиками для повторного использования кода, однако эта концепция изначально неизвестна в сфере реляционных баз данных (поскольку им приходится эмулировать наследование с использованием внешних ключей и ограничений). Даже если и придется прибегнуть к некоторым трюкам при отображении наследования, JPA поддерживает его и предлагает вам три разные стратегии на выбор.

ПРИМЕЧАНИЕ

JPA нацелен на реляционные базы данных. Метаданные отображения (аннотации или XML) предназначены для отображения сущностей в структурированные таблицы, а атрибутов — в столбцы. Благодаря разным структурам хранения данных началась новая эра баз данных NoSQL (Not Only SQL — «Не только SQL») (или бессхемных баз данных): ключ/значение, столбец, документ или граф. В настоящее время JPA не позволяет отображать сущности в эти структуры. Hibernate OGM — фреймворк с открытым исходным кодом, который пытается решить этот вопрос. У EclipseLink тоже имеется несколько расширений для отображения NoSQL-структур. Рассмотрение Hibernate OGM и EclipseLink-расширений выходит за рамки этой книги, однако вам следует взглянуть на них, если вы планируете использовать базы данных NoSQL.

Выполнение з просов к сущностям

JPA позволяет вам отображать сущности в базах данных, а также выполнять к ним запросы с использованием разных критериев. Мощь JPA заключается в том, что он дает возможность выполнять запросы к сущностям объектно-ориентированным путем без необходимости применения внешних ключей или столбцов, относящихся к основной базе данных. Центральным элементом API-интерфейса, отвечающего за оркестровку сущностей, является `javax.persistence.EntityManager`. Его роль состоит в управлении сущностями, их чтении и записи в определенную базу данных наряду с обеспечением возможности проведения простых операций CRUD (create, read, update, delete — «создание», «чтение», «обновление», «удаление»), а также комплексных запросов с применением JPQL. С технической точки зрения `EntityManager` представляет собой всего лишь интерфейс, реализация которого обеспечивается поставщиком постоянства (например, EclipseLink). В приведенном далее фрагменте кода показано, как получить `EntityManager` и обеспечить постоянство сущности `Book`:

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("chapter04PU");
EntityManager em = emf.createEntityManager();
em.persist(book);
```

На рис. 4.2 продемонстрировано, как интерфейс менеджера сущностей может быть использован классом (который здесь имеет имя `Main`) для манипулирования сущностями (в данном случае `Book`). С помощью таких методов, как `persist()` и `find()`, `EntityManager` скрывает JDBC-вызовы базы данных и оператор SQL (Structured Query Language — язык структурированных запросов) `INSERT` или `SELECT`.

Менеджер сущностей также позволяет вам выполнять запросы к сущностям. *Запрос* в данном случае аналогичен запросу к базе данных за исключением того, что вместо использования SQL интерфейс JPA выполняет запросы к сущностям с применением JPQL. В его синтаксисе используется привычная точечная (`.`) нотация объектов. Для извлечения информации обо всех книгах, в названии которых присутствует `H2G2`, вы можете написать следующее:

```
SELECT b FROM Book b WHERE b.title = 'H2G2'
```

Следует отметить, что `title` является именем атрибута `Book`, а не именем столбца таблицы. JPQL-операторы манипулируют объектами и атрибутами, а не таблицами и столбцами. JPQL-оператор может выполняться с использованием динамических

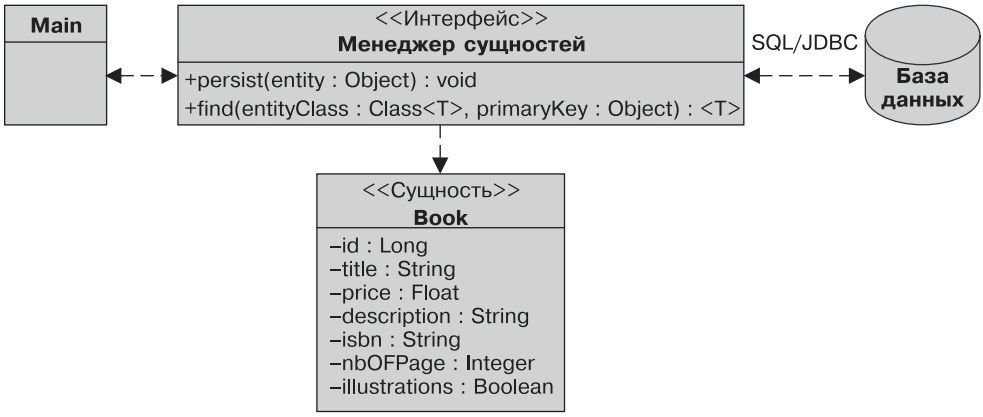


Рис. 4.2. Менеджер сущностей вз имодействует с сущностью и основной б зой д нных

(генерируемых динамически во время выполнения) или статических (определяемых статически во время компиляции) запросов. Вы также можете выполнять «родные» SQL-операторы и даже хранимые процедуры. Статические запросы, также известные как именованные, определяются с использованием либо аннотаций (`@NamedQuery`), либо XML-метаданных. Приведенный ранее JPQL-оператор может, к примеру, быть определен как именованный запрос в отношении сущности `Book`. В листинге 4.3 показана сущность `Book` с определением именованного запроса `findBookH2G2` с помощью аннотации `@NamedQuery` (более подробно о запросах мы поговорим в главе 6).

Листинг 4.3. Сущность Book с именов нным з просом findBookH2G2

```
@Entity
@NamedQuery(name = "findBookH2G2",
            query = "SELECT b FROM Book b WHERE b.title ='H2G2'")
public class Book {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
// Конструкторы, геттеры, сеттеры
}
```

`EntityManager` можно получить в стандартном Java-классе с использованием фабрики. В листинге 4.4 показан такой класс, который создает сущность `Book`, обеспечивает ее постоянство в таблице и выполняет именованный запрос. Соответствующие действия он предпринимает с этапа 1 по 5.

1. Создает экземпляр сущности `Book`. Сущности являются аннотированными POJO, управляемыми поставщиком постоянства. С точки зрения Java экземпляр класса, как и любой POJO, необходимо создавать с помощью ключевого слова

new. Важно подчеркнуть, что до этой точки в коде поставщик постоянства не осведомлен об объекте Book.

2. Получает EntityManager и транзакцию. Это важная часть кода, поскольку EntityManager необходим для манипулирования сущностями. Прежде всего создается EntityManagerFactory для единицы сохраняемости chapter04PU. После этого EntityManagerFactory задействуется для получения EntityManager (переменная em), используется повсюду в коде для получения транзакции (переменная tx) и обеспечения постоянства и извлечения Book.
3. Обеспечивает постоянство Book в базе данных. Код начинает транзакцию (tx.begin()) и использует метод EntityManager.persist() для вставки экземпляра Book. После фиксации транзакции (tx.commit()) информация сбрасывается в базу данных.
4. Выполняет именованный запрос. Опять-таки EntityManager используется для извлечения Book посредством именованного запроса findBookH2G2.
5. Закрывает EntityManager и EntityManagerFactory.

Листинг 4.4. Кл сс Main, который обеспечив ет постоянство и извлек ет сущность Book

```
public class Main {
  public static void main(String[] args) {
    // 1. Создает экземпляр Book
    Book book = new Book("H2G2", "Автостопом по Галактике", 12.5F, ➥
"1-84023-742-2", 354, false);
    // 2. Получает EntityManager и транзакцию
    EntityManagerFactory emf = ➥
Persistence.createEntityManagerFactory("chapter04PU");
    EntityManager em = emf.createEntityManager();
    // 3. Обеспечивает постоянство Book в базе данных
    EntityTransaction tx = em.getTransaction();
    tx.begin();
    em.persist(book);
    tx.commit();
    // 4. Выполняет именованный запрос
    book = em.createNamedQuery("findBookH2G2", Book.class).getSingleResult();
    // 5. Закрывает EntityManager и EntityManagerFactory
    em.close();
    emf.close();
  }
}
```

Обратите внимание на отсутствие SQL-запросов и JDBC-вызовов в листинге 4.4. Как показано на рис. 4.2, класс Main взаимодействует с основной базой данных с помощью интерфейса EntityManager, который обеспечивает набор стандартных методов, позволяющих вам осуществлять операции с сущностью Book. EntityManager «за кадром» полагается на поставщика постоянства для взаимодействия с базами данных. При вызове метода EntityManager поставщик постоянства генерирует и выполняет SQL-оператор с помощью соответствующего JDBC-драйвера.

Единиц сохр няемости

Какой JDBC-драйвер вам следует использовать? Как вам нужно подключаться к базе данных? Каково имя базы данных? Эта информация отсутствует в приводившемся ранее коде. Когда класс `Main` (см. листинг 4.4) создает `EntityManagerFactory`, он передает имя единицы сохраняемости в качестве параметра; в данном случае она имеет имя `chapter04PU`. Единица сохраняемости позволяет `EntityManager` узнать тип базы данных для использования и параметры подключения, определенные в файле `persistence.xml`, который показан в листинге 4.5 и должен быть доступен по пути к соответствующему классу.

Листинг 4.5. Ф йл persistence.xml, определяющий единицу сохр няемости

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence ➥
             http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
             version="2.1">
  <persistence-unit name="chapter04PU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <class>org.agoncal.book.javaee7.chapter04.Book</class>
    <properties>
      <property name="javax.persistence.schema-generation-action" ➥
value="drop-and-create"/>
      <property name="javax.persistence.schema-generation-target"
value="database"/>
      <property name="javax.persistence.jdbc.driver" ➥
                value="org.apache.derby.jdbc.ClientDriver"/>
      <property name="javax.persistence.jdbc.url" ➥
                value="jdbc:derby://localhost:1527/chapter04DB;create=true"/>
      <property name="javax.persistence.jdbc.user" value="APP"/>
      <property name="javax.persistence.jdbc.password" value="APP"/>
    </properties>
  </persistence-unit>
</persistence>
```

Единица сохраняемости `chapter04PU` определяет JDBC-подключение для базы данных Derby `chapter04DB`, которая функционирует на локальном хосте с помощью порта 1527. К ней подключается пользователь (`APP`) с применением пароля (`APP`) по заданному URL-адресу. Тег `<class>` дает указание поставщику постоянства управлять классом `Book` (существуют и другие теги для неявного или явного обозначения классов с управляемым постоянством, например `<mapping-file>`, `<jar-file>` или `<exclude-unlisted-classes>`). Без единицы сохраняемости сущностями можно манипулировать как POJO, однако их постоянство при этом обеспечиваться не будет.

Жизненный цикл сущности и обр тные вызовы

Сущности представляют собой всего лишь POJO. Когда `EntityManager` управляет POJO, у них имеется идентификатор постоянства (ключ, который уникально идентифицирует экземпляр и является эквивалентом первичного ключа), а база данных

синхронизирует их состояние. Когда управление ими не осуществляется (то есть они обособлены от EntityManager), их можно использовать как любой другой Java-класс. Это означает, что у сущностей имеется жизненный цикл (рис. 4.3). Когда вы создадите экземпляр сущности Book с помощью оператора new, объект будет располагаться в памяти, а JPA ничего не будет знать о нем (этот объект даже может перестать существовать в результате сборки мусора) Когда им начнет управлять EntityManager, таблица BOOK отобразит и синхронизирует его состояние. Вызов метода EntityManager.remove() приведет к удалению соответствующей информации из базы данных, однако Java-объект продолжит находиться в памяти, пока не исчезнет в результате сборки мусора.

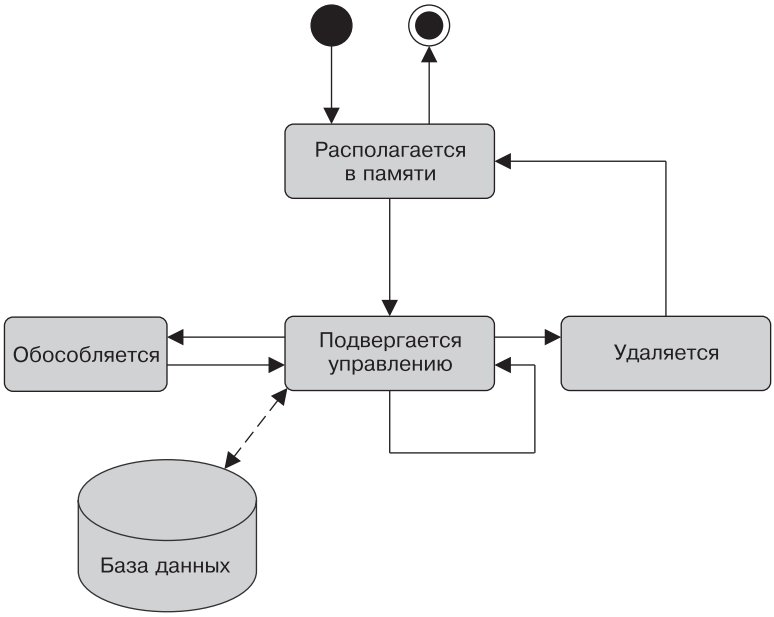


Рис. 4.3. Жизненный цикл сущности

Операции с сущностями подпадают под четыре категории: обеспечение постоянства, обновление, удаление и загрузка — аналогичные категориям операций с базами данных, к которым относятся соответственно вставка, обновление, удаление и выборка. Для каждой операции имеют место события с приставками pre и post (за исключением загрузки, для которой имеет место только событие с приставкой post). Эти события могут быть перехвачены EntityManager для вызова бизнес-метода. Как вы увидите в главе 6, в вашем распоряжении будут аннотации @PrePersist, @PostPersist и т. д. JPA позволяет вам привязывать бизнес-логику к определенной сущности, когда имеют место эти события. Упомянутые аннотации могут быть применены к методам сущностей (также известным как методы обратного вызова) или внешним классам (также известным как слушатели). Вы можете представлять себе методы обратного вызова и слушатели как аналогичные триггеры в реляционной базе данных.

Интегр ция с Bean Validation

Технология Bean Validation, которая была разъяснена в предыдущей главе, связана с Java EE несколькими способами. Один из них выражается в ее интеграции с JPA и жизненным циклом сущностей. Сущности могут включать ограничения Bean Validation и автоматически подвергаться валидации. Фактически автоматическая валидация обеспечивается благодаря тому, что JPA возлагает ее проведение на реализацию Bean Validation при наступлении событий жизненного цикла сущности `pre-persist`, `pre-update` и `pre-remove`. Разумеется, при необходимости валидация по-прежнему может проводиться вручную вызовом метода `validate` класса `Validator` в отношении сущности.

В листинге 4.6 показана сущность `Book` с двумя ограничениями Bean Validation (`@NotNull` и `@Size`). Если значением атрибута `title` окажется `null`, а вы захотите обеспечить постоянство этой сущности (вызвав `EntityManager.persist()`), то во время выполнения JPA будет выброшено исключение `ConstraintViolation`, а соответствующая информация не будет помещена в базу данных.

Листинг 4.6. Сущность с ннот циями Bean Validation

```
@Entity
public class Book {
  @Id @GeneratedValue
  private Long id;
  @NotNull
  private String title;
  private Float price;
  @Size(min = 10, max = 2000)
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
// Конструкторы, геттеры, сеттеры
}
```

Обзор специфик ции JPA

Версия JPA 1.0 была создана вместе с Java EE 5 для решения проблемы обеспечения постоянства данных. Она объединила объектно-ориентированные и реляционные модели. В Java EE 7 версия JPA 2.1 идет тем же путем простоты и надежности, привнося при этом новую функциональность. Вы можете использовать этот API-интерфейс для доступа к реляционным данным Enterprise JavaBeans, веб-компонентам и приложениям Java SE и манипулирования ими.

JPA — это абстракция над JDBC, которая дает возможность быть независимым от SQL. Все классы и аннотации этого API-интерфейса располагаются в пакете `javax.persistence`. Рассмотрим основные компоненты JPA.

- Объектно-реляционное отображение, которое представляет собой механизм отображения объектов в данные, хранящиеся в реляционной базе данных.

- API менеджера сущностей для осуществления операций, связанных с базами данных, например CRUD-операций.
- JPQL, который позволяет вам извлекать данные с помощью объектно-ориентированного языка запросов.
- Транзакции и механизмы блокировки, которые предусматривает Java Transaction API (JTA) при одновременном доступе к данным. JPA также поддерживает ресурсные локальные (не-JTA) транзакции.
- Обратные вызовы и слушатели для добавления бизнес-логики в жизненный цикл того или иного постоянного объекта.

Кр тк я история JPA

Решения, которые позволяют выполнять объектно-реляционное отображение, существуют уже долгое время, причем они появились раньше Java. Продукты вроде TopLink начинали со Smalltalk в 1994 году, перед тем как перейти на Java. Коммерческие продукты для выполнения объектно-реляционного отображения, например TopLink, доступны с самых первых дней существования языка Java. Они стали успешными, но никогда не были стандартизированы для платформы Java. Схожий подход к объектно-реляционному отображению был стандартизирован в форме технологии JDO, которой так и не удалось значительно проникнуть на рынок.

В 1998 году появилась версия EJB 1.0, которая позднее сопутствовала J2EE 1.2. Это был тяжеловесный, распределенный компонент, использовавшийся для обработки транзакционной бизнес-логики. Технология Entity Bean CMP, представленная в EJB 1.0 как опциональная, стала обязательной в EJB 1.1 и совершенствовалась с выходом последующих версий вплоть до EJB 2.1 (J2EE 1.4). Постоянство могло обеспечиваться только внутри контейнера благодаря сложному механизму создания экземпляров с использованием домашних, локальных или удаленных интерфейсов. Возможности в плане объектно-реляционного отображения тоже оказались весьма ограниченными, поскольку наследование было сложно отображать.

Параллельно с миром J2EE существовало популярное решение с открытым исходным кодом, которое привело к удивительным изменениям в «направлении» постоянства: имеется в виду технология Hibernate, вернувшая назад легковесную, объектно-ориентированную постоянную модель.

Спустя годы жалоб на компоненты Entity CMP 2.*x* и в знак признания успеха и простоты фреймворков с открытым исходным кодом вроде Hibernate архитектура модели постоянства Enterprise Edition была полностью пересмотрена в Java EE 5. Версия JPA 1.0 была создана с использованием весьма облегченного подхода, который позаимствовал многие принципы проектирования Hibernate. Спецификация JPA 1.0 была связана с EJB 3.0 (JSR 220). В 2009 году версия JPA 2.0 (JSR 317) сопутствовала Java EE 6 и привнесла новые API-интерфейсы, расширила JPQL и добавила новую функциональность, такую, например, как кэш второго уровня, пессимистическая блокировка или Criteria API.

Сегодня, когда уже есть Java EE 7, версия JPA 2.1 стремится к легкости разработки и привносит новые функции. Она эволюционировала в JSR 338.

Что нового в JPA 2.1

Если версия JPA 1.0 стала революцией по сравнению со своим предком Entity CMP 2.*x* в силу совершенно новой модели постоянства, JPA 2.0 оказалась продолжением JPA 1.0, а сегодня версия JPA2.1 идет тем же путем и привносит множество новых опций и усовершенствований.

- Генерирование схем — JPA 2.1 включает механизм генерирования стандартизированных схем баз данных, привнося новый API-интерфейс и набор свойств (которые определяются в файле `persistence.xml`).
- Преобразователи — новые классы, которые обеспечивают преобразование между представлениями баз данных и атрибутов.
- Поддержка CDI — теперь можно внедрять зависимости в слушатели событий.
- Поддержка хранимых процедур — JPA 2.1 позволяет выполнять динамически генерируемые и именованные запросы к хранимым процедурам.
- Запросы с использованием критериев на пакетное обновление и удаление — Criteria API позволял выполнять только запросы на выборку; теперь стали возможны запросы на обновление и удаление.
- Понижающее приведение — новый оператор `TREAT` обеспечивает доступ к специфичному для подклассов состоянию в запросах.

В табл. 4.1 приведены основные пакеты, определенные в JPA 2.1 на сегодняшний день.

Таблица 4.1. Основные JPA-пакеты

Пакет	Описание
javax.persistence	API-интерфейс для управления постоянством и объектно-реляционным отображением
javax.persistence.criteria	Java Persistence Criteria API
javax.persistence.metamode	Java Persistence Metamodel API
javax.persistence.spi	SPI для поставщиков Java Persistence

Эталонная реализация

EclipseLink 2.5 представляет собой эталонную реализацию JPA 2.1, имеющую открытый исходный код. Она обеспечивает мощный и гибкий фреймворк для сохранения Java-объектов в реляционных базах данных. EclipseLink является реализацией JPA, однако также поддерживает XML-постоянство с помощью Java XML Binding (JAXB) и других средств вроде Service Data Objects (SDO). EclipseLink предусматривает поддержку не только объектно-реляционного отображения, но и технологии Object XML Mapping (OXM), постоянства объектов в информационных системах предприятий (Enterprise Information System — EIS) с использованием Java EE Connector Architecture (JCA) и веб-служб баз данных.

Истоки EclipseLink берут свое начало в продукте TopLink от компании Oracle, переданном организации Eclipse Foundation в 2006 году. EclipseLink является

эталонной реализацией JPA и фреймворком постоянства, используемым в этой книге. Его также называют поставщиком постоянства, или просто поставщиком.

На момент написания этой книги EclipseLink был лишь реализацией JPA 2.1. Однако вскоре последуют Hibernate и OpenJPA и у вас будет несколько реализаций на выбор.

Все вместе

Теперь, когда вы немного познакомились с JPA, EclipseLink, сущностями, менеджером сущностей и JPQL, сведем их воедино и напишем небольшое приложение, которое будет обеспечивать постоянство сущности в базе данных. Идея заключается в том, чтобы написать простую сущность `Book` с ограничениями Bean Validation и класс `Main`, который позаботится о ее постоянстве. Затем вы проведете компиляцию соответствующего кода с помощью Maven и выполните его с использованием EclipseLink и клиентской базы данных Derby. Чтобы продемонстрировать, насколько легко провести интеграционное тестирование сущности, я покажу вам, как написать тестовый класс (`BookIT`) с применением JUnit 4 и задействовать встроенный режим Derby для обеспечения постоянства данных с использованием базы данных в оперативной памяти.

В этом примере мы будем придерживаться структуры каталогов Maven, в силу чего классы и файлы, показанные на рис. 4.4, должны будут располагаться в следующих каталогах:

- `src/main/java` — для сущности `Book` и класса `Main`;
- `src/main/resources` — для файла `persistence.xml`, которым будут пользоваться классы `Main` и `BookIT`, а также сценарий загрузки базы данных `insert.sql`;
- `src/test/java` — для класса `BookIT`, который будет применяться для интеграционного тестирования;
- `pom.xml` — для POM-модели Maven, которая описывает проект и его зависимости от других внешних модулей и компонентов.

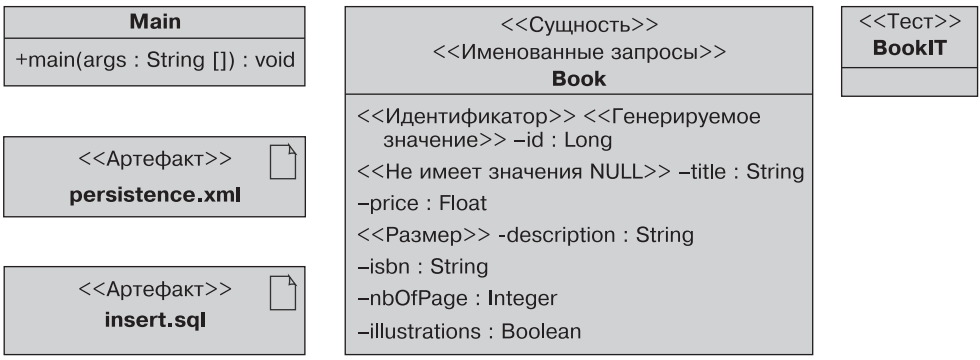


Рис. 4.4. Все вместе

Н пис ние сущности Book

Сущность Book, показанную в листинге 4.7, необходимо создать и разместить в каталоге src/main/java. У нее будет несколько атрибутов (title, price и т. д.) с разными типами данных (String, Float, Integer и Boolean), аннотации Bean Validation (@NotNull и @Size), а также аннотации JPA.

- @Entity проинформирует поставщика постоянства о том, что этот класс является сущностью и ему следует управлять им.
- Аннотации @NamedQueries и @NamedQuery будут определять два именованных запроса, которые станут использовать JPQL для извлечения информации о книгах из базы данных.
- @Id будет определять атрибут id как первичный ключ.
- Аннотация @GeneratedValue проинформирует поставщика постоянства о необходимости автоматического генерирования первичного ключа с использованием инструмента id основной базы данных.

Листинг 4.7. Сущность Book и именов нный з прос

```
package org.agoncal.book.javaee7.chapter04;
@Entity
@NamedQueries({
  @NamedQuery(name = "findAllBooks", query = "SELECT b FROM Book b"),
  @NamedQuery(name = "findBookH2G2", query = "SELECT b FROM Book b
WHERE b.title ='H2G2'")
})
public class Book {
  @Id @GeneratedValue
  private Long id;
  @NotNull
  private String title;
  private Float price;
  @Size(min = 10, max = 2000)
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Обратите внимание, что для лучшей удобочитаемости я убрал конструктор, геттеры и сеттеры этого класса. Как видно из кода, если не считать нескольких аннотаций, Book представляет собой простой POJO. А теперь напишем класс Main, который будет обеспечивать постоянство Book в базе данных.

Н пис ние кл сс Main

Класс Main, показанный в листинге 4.8, будет располагаться в том же пакете, что и сущность Book. Он начнет с того, что создаст новый экземпляр Book (с использованием ключевого слова Java new) и задаст значения для его атрибутов. Здесь не

будет ничего особенного, лишь чистый Java-код. Затем он воспользуется классом Persistence для получения экземпляра EntityManagerFactory, ссылающегося на единицу сохраняемости с именем chapter04PU, которую я охарактеризую позднее, в разделе «Написание единицы сохраняемости». EntityManagerFactory создаст экземпляр EntityManager (переменная em). Как уже отмечалось ранее, EntityManager является центральным элементом JPA в том смысле, что способен начать транзакцию, обеспечить постоянство объекта Book с помощью метода EntityManager.persist(), а затем произвести фиксацию транзакции. В конце метода main() как EntityManager, так и EntityManagerFactory будут закрыты, чтобы высвободить ресурсы поставщика.

Листинг 4.8. Кл cc Main, который обеспечив ет постоянство сущности Book

```
package org.agoncal.book.javaee7.chapter04;
public class Main {
  public static void main(String[] args) {
    // Создает экземпляр Book
    Book book = new Book("H2G2", "Автостопом по Галактике", ➥
    12.5F, "1-84023-742-2", 354, false);
    // Получает EntityManager и транзакцию
    EntityManagerFactory emf = ➥
Persistence.createEntityManagerFactory("chapter04PU");
    EntityManager em = emf.createEntityManager();
    // Обеспечивает постоянство Book в базе данных
    EntityTransaction tx = em.getTransaction();
    tx.begin();
    em.persist(book);
    tx.commit();
    // Закрывает EntityManager и EntityManagerFactory
    em.close();
    emf.close();
  }
}
```

Опять-таки ради удобочитаемости я убрал обработку исключений. Если будет иметь место исключение постоянства, то вам придется откатить транзакцию, зарегистрировать сообщение и закрыть EntityManager в финальном блоке.

Н пис ние интегр ционного тест BookIT

Одна из жалоб на предыдущие версии Entity CMP 2.*x* заключалась в сложности интеграционного тестирования постоянных компонентов. Один из основных выигрышных моментов JPA состоит в том, что вы можете с легкостью тестировать сущности без необходимости в работающем сервере приложений или реальной базе данных. Но какие именно функции вы можете протестировать? Сущности как таковые обычно не нуждаются в тестировании в изоляции. Большинство методов сущностей представляют собой простые геттеры или сеттеры. Проверка того, что сеттер присваивает значение атрибуту, а также того, что соответствующий геттер извлекает то же значение, не имеет особой ценности (если только побочный эффект не проявится в геттерах или сеттерах). Поэтому модульное тестирование сущностей представляет ограниченный интерес.

А как насчет тестирования запросов к базе данных? Нужно ли убедиться в том, что запрос findBookH2G2 корректен? Или вводить информацию в базу данных и тестировать комплексные запросы, возвращающие множественные значения? Такие интеграционные тесты потребовали бы реальной базы данных с реальной информацией, либо вы стали бы проводить модульное тестирование в изоляции, пытаясь симулировать запрос. Хорошее компромиссное решение — использовать базу данных в оперативной памяти и JPA-транзакции. CRUD-операции и JPQL-запросы могут быть протестированы с применением очень легковесной базы данных, которая не потребует запуска отдельного процесса (понадобится лишь добавить файл с расширением .jar, используя путь к соответствующему классу). Рассмотрим, как придется задействовать наш класс BookIT во встроенном режиме Derby.

Maven использует два разных каталога, один из которых применяется для размещения кода главного приложения, а другой — для тестовых классов. Класс BookIT, показанный в листинге 4.9, располагается в каталоге src/test/java и осуществляет тестирование на предмет того, может ли менеджер сущностей обеспечить постоянство сущности Book и извлекать ее из базы данных, а также удостоверяется в том, что применяются ограничения Bean Validation.

Листинг 4.9. Тестовый кл сс, который созд ет и извлек ет Book из б зы д нных

```
public class BookIT {
  private static EntityManagerFactory emf = ➥
                Persistence.createEntityManagerFactory("chapter04TestPU");
  private EntityManager em;
  private EntityTransaction tx;
  @Before
  public void initEntityManager() throws Exception {
    em = emf.createEntityManager();
    tx = em.getTransaction();
  }
  @After
  public void closeEntityManager() throws Exception {
    if (em != null) em.close();
  }
  @Test
  public void shouldFindjavaee7Book() throws Exception {
    Book book = em.find(Book.class, 1001L);
    assertEquals("Изучаем Java EE 7", book.getTitle());
  }
  @Test
  public void shouldCreateH2G2Book() throws Exception {
    // Создает экземпляр Book
    Book book = new Book("H2G2", "Автостопом по Галактике", ➥
    12.5F, "1-84023-742-2", 354, false);
    // Обеспечивает постоянство Book в базе данных
    tx.begin();
    em.persist(book);
    tx.commit();
    assertNotNull("ID не может быть пустым", book.getId());
```

```
    // Извлекает информацию обо всех соответствующих книгах из базы данных
    book = em.createNamedQuery("findBookH2G2", Book.class).getSingleResult();
    assertEquals("Автостопом по Галактике", book.getDescription());
  }
  @Test(expected = ConstraintViolationException.class)
  public void shouldRaiseConstraintViolationCauseNullTitle() {
    Book book = new Book(null, "Пустое название, ошибка", 12.5F, ➥
    "1-84023-742-2", 354, false);
    em.persist(book);
  }
}
```

Как и классу `Main`, `BookIT` в листинге 4.9 необходимо создать экземпляр `EntityManager` с использованием `EntityManagerFactory`. Для инициализации этих компонентов можно прибегнуть к фикстурам JUnit 4. Аннотации `@Before` и `@After` позволяют выполнять некоторый код до и после выполнения теста. Это идеальное место для создания и закрытия экземпляра `EntityManager` и получения транзакции.

Вариант тестирования `shouldFindJavaEE7Book()` опирается на информацию, которая уже присутствует в базе данных (подробнее о сценарии `inset.sql` мы поговорим позднее), и при поиске `Book` с идентификатором `1001` мы убеждается в том, что названием является `"Изучаем Java EE 7"`. Метод `shouldCreateH2G2Book()` обеспечивает постоянство `Book` (с помощью метода `EntityManager.persist()`) и проверяет, был ли идентификатор автоматически сгенерирован EclipseLink (с использованием `assertNotNull`). Если да, то выполняется именованный запрос и осуществляется проверка, является ли `"Автостопом по Галактике"` описанием возвращенной сущности `Book`. Последний вариант тестирования создает `Book` с атрибутом `Nulltitle`, обеспечивает постоянство `Book` и удостоверяется в том, что было сгенерировано исключение `ConstraintViolationException`.

Написание единицы сохраняемости

Как вы можете видеть в классе `Main` (см. листинг 4.8), `EntityManagerFactory` требуется единица сохраняемости с именем `chapter04PU`. А для интеграционного теста `BookIT` (см. листинг 4.9) используется другая единица сохраняемости (`chapter04TestPU`). Эти две единицы сохраняемости должны быть определены в файле `persistence.xml`, находящемся в каталоге `src/main/resources/META-INF` (листинг 4.10). Этот файл, необходимый согласно спецификации JPA, важен, поскольку соединяет поставщика JPA (которым в нашем случае выступает EclipseLink) с базой данных (Derby). Он содержит всю информацию, необходимую для подключения к базе данных (URL-адрес, JDBC-драйвер, сведения о пользователе, пароль), и сообщает поставщику режим генерирования схемы базы данных (`drop-and-create` означает, что таблицы будут удалены, а затем снова созданы). Элемент `<provider>` определяет поставщика постоянства, которым в нашем случае является EclipseLink. Единицы сохраняемости позволяют узнать обо всех сущностях, которыми должен управлять менеджер сущностей. Здесь в теге `<class>` предусмотрена ссылка на сущность `Book`.

Две единицы сохраняемости отличаются в том смысле, что `chapter04PU` использует работающую базу данных Derby, а `chapter04TestPU` — ту, что располагается в оперативной памяти. Обратите внимание, что они обе задействуют сценарий `inset.sql` для ввода информации в базу данных во время выполнения.

Листинг 4.10. Ф йл persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
             http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
             version="2.1">
  <persistence-unit name="chapter04PU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <class>org.agoncal.book.javaee7.chapter04.Book</class>
    <properties>
      <property name="javax.persistence.schema-generation-action"
value="drop-and-create"/>
      <property name="javax.persistence.schema-generation-target"
                value="database-and-scripts"/>
      <property name="javax.persistence.jdbc.driver"
                value="org.apache.derby.jdbc.ClientDriver"/>
      <property name="javax.persistence.jdbc.url"
                value="jdbc:derby://localhost:1527/chapter04DB;create=true"/>
      <property name="javax.persistence.jdbc.user" value="APP"/>
      <property name="javax.persistence.jdbc.password" value="APP"/>
      <property name="javax.persistence.sql-load-script-source"
value="insert.sql"/>
    </properties>
  </persistence-unit>
  <persistence-unit name="chapter04TestPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <class>org.agoncal.book.javaee7.chapter04.Book</class>
    <properties>
      <property name="javax.persistence.schema-generation-action"
value="drop-and-create"/>
      <property name="javax.persistence.schema-generation-target"
value="database"/>
      <property name="javax.persistence.jdbc.driver"
                value="org.apache.derby.jdbc.EmbeddedDriver"/>
      <property name="javax.persistence.jdbc.url"
                value="jdbc:derby:memory:chapter04DB;create=true"/>
      <property name="javax.persistence.sql-load-script-source"
value="insert.sql"/>
    </properties>
  </persistence-unit>
</persistence>
```

Н пис ние SQL-сцен рия для з грузки д нных

Обе единицы сохраняемости, определенные в листинге 4.10, загружают сценарий `insert.sql` (с использованием свойства `javax.persistence.sql-loadscript-source`). Это означает, что сценарий, показанный в листинге 4.11, выполняется для инициализации базы данных и вводит информацию о трех книгах.

Листинг 4.11. Ф йл insert.sql

```
INSERT INTO BOOK(ID, TITLE, DESCRIPTION, ILLUSTRATIONS, ISBN, NBOFPAGE,
                 PRICE) values (1000, Изучаем Java EE 6',
                 'Лучшая книга о Java EE', 1, '1234-5678', 450, 49)
INSERT INTO BOOK(ID, TITLE, DESCRIPTION, ILLUSTRATIONS, ISBN, NBOFPAGE,
                 PRICE) values (1001, 'Изучаем Java EE 7', 'Нет, это лучшая',
                 1, '5678-9012', 550, 53)
INSERT INTO BOOK(ID, TITLE, DESCRIPTION, ILLUSTRATIONS, ISBN, NBOFPAGE,
                 PRICE) values (1010, 'Властелин колец', 'Одно кольцо
                 для управления всеми остальными', 0, '9012-3456', 222, 23)
```

Если вы внимательно посмотрите на интеграционный тест BookIT (метод shouldFindJavaEE7Book), то увидите, что в случае с ним ожидается, что в базе данных будет идентификатор Book в виде 1001. Благодаря инициализации базы данных соответствующие действия предпринимаются до выполнения тестов.

Компиляция и тестиров ние с использов нием Maven

У нас есть все составляющие для компиляции и тестирования сущности перед выполнением приложения Main: сущность Book, интеграционный тест BookIT и единицы сохраняемости, которые привязывают эту сущность к базе данных Derby. Для компиляции этого кода вы воспользуетесь Maven вместо того, чтобы прибегать непосредственно к команде компилятора javac. Вам сначала потребуется создать файл pom.xml, описывающий проект и его зависимости вроде JPA и Bean Validation API. Вам также придется проинформировать Maven о том, что вы используете Java SE 7, сконфигурировав maven-compiler-plugin, как показано в листинге 4.12.

Листинг 4.12. Maven-ф йл pom.xml для компиляции, тестиров ния и выполнения приложения

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <artifactId>parent</artifactId>
    <groupId>org.agoncal.book.javaee7</groupId>
    <version>1.0</version>
  </parent>
  <groupId>org.agoncal.book.javaee7</groupId>
  <artifactId>chapter04</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
```

```
      <groupId>org.eclipse.persistence</groupId>
      <artifactId>org.eclipse.persistence.jpa</artifactId>
      <version>2.5.0</version>
    </dependency>
    <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-validator</artifactId>
      <version>5.0.0</version>
    </dependency>
    <dependency>
      <groupId>org.apache.derby</groupId>
      <artifactId>derbyclient</artifactId>
      <version>10.9.1.0</version>
    </dependency>
    <dependency>
      <groupId>org.apache.derby</groupId>
      <artifactId>derby</artifactId>
      <version>10.9.1.0</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>2.5.1</version>
        <configuration>
          <source>1.7</source>
          <target>1.7</target>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-failsafe-plugin</artifactId>
        <version>2.12.4</version>
        <executions>
          <execution>
            <id>integration-test</id>
            <goals>
              <goal>integration-test</goal>
              <goal>verify</goal>
            </goals>
          </execution>
```

```
        </executions>
      </plugin>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>1.2.1</version>
        <executions>
          <execution>
            <goals>
              <goal>java</goal>
            </goals>
          </execution>
        </executions>
        <configuration>
          <mainClass>org.agoncal.book.javaee7.chapter04.Main</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Прежде всего, чтобы вы смогли произвести компиляцию кода, вам потребуется JPA API, который определяет все аннотации и классы, имеющиеся в пакете `javax.persistence`. Все это, а также время выполнения EclipseLink (то есть поставщика постоянства) будет обеспечиваться с помощью идентификатора артефакта `org.eclipse.persistence.jpa`. Как можно было видеть в предыдущей главе, Bean Validation API заключен в артефакт `hibernate-validator`. Кроме того, вам потребуются JDBC-драйверы для подключения к Derby. Идентификатор артефакта `derbyclient` ссылается на файл с расширением `.jar`, содержащий JDBC-драйвер для подключения к Derby, работающей в серверном режиме (база данных функционирует как отдельный процесс и прослушивает порт), а идентификатор артефакта `derby` включает в себя классы для использования Derby как встраиваемой системы управления базами данных. Обратите внимание, что область этого идентификатора артефакта ограничивается тестированием (`<scope>test</scope>`), как и артефакта для JUnit 4.

Для компиляции классов откройте интерпретатор командной строки в корневом каталоге, содержащем файл `pom.xml`, и введите приведенную далее Maven-команду:

```
$ mvn compile
```

После этого вы должны будете увидеть сообщение `BUILD SUCCESS`, говорящее о том, что компиляция прошла успешно. Maven создаст подкаталог target со всеми файлами наряду с `persistence.xml`. Для выполнения интеграционных тестов снова прибегните к Maven, введя следующую команду:

```
$ mvn integration-test
```

Вы должны будете увидеть несколько журналов касаемо Derby при создании базы данных и таблиц в памяти. Затем окажется задействован класс `BookIT`, а отчет Maven проинформирует вас о том, что результаты применения трех вариантов тестирования оказались успешными:

```
Results :
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 5.192s
[INFO] Finished
[INFO] Final Memory: 18M/221M
[INFO] ------------------------------------------------------------------------
```

Применение кл сс Main с использов нием Maven

Перед тем как применять класс Main, вам потребуется запустить Derby. Самый легкий способ сделать это — открыть каталог $DERBY_HOME/bin и выполнить сценарий startNetworkServer. Derby запустится и выведет в консоли следующие сообщения:

```
Security manager installed using the Basic server security policy.
Apache Derby Network Server - 10.9.1.0 - (802917) started and ready to accept
connections on port 1527
```

Процесс Derby прослушивает порт 1527 и ждет, когда JDBC-драйвер отправит какой-нибудь SQL-оператор. Для применения класса Main вы можете воспользоваться командой интерпретатора java либо прибегнуть к exec-maven-plugin, как показано далее:

```
$ mvn exec:java
```

В результате применения класса Main произойдет несколько вещей. Прежде всего Derby автоматически создаст базу данных chapter04DB, как только будет инициализирована сущность Book. Это случится потому, что в файле persistence.xml вы добавили свойство create=true в URL-адрес JDBC:

```
<property name="javax.persistence.jdbc.url" ➥
          value="jdbc:derby://localhost:1527/chapter04DB;create=true"/>
```

Это сокращение очень полезно, когда вы находитесь в режиме разработки, поскольку вам не требуются никакие SQL-сценарии для создания базы данных. Затем свойство javax.persistence.schema-generation-action проинформирует EclipseLink о необходимости автоматически удалить и заново создать таблицу BOOK. И наконец, сущность Book будет вставлена в таблицу (с автоматически сгенерированным идентификатором).

Воспользуемся Derby-командами для вывода на экран структуры таблицы: введем команду ij в консоли (каталог $DERBY_HOME/bin должен быть в вашей переменной PATH). Это приведет к запуску интерпретатора Derby, и вы сможете выполнить команды для подключения к базе данных, вывести на экран таблицы базы данных chapter04DB (show tables), проверить структуру таблицы BOOK (describebook) и даже увидеть ее содержимое, введя SQL-операторы, например SELECT * FROM BOOK.

```
$ ij
version 10.9.1.0
```

```
ij> connect 'jdbc:derby://localhost:1527/chapter04DB';
ij> show tables;
TABLE_SCHEM     |TABLE_NAME     |REMARKS
------------------------------------------------------------------------
APP             |BOOK           |
APP             |SEQUENCE       |
ij> describe book;
COLUMN_NAME   |TYPE_NAME|DEC&|NUM&|COLUM&|COLUMN_DEF|CHAR_OCTE&|IS_NULL&
------------------------------------------------------------------------
ID            |BIGINT   |0   |10  |19    |NULL      |NULL      |NO
TITLE         |VARCHAR  |NULL|NULL|255   |NULL      |510       |YES
PRICE         |DOUBLE   |NULL|2   |52    |NULL      |NULL      |YES
ILLUSTRATIONS |SMALLINT |0   |10  |5     |0         |NULL      |YES
DESCRIPTION   |VARCHAR  |NULL|NULL|255   |NULL      |510       |YES
ISBN          |VARCHAR  |NULL|NULL|255   |NULL      |510       |YES
NBOFPAGE      |INTEGER  |0   |10  |10    |NULL      |NULL      |YES
```

Возвращаясь к коду сущности `Book` (см. листинг 4.7), отмечу, что, поскольку вы использовали аннотацию `@GeneratedValue` (для автоматического генерирования идентификатора), EclipseLink создал таблицу последовательности для сохранения нумерации (таблица `SEQUENCE`). В случае со структурой таблицы `BOOK` JPA придерживался определенных соглашений по умолчанию, присваивая имена таблице и столбцам в соответствии с именем сущности и атрибутов (например, `String` отображается в `VARCHAR(255)`).

Проверк сгенериров нной схемы

В файле `persistence.xml`, описанном в листинге 4.10, мы проинформировали EclipseLink о необходимости сгенерировать схему базы данных и сценарии для удаления и создания с помощью следующего свойства:

```
<property name="javax.persistence.schema-generation.database.action" ➥
          value="drop-and-create"/>
<property name="javax.persistence.schema-generation.scripts.action" ➥
          value="drop-and-create"/>
```

По умолчанию поставщик постоянства сгенерирует два SQL-сценария: `createDDL.jdbc` (листинг 4.13) со всеми SQL-операторами для создания всей базы данных целиком и `dropDDL.jdbc` (листинг 4.14) для удаления всех таблиц. Это удобно, когда вам необходимо выполнить сценарии с целью создания базы данных в своем процессе непрерывной интеграции.

Листинг 4.13. Сцен рий createDDL.jdbc

```
CREATE TABLE BOOK (ID BIGINT NOT NULL, DESCRIPTION VARCHAR(255), ➥
                   ILLUSTRATIONS SMALLINT DEFAULT 0, ISBN VARCHAR(255), ➥
NBOFPAGE INTEGER, PRICE FLOAT, TITLE VARCHAR(255), PRIMARY KEY (ID))
CREATE TABLE SEQUENCE (SEQ_NAME VARCHAR(50) NOT NULL, SEQ_COUNT DECIMAL(15), ➥
                       PRIMARY KEY (SEQ_NAME))
INSERT INTO SEQUENCE (SEQ_NAME, SEQ_COUNT) values ('SEQ_GEN', 0)
```

Листинг 4.14. Сцен рий dropDDL.jdbc

```
DROP TABLE BOOK
DELETE FROM SEQUENCE WHERE SEQ_NAME = 'SEQ_GEN'
```

Резюме

В данной главе приведен беглый обзор JPA 2.1. Как и большинство других спецификаций Java EE 7, JPA сосредоточен на простой объектной архитектуре, оставляя позади своего предка — тяжеловесную компонентную модель (также известную как EJB CMP 2.*x*). В этой главе также рассмотрены сущности, которые представляют собой постоянные объекты, отображающие метаданные с помощью аннотаций или XML.

Благодаря разделу «Все вместе» вы увидели, как следует выполнять JPA-приложения с использованием EclipseLink и Derby. Интеграционное тестирование — важная задача в проектах, а с помощью JPA и баз данных в оперативной памяти вроде Derby сейчас стало очень легко проводить тесты касаемо постоянства.

Из последующих глав вы больше узнаете об основных компонентах JPA. В главе 5 вы увидите, как отображать сущности, связи и наследование в базу данных. А в главе 6 внимание будет сосредоточено на API менеджера сущностей, синтаксисе JPQL. Вы узнаете о том, как использовать запросы и механизмы блокировки, а также будет объяснен жизненный цикл сущностей и то, как добавлять бизнес-логику, когда речь идет о методах сущностей и слушателях.

Гл в 5

Объектно-реляционное отобр жение

В предыдущей главе я подробно рассказал вам об основах объектно-реляционного отображения (Object-Relational Mapping — ORM), которое по сути является отображением сущностей в таблицах и атрибутов в столбцах. Я также поведал вам о конфигурации в порядке исключения, которая позволяет поставщику JPA отображать сущность в таблицу базы данных с использованием всех правил по умолчанию. Однако такие правила не всегда оказываются подходящими, особенно если вы отображаете свою доменную модель в существующую базу данных. JPA сопутствует богатый набор метаданных, благодаря чему вы можете настраивать отображение.

В этой главе я рассмотрю элементарное отображение, а также сконцентрируюсь на более сложных отображениях, таких как отображение связей, композиции и наследования. Доменная модель состоит из объектов, которые взаимодействуют друг с другом. У объектов и баз данных имеются разные способы сохранения информации о связях (ссылок в объектах и внешних ключей в базах данных). Наследование не является чертой, которая от природы имеется у реляционных баз данных, и, следовательно, отображение не столь очевидно. В этой главе я тщательно разберу некоторые подробности и приведу примеры, демонстрирующие то, как следует отображать атрибуты, связи и наследования из доменной модели в базе данных.

Элемент рное отобр жение

Способ, которым Java обрабатывает данные, значительно отличается, если сравнивать его с подходом к обработке информации, используемым реляционными базами данных. В случае с Java мы применяем классы для описания как атрибутов для размещения данных, так и методов для доступа и манипулирования данными. Определив класс, мы можем создать столько его экземпляров, сколько нам потребуется, указав ключевое слово `new`. В реляционной базе данных информация хранится в структурах (столбцах и строках), не являющихся объектными, а динамическое поведение представляет собой хранимую функциональность вроде триггеров таблиц и хранимых процедур, которые не связаны тесно со структурами данных так, как с объектами. Иногда отображение Java-объектов в основной базе данных может быть легким, при этом могут применяться правила по умолчанию.

А иной раз эти правила не отвечают вашим нуждам и приходится настраивать отображение. Аннотации элементарного отображения сосредоточены на настройке требуемой таблицы, первичного ключа и столбцов и позволяют вам модифицировать определенные соглашения об именовании или типизацию (речь может идти о не имеющем значения `null` столбце, длине и т. д.).

T блицы

Согласно правилам отображения с использованием подхода «конфигурация в порядке исключения» имена сущности и таблицы должны совпадать (сущность `Book` будет отображаться в таблице `BOOK`, сущность `AncientBook` — в таблице `ANCIENTBOOK` и т. д.). Это, возможно, будет устраивать вас в большинстве ситуаций, однако вам может потребоваться отобразить свои данные в другой таблице или даже отобразить одну сущность в нескольких таблицах.

@Table

Аннотация `@javax.persistence.Table` дает возможность изменять значения по умолчанию, связанные с определенной таблицей. Например, вы можете указать имя таблицы, в которой будут храниться данные, каталог и схему базы данных. Вы также можете определить уникальные ограничения для таблицы с помощью аннотации `@UniqueConstraint` в сочетании с `@Table`. Если пренебречь аннотацией `@Table`, то имя таблицы будет соответствовать имени сущности. Если вы захотите изменить имя с `BOOK` на `T_BOOK`, то поступите так, как показано в листинге 5.1.

Листинг 5.1. Сущность Book, отобр ж ем я в т блице T_BOOK

```
@Entity
@Table(name = "t_book")
public class Book {

@Id
  private Long id;
  private String title;
  private Float price;
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;

  // Конструкторы, геттеры, сеттеры
}
```

ПРИМЕЧАНИЕ

Я включил в аннотацию @Table имя таблицы в нижнем регистре (t_book). По умолчанию большинство баз данных будут отображать имя сущности в имя таблицы в верхнем регистре (именно так дело обстоит в случае с Derby), если только вы не сконфигурируете их на соблюдение регистра.

@SecondaryTable

До сих пор я исходил из того, что сущность будет отображаться в одну таблицу, также известную как *первичная таблица*. Но если у вас есть уже существующая модель данных, необходимо разбросать данные по нескольким таблицам, или *вторичным таблицам*. Для этого вам потребуется прибегнуть к аннотации `@SecondaryTable`, чтобы ассоциировать вторичную таблицу с сущностью, или `@SecondaryTables` (с буквой s на конце) в случае с несколькими вторичными таблицами. Вы можете распределить данные требуемой сущности по столбцам как первичной таблицы, так и вторичных таблиц, просто определив вторичные таблицы с использованием аннотаций, а затем указав для каждого атрибута то, к какой таблице он относится (с помощью аннотации `@Column`, более подробное описание которой я приведу в подразделе «@Column»). В листинге 5.2 показано отображение атрибутов сущности `Address` в одну первичную таблицу и две вторичные таблицы.

Листинг 5.2. Атрибуты сущности Address, отобр жемые в три р зные т блицы

```
@Entity
@SecondaryTables({
  @SecondaryTable(name = "city"),
 @SecondaryTable(name = "country")
})
public class Address {

  @Id
  private Long id;
  private String street1;
  private String street2;
  @Column(table = "city")
  private String city;
  @Column(table = "city")
  private String state;
  @Column(table = "city")
  private String zipcode;
  @Column(table = "country")
  private String country;

  // Конструкторы, геттеры, сеттеры
}
```

По умолчанию атрибуты сущности `Address` будут отображаться в первичную таблицу (которая по умолчанию имеет имя сущности, поэтому называется `ADDRESS`). Аннотация `@SecondaryTables` проинформирует вас о том, что в данном случае есть две вторичные таблицы: `CITY` и `COUNTRY`. Затем вам потребуется указать, какой атрибут в какой таблице будет располагаться (с использованием аннотации `@Column(table="city")` или `@Column(table="country")`). На рис. 5.1 показана структура таблиц, в которых будет отображаться сущность `Address`. Каждая таблица содержит разные атрибуты, однако у всех имеется один и тот же первичный ключ (для соединения таблиц). Опять-таки не забывайте, что Derby преобразует имена таблиц в нижнем регистре (`city`) в имена таблиц в верхнем регистре (`CITY`).

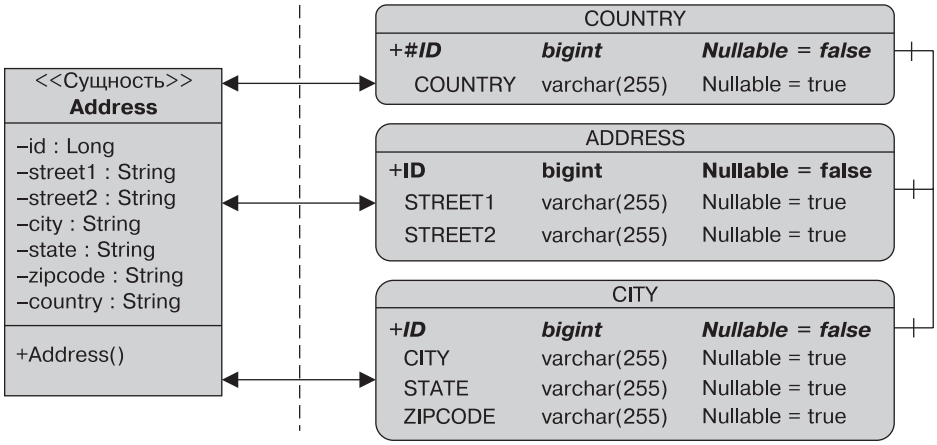


Рис. 5.1. Сущность Address отобр жется в трех т блиц х

Как вы, вероятно, уже поняли, для одной и той же сущности может быть несколько аннотаций. Если вы захотите переименовать первичную таблицу, то можете добавить аннотацию `@Table`, как показано в листинге 5.3.

Листинг 5.3. Первичн я т блиц переименовыв ется в T_ADDRESS

```
@Entity
@Table(name = "t_address")
@SecondaryTables({
  @SecondaryTable(name = "t_city"),
  @SecondaryTable(name = "t_country")
})
public class Address {

  // Атрибуты, конструктор, геттеры, сеттеры
}
```

ПРИМЕЧАНИЕ

При использовании вторичных таблиц вы должны принимать во внимание производительность. Каждый раз, когда вы получаете доступ к сущности, поставщик постоянства обращается к нескольким таблицам, которые ему приходится соединять. С другой стороны, вторичные таблицы могут оказаться кстати, когда у вас имеются «затратные» атрибуты вроде больших двоичных объектов (Binary Large Object — BLOB), которые вы хотите изолировать в другой таблице.

Первичные ключи

Первичные ключи в реляционных базах данных уникально идентифицируют все строки таблиц. Первичный ключ охватывает либо один столбец, либо набор столбцов. Такой ключ должен быть уникальным, поскольку он идентифицирует одну строку (значение `null` не допускается). Примерами первичных ключей являются идентификатор клиента, телефонный номер, номер получения, а также ISBN-номер. JPA требует, чтобы у сущностей был идентификатор, отображаемый в пер-

вичный ключ, который будет придерживаться аналогичного правила: уникально идентифицировать сущность с помощью либо одного атрибута, либо набора атрибутов (составной ключ). Значение этого первичного ключа сущности нельзя обновить после того, как оно было присвоено.

@Id и @GeneratedValue

Простой (то есть не являющийся составным) первичный ключ должен соответствовать одному атрибуту класса-сущности. Аннотация `@Id`, которую вы видели ранее, используется для обозначения простого первичного ключа. `@javax.persistence.Id` аннотирует атрибут как уникальный идентификатор. Он может относиться к одному из таких типов, как:

- примитивные Java-типы: `byte`, `int`, `short`, `long`, `char`;
- классы-обертки примитивных Java-типов: `Byte`, `Integer`, `Short`, `Long`, `Character`;
- массивы примитивных типов или классов-адаптеров: `int[]`, `Integer[]` и т. д.;
- строки, числа и даты: `java.lang.String`, `java.math.BigInteger`, `java.util.Date`, `java.sql.Date`.

При создании сущности значение этого идентификатора может быть сгенерировано либо вручную с помощью приложения, либо автоматически поставщиком постоянства с использованием аннотации `@GeneratedValue`. Эта аннотация способна иметь четыре возможных значения:

- `SEQUENCE` и `IDENTITY` определяют использование SQL-последовательности базы данных или столбца идентификаторов соответственно;
- `TABLE` дает указание поставщику постоянства сохранить имя последовательности и ее текущее значение в таблице, увеличивая это значение каждый раз, когда будет обеспечиваться постоянство нового экземпляра сущности. Например, EclipseLink создаст таблицу `SEQUENCE` с двумя столбцами: в одном будет имя последовательности (произвольное), а в другом — значение последовательности (целое число, автоматически инкрементируемое Derby);
- генерирование ключа выполняется автоматически (`AUTO`) основным поставщиком постоянства, который выберет подходящую стратегию для определенной базы данных (EclipseLink будет использовать стратегию `TABLE`). `AUTO` является значением по умолчанию аннотации `@GeneratedValue`.

Если аннотация `@GeneratedValue` не будет определена, приложению придется создать собственный идентификатор, применив любой алгоритм, который возвратит уникальное значение. В коде, приведенном в листинге 5.4, показано, как получить автоматически генерируемый идентификатор. Будучи значением по умолчанию, `GenerationType.AUTO` позволило бы мне не включать в этот код элемент `strategy`. Обратите внимание, что атрибут `id` аннотирован дважды: один раз с использованием `@Id` и еще один раз — посредством `@GeneratedValue`.

Листинг 5.4. Сущность Book с втом тически генерируемым идентифик тором

```
@Entity
public class Book {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String title;
    private Float price;
    private String description;
    private String isbn;
    private Integer nbOfPage;
    private Boolean illustrations;

    // Конструкторы, геттеры, сеттеры
}
```

Сост вные первичные ключи

При отображении сущностей правильным подходом будет обозначить один специально отведенный для этого столбец как первичный ключ. Однако бывают ситуации, когда требуется составной первичный ключ (скажем, если приходится выполнять отображение в унаследованную базу данных или первичные ключи должны придерживаться определенных бизнес-правил — например, необходимо включить значение или код страны и отметку времени). Для этого должен быть определен класс первичного ключа, который будет представлять составной ключ. Кроме того, у нас есть две доступные аннотации для определения этого класса в зависимости от того, как мы хотим структурировать сущность: `@EmbeddedId` и `@IdClass`. Как вы еще увидите, конечный результат будет одинаковым и в итоге у вас окажется одна и та же схема базы данных, однако способы выполнения запросов к сущности будут немного различаться.

Например, приложению CD-BookStore необходимо часто выкладывать информацию на главной странице, где вы сможете читать ежедневные новости о книгах, музыке или артистах. У новостей будет иметься содержимое, название и, поскольку они могут быть написаны на нескольких языках, код языка (`EN` для английского, `PT` — для португальского и т. д.). Первичным ключом для новостей тогда сможет быть название и код языка, поскольку статья может быть переведена на разные языки, но с сохранением ее оригинального названия. Таким образом, класс первичного ключа `NewsId` будет включать два атрибута, имеющие тип `String`: `title` и `language`. Классы первичных ключей должны включать определения методов для `equals()` и `hashCode()` для управления запросами и внутренними коллекциями (равенство в случае с этими методами должно быть таким же, как и равенство, которое имеет место в случае с базой данных), а их атрибуты должны быть допустимых типов, входящих в приведенный ранее набор. Они также должны быть открытыми, реализовывать интерфейс `Serializable`, если им потребуется пересекать архитектурные уровни (например, управление ими будет осуществляться на постоянном уровне, а использование — на уровне представления), и располагать конструктором без аргументов.

@EmbeddedId. Как вы еще увидите позднее в этой главе, JPA использует встроенные объекты разных типов. Резюмируя, отмечу, что у встроенного объекта нет какого-либо идентификатора (собственного первичного ключа), а его атрибуты в итоге окажутся столбцами таблицы содержащей их сущности.

В листинге 5.5 класс NewsId показан как встраиваемый. Он представляет собой всего лишь встроенный объект (аннотированный с использованием @Embeddable), который в данном случае включает два атрибута (title и language). У этого класса должен быть конструктор без аргументов, геттер, сеттер, а также реализации equals() и hashCode(). Это означает, что он должен придерживаться соглашений JavaBeans. У этого класса как такового нет собственного идентификатора (отсутствует аннотация @Id). Это характерно для встраиваемых классов.

Листинг 5.5. Кл сс первичного ключ сн бж ется ннот цией @Embeddable

```
@Embeddable
public class NewsId {

  private String title;
  private String language;

  // Конструкторы, геттеры, сеттеры
}
```

В случае с сущностью News, показанной в листинге 5.6, затем придется встроить класс первичного ключа NewsId с применением аннотации @EmbeddedId. Благодаря такому подходу не потребуется использовать @Id. Каждая аннотация @EmbeddedId должна ссылаться на встраиваемый класс, помеченный @Embeddable.

Листинг 5.6. Вложение кл сс первичного ключ с применением ннот ции @EmbeddedId для сущности

```
@Entity
public class News {

  @EmbeddedId
  private NewsId id;
  private String content;

  // Конструкторы, геттеры, сеттеры
}
```

В следующей главе я опишу, как находить сущности с использованием их первичного ключа. Вот первый, но мимолетный взгляд на то, как это происходит: первичный ключ — это класс с конструктором. Вам придется создать экземпляр этого класса со значениями, формирующими ваш уникальный ключ, и передать соответствующий объект менеджера сущностей (атрибут em), как показано в этом коде:

```
NewsId pk = new NewsId("Richard Wright has died on September 2008", "EN")
News news = em.find(News.class, pk);
```

@IdClass. Другой метод объявления составного ключа — использование аннотации @IdClass. Это иной подход, в соответствии с которым каждый атрибут класса первичного ключа также должен быть объявлен в классе-сущности и аннотирован с помощью @Id.

Составной первичный ключ в примере NewsId, приведенном в листинге 5.7, является всего лишь POJO, которому не требуется какая-либо аннотация (в предыдущем

примере в листинге 5.6 класс первичного ключа приходилось аннотировать с помощью @EmbeddedId).

Листинг 5.7. У кл сс первичного ключ ннот ция отсутствует

```
public class NewsId {

private String title;
  private String language;

  // Конструкторы, геттеры, сеттеры
}
```

Затем для сущности News, показанной в листинге 5.8, придется определить первичный ключ с использованием аннотации @IdClass, и аннотировать каждый ключ посредством @Id. Для обеспечения постоянства сущности News вам потребуется задать значения для атрибутов title и language.

Листинг 5.8. Определение первичного кл сс с использов нием ннот ции @IdClass для сущности

```
@Entity
@IdClass(NewsId.class)
public class News {

  @Id private String title;
  @Id private String language;
  private String content;

  // Конструкторы, геттеры, сеттеры
}
```

Оба варианта — @EmbeddedId и @IdClass — будут отображены в одну и ту же табличную структуру. Она определена в листинге 5.9 с использованием языка описания данных (Data Definition Language — DDL). Атрибуты сущности и первичный ключ в итоге окажутся в одной и той же таблице, а первичный ключ будет сформирован с использованием атрибутов составного класса (title и language).

Листинг 5.9. DDL-код т блицы NEWS с сост вным первичным ключом

```
create table NEWS (
    CONTENT VARCHAR(255),
    TITLE VARCHAR(255) not null,
    LANGUAGE VARCHAR(255) not null,
    primary key (TITLE, LANGUAGE)
);
```

Подход с использованием @IdClass более предрасположен к ошибкам, поскольку вам потребуется определить каждый атрибут первичного ключа как в @IdClass, так и в сущности, позаботившись об использовании одинакового имени и Java-типа. Преимущество заключается в том, что вам не придется изменять код класса первичного ключа (никаких аннотаций не потребуется). Например, вы могли бы использовать унаследованный класс, который по причинам правового характера вам

не разрешено изменять, однако при этом вам предоставляется возможность его повторного использования.

Одно из видимых отличий заключается в способе, которым вы ссылаетесь на сущность при использовании JPQL. В случае с `@IdClass` вы сделали бы что-то вроде следующего:

```
select n.title from News n
```

А в случае с `@EmbeddedId` у вас получилось бы что-то вроде показанного далее:

```
select n.newsId.title from News n
```

Атрибуты

У сущности должен иметься первичный ключ (простой или составной), чтобы у нее был идентификатор в реляционной базе данных. Она также обладает всевозможными атрибутами, которые обуславливают ее состояние и должны быть отображены в таблицу. Это состояние способно включать почти любой Java-тип, который вам может потребоваться отобразить:

- примитивные Java-типы и классы-обертки (`int`, `double`, `float` и т. д.) (`Integer`, `Double`, `Float` и т. д.);
- массивы байтов и символов (`byte[]`, `Byte[]`, `char[]`, `Character[]`);
- строковые, связанные с большими числами, и временные типы (`java.lang.String`, `java.math.BigInteger`, `java.math.BigDecimal`, `java.util.Date`, `java.util.Calendar`, `java.sql.Date`, `java.sql.Time`, `java.sql.Timestamp`);
- перечислимые типы, а также определяемые пользователем типы, которые реализуют интерфейс `Serializable`;
- коллекции базовых и встраиваемых типов.

Разумеется, у сущности также могут иметься атрибуты сущности, коллекции сущностей или встраиваемые классы. Это требует рассказа о связях между сущностями (которые будут рассмотрены в разделе «Отображение связей»).

Как вы уже видели ранее, при конфигурации в порядке исключения атрибуты отображаются с использованием правил отображения по умолчанию. Однако иногда возникает необходимость настроить детали этого отображения. Именно здесь в дело вступают аннотации JPA (или их XML-эквиваленты).

@Basic

Опциональная аннотация `@javax.persistence.Basic` (листинг 5.10) является отображением в столбец базы данных, относящимся к самому простому типу, поскольку она переопределяет базовое постоянство.

Листинг 5.10. Элементы ннот ции @Basic

```
@Target({METHOD, FIELD}) @Retention(RUNTIME)
public @interface Basic {
  FetchType fetch() default EAGER;
  boolean optional() default true;
}
```

У этой аннотации есть два параметра: optional и fetch. Элемент optional подсказывает вам, может ли null быть значением атрибута. Однако он игнорируется для примитивных типов. Элемент fetch может принимать два значения: LAZY или EAGER. Он намекает на то, что во время выполнения поставщика постоянства выборка данных должна быть отложенной (только когда приложение запрашивает соответствующее свойство) или быстрой (когда сущность изначально загружается поставщиком).

Возьмем, к примеру, сущность Track, показанную в листинге 5.11. В CD-альбом входит несколько треков, имеющих название, описание и WAV-файл определенной длительности, который вы можете прослушать. WAV-файл представляет собой BLOB-объект, который может иметь объем несколько мегабайт. Вам не нужно, чтобы при доступе к сущности Track WAV-файл тут же быстро загружался; вы можете снабдить атрибут аннотацией @Basic(fetch = FetchType.LAZY), и извлечение информации из базы данных будет отложенным (например, только при доступе к атрибуту wav с использованием его геттера).

Листинг 5.11. Сущность Entity с отложенной з грузкой для трибут wav

```
@Entity
public class Track {

  @Id @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  private String title;
  private Float duration;
  @Basic(fetch = FetchType.LAZY)
  @Lob
  private byte[] wav;
  private String description;

  // Конструкторы, геттеры, сеттеры
}
```

Обратите внимание, что атрибут wav типа byte[] также аннотирован с помощью @Lob для сохранения значения как большого объекта (Large Object — LOB). Столбцы базы данных, в которых могут храниться большие объекты такого типа, требуют специальных JDBC-вызовов для доступа к ним из Java-кода. Для информирования поставщика в случае с базовым отображением должна быть добавлена опциональная аннотация @Lob.

@Column

Аннотация @javax.persistence.Column, показанная в листинге 5.12, определяет свойства столбца. Вы можете изменить имя столбца (которое по умолчанию отображается в совпадающее с ним имя атрибута), а также указать размер и разрешить (или запретить) столбцу иметь значение null, быть уникальным или позволить его значению быть обновляемым или вставляемым. В листинге 5.12 приведен API-интерфейс аннотации @Column с элементами и их значениями по умолчанию.

Листинг 5.12. Элементы ннот ции @Column

```
@Target({METHOD, FIELD}) @Retention(RUNTIME)
public @interface Column {
```

```
  String name() default "";
  boolean unique() default false;
  boolean nullable() default true;
  boolean insertable() default true;
  boolean updatable() default true;
  String columnDefinition() default "";
  String table() default "";
  int length() default 255;
  int precision() default 0; // десятичная точность
  int scale() default 0;     // десятичная система счисления
}
```

Для переопределения отображения по умолчанию оригинальной сущности `Book` (см. листинг 5.1) вы можете использовать аннотацию `@Column` разными способами (листинг 5.13). Например, вы можете изменить имя столбца `title` и `nbOfPage` либо длину `description` и не разрешить значения `null`. Следует отметить, что допустимы не все комбинации атрибутов для типов данных (например, `length` применим только к столбцу со строковым значением, а `scale` и `precision` — только к десятичному столбцу).

Листинг 5.13. Н стройк отобр жения сущности Book

```
@Entity
public class Book {
  @Id @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  @Column(name = "book_title", nullable = false, updatable = false)
  private String title;
  private Float price;
  @Column(length = 2000)
  private String description;
  private String isbn;
  @Column(name = "nb_of_page", nullable = false)
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Сущность `Book` из листинга 5.13 будет отображена в определение таблицы, показанное в листинге 5.14.

Листинг 5.14. Определение т блицы BOOK

```
create table BOOK (
  ID BIGINT not null,
  BOOK_TITLE VARCHAR(255) not null,
  PRICE DOUBLE(52, 0),
  DESCRIPTION VARCHAR(2000),
  ISBN VARCHAR(255),
  NB_OF_PAGE INTEGER not null,
  ILLUSTRATIONS SMALLINT,
  primary key (ID)
);
```

Большинство элементов аннотации `@Column` влияют на отображение. Если вы измените значение длины `description` на `2000`, то значение длины целевого столбца тоже будет задано как равное `2000`. Параметры `updatable` и `insertable` по умолчанию имеют значение `true`, которое подразумевает, что любой атрибут может быть вставлен или обновлен в базе данных, а также оказывают влияние во время выполнения. Вы сможете задать для них значение `false`, когда вам понадобится, чтобы поставщик постоянства позаботился о том, что он не вставит или не обновит данные в таблице в ответ на изменения в сущности. Следует отметить, что это не подразумевает, что атрибут сущности не изменится в памяти. Вы по-прежнему сможете изменить соответствующее значение, однако оно не будет синхронизировано с базой данных. Причина этого состоит в том, что сгенерированный SQL-оператор (`INSERT` или `UPDATE`) не будет включать столбцы. Другими словами, эти элементы не влияют на реляционное отображение, но влияют на динамическое поведение менеджера сущностей при доступе к реляционным данным.

ПРИМЕЧАНИЕ

Как можно было видеть в главе 3, Bean Validation определяет ограничения только в рамках пространства Java. Поэтому @NotNull обеспечивает считывание фрагмента Java-кода, который убеждается в том, что значением атрибута не является null. С другой стороны, аннотация JPA @Column(nullable = false) используется в пространстве базы данных для генерирования схемы базы данных. Аннотации JPA и Bean Validation могут сосуществовать в случае с тем или иным атрибутом.

@Temporal

При использовании Java вы можете задействовать `java.util.Date` и `java.util.Calendar`, чтобы сохранить данные и, кроме того, получить в свое распоряжение несколько их представлений, например, в виде даты, часа или миллисекунд. Чтобы указать это при объектно-реляционном отображении, вы можете воспользоваться аннотацией `@javax.persistence.Temporal`. Она принимает три возможных значения: `DATE`, `TIME` или `TIMESTAMP` (например, текущая дата, только время или и то и другое). В листинге 5.15 определена сущность `Customer`, у которой имеется атрибут `dateOfBirth`, а также технический атрибут, обеспечивающий сохранение значения точного времени ее создания в системе (в данной ситуации используется значение `TIMESTAMP`).

Листинг 5.15. Сущность Customer с двумя трибут ми @Temporal

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Temporal(TemporalType.DATE)
  private Date dateOfBirth;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  // Конструкторы, геттеры, сеттеры
}
```

Сущность Customer из листинга 5.15 будет отображена в таблице, определенной в листинге 5.16. Атрибут dateOfBirth будет отображен в столбце типа DATE, а creationDate — в столбце типа TIMESTAMP.

Листинг 5.16. Определение т блицы CUSTOMER

```
create table CUSTOMER (
  ID BIGINT not null,
  FIRSTNAME VARCHAR(255),
  LASTNAME VARCHAR(255),
  EMAIL VARCHAR(255),
  PHONENUMBER VARCHAR(255),
  DATEOFBIRTH DATE,
  CREATIONDATE TIMESTAMP,
  primary key (ID)
);
```

@Transient

При использовании JPA, как только класс окажется снабжен аннотацией @Entity, все его атрибуты будут автоматически отображены в таблице. Если вам не нужно, чтобы атрибут отображался, вы можете воспользоваться аннотацией @javax.persistence.Transient или ключевым Java-словом transient. К примеру, возьмем упоминавшуюся выше сущность Customer и добавим атрибут age (листинг 5.17). Поскольку возраст может быть автоматически вычислен исходя из даты рождения, атрибут age не нужно отображать и, следовательно, он может быть временным.

Листинг 5.17. Сущность Customer с временным трибутом age

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Temporal(TemporalType.DATE)
  private Date dateOfBirth;
  @Transient
  private Integer age;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  // Конструкторы, геттеры, сеттеры
}
```

В результате атрибут age не нужно отображать в каком-либо столбце AGE.

@Enumerated

В Java SE 5 были представлены перечислимые типы, которые теперь настолько часто используются, что обычно становятся частью жизни разработчиков. Значения перечислимых типов представляют собой константы, которым неявно присваиваются порядковые номера согласно той последовательности, где они объявляются.

Такой порядковый номер нельзя модифицировать во время выполнения, однако его можно использовать для сохранения значения перечислимого типа в базе данных. В листинге 5.18 показано перечисление CreditCardType.

Листинг 5.18. Перечисление CreditCardType

```
public enum CreditCardType {
  VISA,
  MASTER_CARD,
  AMERICAN_EXPRESS
}
```

Порядковыми номерами, присвоенными значениям этого перечислимого типа во время компиляции, являются 0 для VISA, 1 для MASTER_CARD и 2 для AMERICAN_EXPRESS. По умолчанию поставщики постоянства будут отображать этот перечислимый тип в базе данных, предполагая, что соответствующий столбец имеет тип Integer. Взглянув на код, приведенный в листинге 5.19, вы увидите сущность CreditCard, которая задействует предыдущее перечисление при отображении по умолчанию.

Листинг 5.19. Отобр жение перечислимого тип с порядковыми номер ми

```
@Entity
@Table(name = "credit_card")
public class CreditCard {
  @Id
  private String number;
  private String expiryDate;
  private Integer controlNumber;
  private CreditCardType creditCardType;
  // Конструкторы, геттеры, сеттеры
}
```

Поскольку применяются правила по умолчанию, перечисление отобразится в целочисленном столбце, и все будет отлично. Но представим, что в верхушку перечисления добавлена новая константа. Поскольку присваивание порядковых номеров осуществляется согласно последовательности, в которой значения объявляются, значения, уже хранящиеся в базе данных, больше не будут соответствовать перечислению. Лучшим решением стало бы сохранение имени значения как строки вместо сохранения порядкового номера. Это можно сделать, добавив аннотацию @Enumerated к атрибуту и указав значение STRING (значением по умолчанию является ORDINAL), как показано в листинге 5.20.

Листинг 5.20. Отобр жение перечислимого тип с использов нием STRING

```
@Entity
@Table(name = "credit_card")
public class CreditCard {
  @Id
  private String number;
  private String expiryDate;
  private Integer controlNumber;
  @Enumerated(EnumType.STRING)
  private CreditCardType creditCardType;
```

```
  // Конструкторы, геттеры, сеттеры
}
```

Теперь столбец базы `CreditCardType` данных будет иметь тип `VARCHAR`, а информация о карточке Visa будет сохранена в строке `"VISA"`.

Тип доступ

До сих пор я показывал вам аннотированные классы (`@Entity` или `@Table`) и атрибуты (`@Basic`, `@Column`, `@Temporal` и т. д.), однако аннотации, применяемые к атрибуту (или доступ к полям), также могут быть применены к соответствующему методу-геттеру (или доступ к свойствам). Например, аннотация `@Id` может быть применена к атрибуту `id` или методу `getId()`. Поскольку это в основном вопрос личных предпочтений, я склонен использовать доступ к свойствам (аннотировать геттеры), так как, по моему мнению, код при этом получается более удобочитаемым. Это позволяет быстро изучить атрибуты сущности, не утопая в аннотациях. Чтобы код в этой книге было легко разобрать, я решил аннотировать атрибуты. Однако в некоторых случаях (например, когда речь идет о наследовании) это не просто дело личного вкуса, поскольку оно может повлиять на ваше отображение.

ПРИМЕЧАНИЕ

В Java поле определяется как атрибут экземпляра. Свойство — это любое поле с методами (геттерами или сеттерами) доступа, которые придерживаются шаблона JavaBean (в начале идет getXXX, setXXX или isXXX в случае с Boolean).

При выборе между доступом к полям (атрибуты) или доступом к свойствам (геттеры) необходимо определить *тип доступа*. По умолчанию для сущности применяется один тип доступа: это либо доступ к полям, либо доступ к свойствам, но не оба сразу (например, поставщик постоянства получает доступ к постоянному состоянию либо посредством атрибутов, либо посредством методов-геттеров). Согласно спецификации поведение приложения, в котором сочетается применение аннотаций к полям и свойствам без указания типа доступа явным образом, является неопределенным. При использовании доступа к полям (листинг 5.21) поставщик постоянства отображает атрибуты.

Листинг 5.21. Сущность Customer с ннотиров нными полями

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  @Column(name = "first_name", nullable = false, length = 50)
  private String firstName;
  @Column(name = "last_name", nullable = false, length = 50)
  private String lastName;
  private String email;
  @Column(name = "phone_number", length = 15)
  private String phoneNumber;
  // Конструкторы, геттеры, сеттеры
}
```

При использовании доступа к свойствам, как показано в листинге 5.22, отображение базируется на геттерах, а не на атрибутах. Все геттеры, не снабженные аннотацией @Transient, являются постоянными.

Листинг 5.22. Сущность Customer с ннотиров нными свойств ми

```
@Entity
public class Customer {
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  // Конструкторы
  @Id @GeneratedValue
  public Long getId() {
    return id;
  }
  public void setId(Long id) {
    this.id = id;
  }
  @Column(name = "first_name", nullable = false, length = 50)
  public String getFirstName() {
    return firstName;
  }
  public void setFirstName(String firstName) {
    this.firstName = firstName;
  }
  @Column(name = "last_name", nullable = false, length = 50)
  public String getLastName() {
    return lastName;
  }
  public void setLastName(String lastName) {
    this.lastName = lastName;
  }
  public String getEmail() {
    return email;
  }
  public void setEmail(String email) {
    this.email = email;
  }
  @Column(name = "phone_number", length = 15)
  public String getPhoneNumber() {
    return phoneNumber;
  }
  public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
  }
}
```

В плане отображения две сущности из листингов 5.21 и 5.22 полностью идентичны, поскольку имена атрибутов в данном случае совпадают с именами геттеров.

Однако вместо использования типа доступа по умолчанию вы можете явным образом указать тип с помощью аннотации `@javax.persistence.Access`.

Эта аннотация принимает два возможных значения — `FIELD` или `PROPERTY`, а также может быть использована в отношении сущности как таковой и/или каждого атрибута или геттера. Например, при применении `@Access(AccessType.FIELD)` к сущности поставщиком постоянства будут приниматься во внимание только аннотации отображения, которыми снабжены атрибуты. Тогда можно будет выборочно обозначить отдельные геттеры для доступа к свойствам посредством `@Access(AccessType.PROPERTY)`.

Типы явного доступа могут быть очень полезны (например, при работе со встраиваемыми объектами или наследованием), однако их смешение часто приводит к ошибкам. В листинге 5.23 показан пример того, что может произойти при смешении типов доступа.

Листинг 5.23. Сущность Customer, в случ е с которой явным обр зом смешив ются типы доступ

```
@Entity
@Access(AccessType.FIELD)
public class Customer {
  @Id @GeneratedValue
  private Long id;
  @Column(name = "first_name", nullable = false, length = 50)
  private String firstName;
  @Column(name = "last_name", nullable = false, length = 50)
  private String lastName;
  private String email;
  @Column(name = "phone_number", length = 15)
  private String phoneNumber;
  // Конструкторы, геттеры, сеттеры
  @Access(AccessType.PROPERTY)
  @Column(name = "phone_number", length = 555)
  public String getPhoneNumber() {
    return phoneNumber;
  }
  public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
  }
}
```

В примере, показанном в листинге 5.23, тип доступа явным образом определяется как `FIELD` на уровне сущности. Это говорит интерфейсу `PersistenceManager` о том, что ему следует обрабатывать только аннотации, которыми снабжены атрибуты. Атрибут `phoneNumber` снабжен аннотацией `@Column`, ограничивающей значение его `length` величиной `15`. Читая этот код, вы ожидаете, что в базе данных в итоге будет `VARCHAR(15)`, однако этого не случится. Метод-геттер показывает, что тип доступа для метода `getPhoneNumber()` был изменен явным образом, поэтому длина равна значению `length` атрибута `phoneNumber` (до `555`). В данном случае сущность `AccessType.FIELD` перезаписывается `AccessType.PROPERTY`. Тогда в базе данных у вас окажется `VARCHAR(555)`.

Коллекции б зовых типов

Коллекции тех или иных элементов очень распространены в Java. Из последующих разделов вы узнаете о связях между сущностями (которые могут быть коллекциями сущностей). По сути, это означает, что у одной сущности имеется коллекция других сущностей или встраиваемых объектов. Что касается отображения, то каждая сущность отображается в свою таблицу, при этом создаются ссылки между первичными и внешними ключами. Как вы уже знаете, сущность представляет собой Java-класс с идентификатором и множеством других атрибутов. Но что, если вам потребуется сохранить коллекцию Java-типов, например `String` или `Integer`? С тех пор как вышла версия JPA 2.0, это можно легко сделать без необходимости решать проблему создания отдельного класса. Для этого предназначены аннотации `@ElementCollection` и `@CollectionTable`.

Мы используем аннотацию `@ElementCollection` как индикатор того, что атрибут типа `java.util.Collection` включает коллекцию экземпляров базовых типов (то есть объектов, которые не являются сущностями) или встраиваемых объектов (подробнее на эту тему мы поговорим в разделе «Встраиваемые объекты»). Фактически этот атрибут может иметь один из следующих типов:

- `java.util.Collection` — общий корневой интерфейс в иерархии коллекций;
- `java.util.Set` — коллекция, предотвращающая вставку элементов-дубликатов;
- `java.util.List` — коллекция, которая применяется, когда требуется извлечь элементы в некоем порядке, определяемом пользователем.

Кроме того, аннотация `@CollectionTable` позволяет вам настраивать детали таблицы коллекции (то есть таблицы, которая будет соединять таблицу сущности с таблицей базовых типов), например изменять ее имя. При отсутствии этой аннотации имя таблицы будет конкатенацией имени содержащей сущности и имени атрибута коллекции, разделенных знаком подчеркивания.

Опять-таки, используя сущность-пример `Book`, взглянем на то, как добавить атрибут для сохранения тегов. Сегодня теги и облака тегов распространены повсеместно. Они обычно применяются при сортировке данных, поэтому представим в этом примере, что вы хотите добавить как можно больше тегов сущности `Book` для ее описания и быстрого поиска. Тег — это всего лишь строка, так что у сущности `Book` может быть коллекция строк для сохранения соответствующей информации, как показано в листинге 5.24.

Листинг 5.24. Сущность Book с коллекцией строк

```
@Entity
public class Book {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
```

```
    @ElementCollection(fetch = FetchType.LAZY)
    @CollectionTable(name = "Tag")
    @Column(name = "Value")
    private List<String> tags = new ArrayList<>();
    // Конструкторы, геттеры, сеттеры
}
```

Аннотация `@ElementCollection`, показанная в листинге 5.24, используется для информирования поставщика постоянства о том, что атрибут `tags` представляет собой список строк, а его выборка должна быть отложенной. При отсутствии `@CollectionTable` имя таблицы по умолчанию было бы `BOOK_TAGS` (конкатенация имени содержащей сущности и имени атрибута коллекции, разделенных знаком подчеркивания), а не `TAG`, как указано в элементе `name` (`name = "Tag"`). Обратите внимание, что я добавил дополнительную аннотацию `@Column`, чтобы переименовать столбец в `VALUE` (в противном случае этот столбец получил бы имя, как у атрибута, `TAGS`). Результат можно увидеть на рис. 5.2.

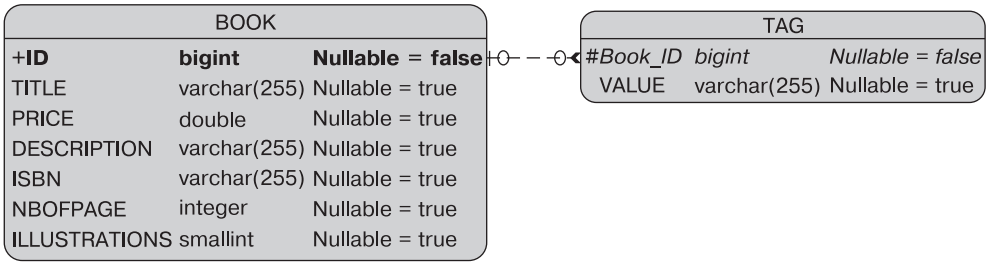


Рис. 5.2. Связь между т блиц ми BOOK и TAG

ПРИМЕЧАНИЕ

В JPA 1.0 этих аннотаций не было. Однако все равно можно было сохранить список примитивных типов, например BLOB, в базе данных. Почему? А потому, что java.util.ArrayList реализует Serializable, а JPA может автоматически отображать сериализуемые объекты в BLOB-объекты. Вместе с тем, если вы взамен используете тип java.util.List, то получите исключение, так как он не расширяет Serializable. Применение @ElementCollection — более элегантный и целесообразный способ сохранения списков базовых типов. Сохранение списков в недоступном двоичном формате не позволит совершать к ним запросы и сделает их непереносимыми в код на других языках (поскольку основной сериализованный объект может быть использован во время выполнения кода, написанного только на Java — не на Ruby, PHP...).

Отобр жение б зовых типов

Как и коллекции, отображения очень полезны при сохранении данных. В JPA 1.0 ключи могли относиться только к базовому типу данных, а значения могли быть только сущностями. Сейчас отображения могут содержать любую комбинацию базовых типов, встраиваемых объектов и сущностей как ключей или значений, что привносит бо́льшую гибкость в процесс отображения. Сосредоточимся на отображениях базовых типов.

Когда отображение задействует базовые типы, аннотации `@ElementCollection` и `@CollectionTable` могут быть использованы тем же путем, который вы видели ранее в случае с коллекциями. Таблица коллекции тогда будет использоваться для хранения данных отображения.

Обратимся к примеру с CD-альбомом, содержащим треки (листинг 5.25). Трек имеет название и позицию (первый трек альбома, второй трек альбома и т. д.). Тогда у вас может быть отображение треков с целочисленным значением, говорящем о позиции определенного трека (ключ отображения), и строкой для указания названия этого трека (значение отображения).

Листинг 5.25. CD- льбом с отобр жением треков

```
@Entity
public class CD {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  @Lob
  private byte[] cover;
  @ElementCollection
  @CollectionTable(name="track")
  @MapKeyColumn (name = "position")
  @Column(name = "title")
  private Map<Integer, String> tracks = new HashMap<>();
  // Конструкторы, геттеры, сеттеры
}
```

Я уже говорил, что аннотация `@ElementCollection` используется как индикатор объектов в отображении, хранящихся в таблице коллекции. Аннотация `@CollectionTable` изменяет имя по умолчанию таблицы коллекции на `TRACK`.

Разница в случае с коллекциями заключается во введении новой аннотации: `@MapKeyColumn`. Она используется для указания отображения ключевого столбца. Если она не будет задана, то столбец получит имя в виде конкатенации имени ссылающегося атрибута связи и `_KEY`. В листинге 5.25 видно, что аннотация переименовала его в `POSITION`, чтобы было яснее, а по умолчанию он назывался бы иначе (`TRACK_KEY`).

Аннотация `@Column` указывает на то, что столбец, содержащий значение отображения, следует переименовать в `TITLE`. Результат можно увидеть на рис. 5.3.

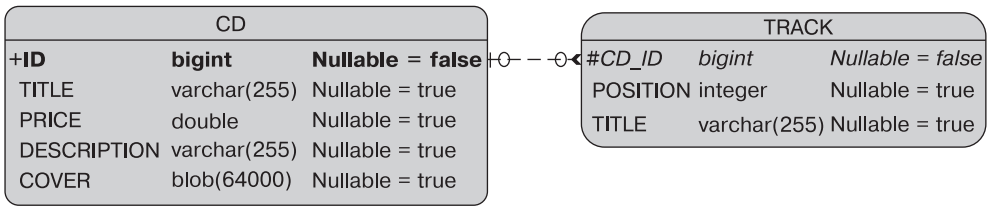


Рис. 5.3. Связь между т блиц ми CD и TRACK

Отобр жение с использов нием XML

Теперь, когда вы ближе познакомились с элементарным отображением с использованием аннотаций, взглянем на XML-отображение. Если вам доводилось применять объектно-реляционный фреймворк вроде ранних версий Hibernate, то вы уже знаете, как отображать свои сущности в отдельном файле XML-дескриптора развертывания. С начала этой главы вы не видели ни одной строки XML-кода, а только аннотации. JPA также предлагает как вариант XML-синтаксис для отображения сущностей. Я не стану вдаваться в подробности XML-отображения, поскольку решил сосредоточиться на аннотациях (так как их легче использовать в книге, а большинство разработчиков выбирает их вместо XML-отображения). Имейте в виду, что у каждой аннотации, которую вы увидите в текущей главе, имеется XML-эквивалент, а этот раздел получился бы огромным, если бы я рассмотрел в нем их все. Я рекомендую вам заглянуть в главу 12 под названием «XML-дескриптор объектно-реляционного отображения» спецификации JPA 2.1, где более детально рассматриваются все XML-теги.

XML-дескрипторы развертывания — альтернатива применению аннотаций. Однако, несмотря на то что у каждой аннотации есть эквивалентный XML-тег и наоборот, разница состоит в том, что XML берет верх над аннотациями. Если вы аннотируете атрибут или сущность с использованием определенного значения и в то же время произведете развертывание XML-дескриптора с использованием другого значения, то преимущество будет за XML.

Вопрос звучит так: когда вам следует использовать аннотации вместо XML и почему? Прежде всего, это дело вкуса, поскольку оба ведут себя абсолютно одинаково. Когда метаданные действительно связаны с кодом (например, речь может идти о первичном ключе), имеет смысл использовать аннотации, поскольку метаданные являются лишь еще одним аспектом программы. Прочие виды метаданных, например длина столбца или другие детали схемы, могут быть изменены в зависимости от среды развертывания (скажем, схема базы данных может отличаться в среде разработки, тестирования или производства). Похожая ситуация возникает, когда основанному на JPA продукту необходимо обеспечить поддержку баз данных от нескольких разных поставщиков. Может потребоваться настроить генерирование определенного идентификатора, параметра столбца и т. д. в зависимости от типа используемой базы данных. Это может быть лучше выражено во внешних XML-дескрипторах развертывания (по одному на среду), благодаря чему код не придется модифицировать.

Снова обратимся к примеру сущности `Book`. На этот раз представьте, что у вас имеется две среды и вы желаете отобразить сущность `Book` в таблицу `BOOK` в среде разработки и в таблицу `BOOK_XML_MAPPING` в среде тестирования. Класс будет аннотирован только с помощью `@Entity` (листинг 5.26) и не станет включать информацию о таблице, в которую его надлежит отобразить (то есть аннотация `@Table` будет отсутствовать). Аннотация `@Id` определяет первичный ключ как генерируемый автоматически, а аннотация `@Column` задает длину описания равной 500 символам.

Листинг 5.26. Сущность Book лишь с несколькими нноt циями

```
@Entity
public class Book {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  @Column(length = 500)
  private String description;
  private String isbn;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

В отдельном файле book_mapping.xml (листинг 5.27), придерживаясь заданной XML-схемы, вы можете изменить отображение для любых данных сущности. Тег <table> позволяет вам изменить имя таблицы, в которую будет отображаться сущность (BOOK_XML_MAPPING вместо BOOK по умолчанию). В теге <attributes> вы можете настроить атрибуты, указав не только их имена и длину, но и их связи с другими сущностями. Например, вы можете изменить отображение для столбца title и количество страниц (nbOfPage).

Листинг 5.27. Отобр жение ф йл META-INF/book_mapping.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<entity-mappings xmlns="http://java.sun.com/xml/ns/persistence/orm"
                 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
            xsi:schemaLocation="http://java.sun.com/xml/ns/persistence/orm ➥
                 http://java.sun.com/xml/ns/persistence/orm_2_1.xsd"
                 version="2.1">
  <entity class="org.agoncal.book.javaee7.chapter05.Book">
    <table name="book_xml_mapping"/>
    <attributes>
      <basic name="title">
        <column name="book_title" nullable="false" updatable="false"/>
      </basic>
      <basic name="description">
        <column length="2000"/>
      </basic>
      <basic name="nbOfPage">
        <column name="nb_of_page" nullable="false"/>
      </basic>
    </attributes>
  </entity>
</entity-mappings>
```

Всегда следует помнить о том, что XML имеет преимущественное значение перед аннотациями. Хотя атрибут description и аннотирован посредством @Column(length = 500), используется та длина столбца, которая указана в файле book_mapping.xml

(см. листинг 5.27). Она равна 2000. Это может привести в замешательство, если вы взглянете на соответствующий код и увидите значение 500, а затем посмотрите на DDL-код и увидите значение 2000. Никогда не забывайте проверять XML-дескриптор развертывания.

Результатом слияния XML-метаданных и метаданных аннотаций является то, что сущность Book будет отображена в структуру таблицы BOOK_XML_MAPPING, определенную в листинге 5.28. Если вы решите полностью проигнорировать аннотации и определить свое отображение с использованием только XML, то сможете добавить тег <xml-mapping-metadata-complete> в файл book_mapping.xml (в данном случае все аннотации будут игнорироваться, даже если XML не будет содержать переопределение).

Листинг 5.28. Структура таблицы BOOK_XML_MAPPING

```
create table BOOK_XML_MAPPING (
  ID BIGINT not null,
  BOOK_TITLE VARCHAR(255) not null,
  DESCRIPTION VARCHAR(2000),
  NB_OF_PAGE INTEGER not null,
  PRICE DOUBLE(52, 0),
  ISBN VARCHAR(255),
  ILLUSTRATIONS SMALLINT,
  primary key (ID)
);
```

Чтобы заставить все это работать, не хватает лишь одной порции информации. Вам нужно указать ссылку на файл book_mapping.xml в своем файле persistence.xml, для чего потребуется прибегнуть к тегу <mapping-file>. Названный файл определяет контекст постоянства сущности и базу данных, в которую она должна быть отображена. Это самая важная порция информации, которая необходима поставщику постоянства для ссылки на внешний XML-файл отображения. Произведите развертывание сущности Book с обоими XML-файлами в каталоге META-INF, и все будет готово (листинг 5.29).

Листинг 5.29. Файл persistence.xml со ссылкой на внешний файл отображения

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://java.sun.com/xml/ns/persistence ➥
             http://java.sun.com/xml/ns/persistence/persistence_2_1.xsd"
             version="2.1">
  <persistence-unit name="chapter05PU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
      <class>org.agoncal.book.javaee7.chapter05.Book</class>
      <mapping-file>META-INF/book_mapping.xml</mapping-file>
      <properties>
        <!-- Свойства поставщика постоянства -->
      </properties>
  </persistence-unit>
</persistence>
```

Встр ив емые объекты

В приводившемся ранее в этой главе подразделе «Составные первичные ключи» вы видели, как класс может быть встроен и использован в качестве первичного ключа с применением аннотации `@EmbeddedId`. Встраиваемые объекты — это объекты, которые сами по себе не имеют постоянного идентификатора; они могут быть только встроены во владеющие сущности. Владеющая сущность может располагать коллекциями встраиваемых объектов, а также одним встраиваемым атрибутом. Они сохраняются как внутренняя часть владеющей сущности и совместно используют идентификатор этой сущности. Это означает, что каждый атрибут встроенного объекта будет отображаться в таблицу сущности. Это связь строгого владения (также называемая композицией), поэтому если удалить сущность, то встроенные объекты тоже окажутся удалены.

Эта композиция между двумя классами задействует аннотации. Для включенного класса используется аннотация `@Embeddable`, в то время как в отношении сущности, которая включает этот класс, используется `@Embedded`. Обратимся к примеру клиента, у которого имеется идентификатор, имя, адрес электронной почты, а также домашний адрес. Все соответствующие атрибуты могли бы располагаться в одной сущности `Customer` (листинг 5.31), однако по причинам объектного моделирования они разделены на два класса: `Customer` и `Address`. Поскольку у `Address` нет собственного идентификатора, и при этом он всего лишь часть состояния `Customer`, этот класс является хорошим кандидатом на то, чтобы стать встраиваемым объектом вместо сущности (листинг 5.30).

Листинг 5.30. Кл сс Address является встр ив емым объектом

```
@Embeddable
public class Address {
  private String street1;
  private String street2;
  private String city;
  private String state;
  private String zipcode;
  private String country;
  // Конструкторы, геттеры, сеттеры
}
```

Как вы можете видеть в листинге 5.30, класс `Address` аннотирован не как сущность, а как встраиваемый объект. Аннотация `@Embeddable` определяет, что `Address` может быть встроен в иной класс-сущность (или иной встраиваемый объект). С другой стороны, для сущности `Customer` приходится использовать аннотацию `@Embedded`, чтобы определить, что `Address` является постоянным атрибутом, который будет сохранен как внутренняя часть и станет совместно использовать его идентификатор (см. листинг 5.31).

Листинг 5.31. Сущность Customer со встроенным Address

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
```

```
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Embedded
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Каждый атрибут Address будет отображаться в таблицу владеющей сущности Customer. Будет только одна таблица со структурой, определенной в листинге 5.32. Как вы увидите позднее в подразделе «Переопределение атрибутов» раздела «Отображение наследования» данной главы, сущности могут переопределять атрибуты встраиваемых объектов (с использованием аннотации @AttributeOverrides).

Листинг 5.32. Структур т блицы CUSTOMER со всеми трибут ми Address

```
create table CUSTOMER (
  ID BIGINT not null,
  LASTNAME VARCHAR(255),
  PHONENUMBER VARCHAR(255),
  EMAIL VARCHAR(255),
  FIRSTNAME VARCHAR(255),
  STREET2 VARCHAR(255),
  STREET1 VARCHAR(255),
  ZIPCODE VARCHAR(255),
  STATE VARCHAR(255),
  COUNTRY VARCHAR(255),
  CITY VARCHAR(255),
  primary key (ID)
);
```

ПРИМЕЧАНИЕ

В предшествующих разделах я показывал вам, как отображать коллекции и отображения базовых типов данных. В JPA 2.1 то же самое возможно с помощью встраиваемых объектов. Вы можете отображать коллекции встраиваемых объектов, а также отображения таких объектов (встраиваемый объект может быть либо ключом, либо значением отображения).

Тип доступ встр ив емого кл сс . Тип доступа встраиваемого класса обуславливается типом доступа класса-сущности, в котором он располагается. Если сущность явным образом использует такой тип доступа, как доступ к свойствам, то встраиваемый объект будет неявно использовать аналогичный тип доступа. Другой тип доступа для встраиваемого класса можно указать с помощью аннотации @Access.

Сущности Customer (листинг 5.33) и Address (листинг 5.34) задействуют разные типы доступа.

Листинг 5.33. Сущность Customer с т ким типом доступ , к к доступ к полям

```
@Entity
@Access(AccessType.FIELD)
public class Customer {
  @Id @GeneratedValue
```

```
  private Long id;
  @Column(name = "first_name", nullable = false, length = 50)
  private String firstName;
  @Column(name = "last_name", nullable = false, length = 50)
  private String lastName;
  private String email;
  @Column(name = "phone_number", length = 15)
  private String phoneNumber;
  @Embedded
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.34. Встр ив емый объект с т ким типом доступ , к к доступ к свойств м

```
@Embeddable
@Access(AccessType.PROPERTY)
public class Address {
  private String street1;
  private String street2;
  private String city;
  private String state;
  private String zipcode;
  private String country;
  // Конструкторы
  @Column(nullable = false)
  public String getStreet1() {
    return street1;
  }
  public void setStreet1(String street1) {
    this.street1 = street1;
  }
  public String getStreet2() {
    return street2;
  }
  public void setStreet2(String street2) {
    this.street2 = street2;
  }
  @Column(nullable = false, length = 50)
  public String getCity() {
    return city;
  }
  public void setCity(String city) {
    this.city = city;
  }
  @Column(length = 3)
  public String getState() {
    return state;
  }
  public void setState(String state) {
    this.state = state;
  }
```

```
  @Column(name = "zip_code", length = 10)
  public String getZipcode() {
    return zipcode;
  }
  public void setZipcode(String zipcode) {
    this.zipcode = zipcode;
  }
  public String getCountry() {
    return country;
  }
  public void setCountry(String country) {
    this.country = country;
  }
}
```

Настоятельно рекомендуется явным образом задавать тип доступа для встраиваемых объектов, чтобы избежать ошибок отображения, когда один встраиваемый объект окажется встроенным во множественные сущности. К примеру, расширим нашу модель, добавив сущность `Order`, как показано на рис. 5.4. `Address` теперь будет встроен в `Customer` и `Order`.



Рис. 5.4. Address встроен в Customer и Order

Для каждой сущности определяется отличающийся тип доступа: `Customer` использует доступ к полям, а `Order` — доступ к свойствам. Поскольку тип доступа встраиваемого объекта обуславливается типом доступа класса-сущности, в котором он объявлен, `Address` будет отображен двумя разными путями, что может привести к проблемам отображения. Чтобы этого не случилось, тип доступа `Address` должен быть задан явным образом.

ПРИМЕЧАНИЕ

Типы явного доступа также очень полезны, когда речь идет о наследовании. По умолчанию листовые сущности наследуют тип доступа от своей корневой сущности. В иерархии сущностей доступ к каждой из них возможен по-разному из других классов в иерархии. Включение аннотации @Access фактически приведет к тому, что режим доступа по умолчанию для иерархии будет локально переопределен.

Отобр жение связей

Мир объектно-ориентированного программирования изобилует классами и ассоциациями между классами. Эти ассоциации являются структурными в том смысле, что связывают объекты одного типа с объектами другого типа, позволяя одному

объекту заставлять другой объект выполнять действия от своего имени. Между классами могут существовать ассоциации нескольких типов.

Прежде всего, у ассоциации есть направление. Она может быть *однонаправленной* (один объект может осуществлять навигацию по направлению к другому объекту) и *двунаправленной* (один объект может осуществлять навигацию по направлению к другому и наоборот). В Java для навигации по объектам используется точечный (.) синтаксис. Например, если написать `customer.getAddress().getCountry()`, то будет осуществляться навигация от объекта `Customer` к объекту `Address`, а затем — к объекту `Country`.

В унифицированном языке моделирования (Unified Modeling Language — UML) для представления однонаправленной ассоциации между двумя классами используется стрелка, указывающая направление. На рис. 5.5 `Class1` (источник) может осуществлять навигацию по направлению к `Class2` (цель), но не наоборот.



Рис. 5.5. Однон пр вленн я ссоци ция между двумя кл сс ми

Для индикации двунаправленной ассоциации стрелки не используются. Как показано на рис. 5.6, `Class1` может осуществлять навигацию по направлению к `Class2` и наоборот. В Java это представляется как наличие у `Class1` атрибута типа `Class2` и наличие у `Class2` атрибута типа `Class1`.



Рис. 5.6. Двун пр вленн я ссоци ция между двумя кл сс ми

Для ассоциации также характерна *множественность* (или *кардинальность*). На каждом из концов ассоциации можно указать, сколько ссылающихся объектов вовлечено в нее. На UML-диаграмме, приведенной на рис. 5.7, показано, что `Class1` ссылается на нуль или более экземпляров `Class2`.

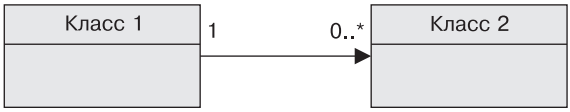


Рис. 5.7. Множественность ссоци ций кл ссов

В UML кардинальность — это диапазон между минимальным и максимальным числами. Таким образом, `0..1` означает, что у вас будет минимум нуль объектов и максимум один объект; `1` означает, что у вас один и только один экземпляр; `1..*` означает, что у вас может быть один или много экземпляров, а `3..6` означа-

ет, что у вас может быть от трех до шести объектов. В Java ассоциация, представляющая более одного объекта, задействует коллекции типа `java.util.Collection`, `java.util.Set`, `java.util.List` или даже `java.util.Map`.

В случае со связью имеет место владение (то есть владелец связи). При однонаправленной связи предполагается владение: нет сомнения, что на рис. 5.5 владельцем является `Class1`, однако при двунаправленной связи, как показано на рис. 5.6, владельца нужно указывать явным образом. В таком случае вы показываете владельца, который определяет физическое отображение и противоположную сторону (не владеющую сторону).

В следующих разделах вы увидите, как отображать коллекции объектов с использованием аннотаций JPA.

Связи в реляционных б з х д нных

В реляционном мире дело обстоит по-другому, поскольку, строго говоря, реляционная база данных — это коллекция *отношений* (также называемых *таблицами*), то есть все, что вы смоделируете, будет таблицей. При моделировании ассоциации у вас не будет списков, наборов или отображений — у вас будут таблицы. В JPA при наличии ассоциации между одним классом и другим в базе данных у вас будет ссылка на таблицу. Эту ссылку можно смоделировать двумя способами: с помощью *внешнего ключа (столбца соединения)* или с использованием *таблицы соединения*. В контексте базы данных столбец, ссылающийся на ключ другой таблицы, называется *столбцом внешнего ключа*.

В качестве примера предположим, что у клиента имеется один домашний адрес. В листингах 5.33 и 5.34 мы моделировали эту связь как встраиваемый объект, однако теперь превратим ее в *связь «один к одному»*. При использовании Java у вас имелся бы класс `Customer` с атрибутом `Address`. В реляционном мире у вас могла бы быть таблица `CUSTOMER`, указывающая на `ADDRESS` с помощью столбца внешнего ключа (или столбца соединения), как показано на рис. 5.8.

Customer

Первичный ключ	Имя	Фамилия	Внешний ключ
1	Джеймс	Роррисон	11
2	Доминик	Джонсон	12
3	Мака	Макарон	13

Address

Первичный ключ	Улица	Город	Страна
11	Алигре	Париж	Франция
12	Бэлхэм	Лондон	Англия
13	Алфама	Лиссабон	Португалия

Рис. 5.8. Связь с использов нием столбц соединения

Есть и второй способ моделирования — с применением таблицы соединения. В таблице `CUSTOMER`, показанной на рис. 5.9, больше нет внешнего ключа `ADDRESS`. Для размещения информации о связи с сохранением внешних ключей была создана промежуточная таблица.

Вы не стали бы использовать таблицу соединения для представления связи «один к одному», поскольку это могло бы отрицательно сказаться на производительности (вам всегда будет нужен доступ к третьей таблице для получения адреса клиента).



Рис. 5.9. Связь с использов нием т блицы соединения

Таблицы соединения обычно применяются при наличии кардинальностей «один ко многим» или «многие ко многим». Как вы увидите в следующем разделе, JPA использует два режима для отображения объектных ассоциаций.

Связи между сущностями

Теперь вернемся к JPA. Большинству сущностей необходима возможность ссылаться на другие сущности или иметь связи с ними. Именно это приводит к созданию графов доменной модели, распространенных в сфере бизнес-приложений. JPA позволяет отображать ассоциации, благодаря чему одна сущность может быть связана с другой в реляционной модели. Как это имеет место в случае с аннотациями элементарного отображения, которые вы видели ранее, JPA задействует конфигурацию в порядке исключения, когда речь идет об ассоциациях. JPA предусматривает используемый по умолчанию способ сохранения связей, однако если он окажется неподходящим для вашей модели базы данных, в вашем распоряжении есть несколько аннотаций, которые вы сможете использовать для настройки отображения.

Между двумя сущностями могут быть отношения «один к одному», «один ко многим», «многие к одному» или «многие ко многим». Каждое соответствующее отображение именуется согласно типу связи источника и цели: аннотации `@OneToOne`, `@OneToMany`, `@ManyToOne` или `@ManyToMany`. Каждая аннотация может быть использована однонаправленным или двунаправленным путем. В табл. 5.1 приведены все возможные комбинации типов связи и направлений.

Т блиц 5.1. Все возможные комбин ции «тип связи — н пр вление»

Тип связи	Н пр вление
«Один к одному»	Однонаправленный подход
«Один к одному»	Двунаправленный подход
«Один ко многим»	Однонаправленный подход

Тип связи	Н пр вление
«Многие к одному»/«один ко многим»	Двунаправленный подход
«Многие к одному»	Однонаправленный подход
«Многие ко многим»	Однонаправленный подход
«Многие ко многим»	Двунаправленный подход

Вы увидите, что названные подходы являются повторяющимися концепциями, которые применимы одинаковым образом ко всем типам связи. Далее вы узнаете, в чем заключается разница между однонаправленными и двунаправленными связями, а затем реализуете некоторые из соответствующих комбинаций. Я не стану подробно разбирать весь перечень комбинаций, а сосредоточусь на одном подмножестве. Объяснение всех комбинаций было бы повторением одного и того же. Важно, чтобы вы поняли, как отображать отношение и направление в связях.

Однон пр вленные и двун пр вленные ссоци ции

С точки зрения объектного моделирования связь между классами естественна. При однонаправленной ассоциации объект `A` указывает только на объект `B`; при двунаправленной ассоциации оба объекта ссылаются друг на друга. Однако потребуется приложить усилия, когда речь зайдет об отображении двунаправленной связи в реляционную базу данных, как показано в следующем примере, включающем клиента, у которого имеется домашний адрес.

При однонаправленной связи у сущности `Customer` имеется атрибут типа `Address` (рис. 5.10). Эта связь является однонаправленной и предполагает навигацию от одной стороны к другой. `Customer` в данном случае называется владельцем связи. В контексте базы данных это означает, что таблица `CUSTOMER` будет располагать внешним ключом (столбцом соединения), указывающим на `ADDRESS`, а если вы будете владеть связью, то сможете настроить отображение этой связи. Например, если вам потребуется изменить имя внешнего ключа, отображение будет выполнено в сущности `Customer` (то есть владельце).

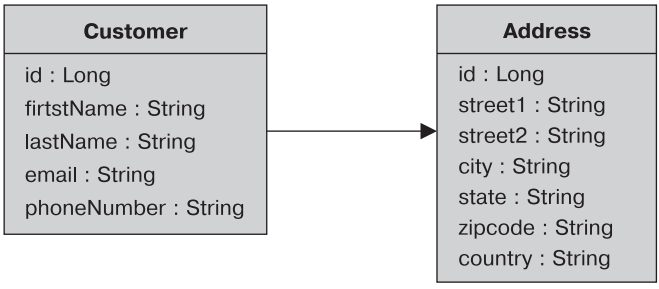


Рис. 5.10. Однон пр вленн я ссоци ция между Customer и Address

Как уже отмечалось, связи также могут быть двунаправленными. Чтобы иметь возможность осуществлять навигацию между `Address` и `Customer`, вам потребуется преобразовать однонаправленную связь в двунаправленную, добавив атрибут

Customer в сущность Address (рис. 5.11). Обратите внимание, что на UML-диаграммах классов не приводятся атрибуты, представляющие связь.

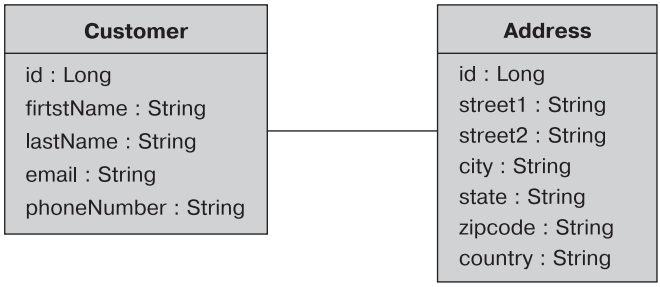


Рис. 5.11. Двун пр вленн я ссоци ция между Customer и Address

В контексте Java-кода и аннотаций это аналогично наличию двух отдельных отображений «один к одному», по одному в каждом направлении. Вы можете представлять себе двунаправленную связь как пару однонаправленных связей, идущих соответственно туда и обратно (рис. 5.12).

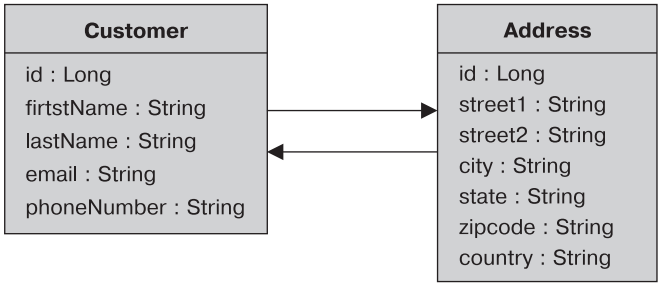


Рис. 5.12. Двун пр вленн я ссоци ция, предст вленн я двумя стрелк ми

Как осуществляется отображение пары однонаправленных связей? Кто является владельцем двунаправленной связи? Кто владеет информацией об отображении столбца соединения или таблицы соединения? Если у однонаправленных связей есть владеющая сторона, то у двунаправленных связей есть как владеющая, так и противоположная сторона, которая должна быть указана явным образом с помощью элемента mappedBy аннотаций @OneToOne, @OneToMany и @ManyToMany. Элемент mappedBy идентифицирует атрибут, который владеет связью и необходим для двунаправленных связей.

Для пояснения сравним Java-код (с одной стороны) и отображение базы данных (с другой стороны). Как вы можете видеть в левой части рис. 5.13, обе сущности указывают друг на друга с помощью атрибутов: у Customer имеется атрибут, аннотированный с использованием @OneToOne, а у сущности Address — атрибут Customer, тоже снабженный аннотацией. В правой части располагается отображение базы данных, где показаны таблицы CUSTOMER и ADDRESS. CUSTOMER является владельцем связи, поскольку содержит внешний ключ ADDRESS.

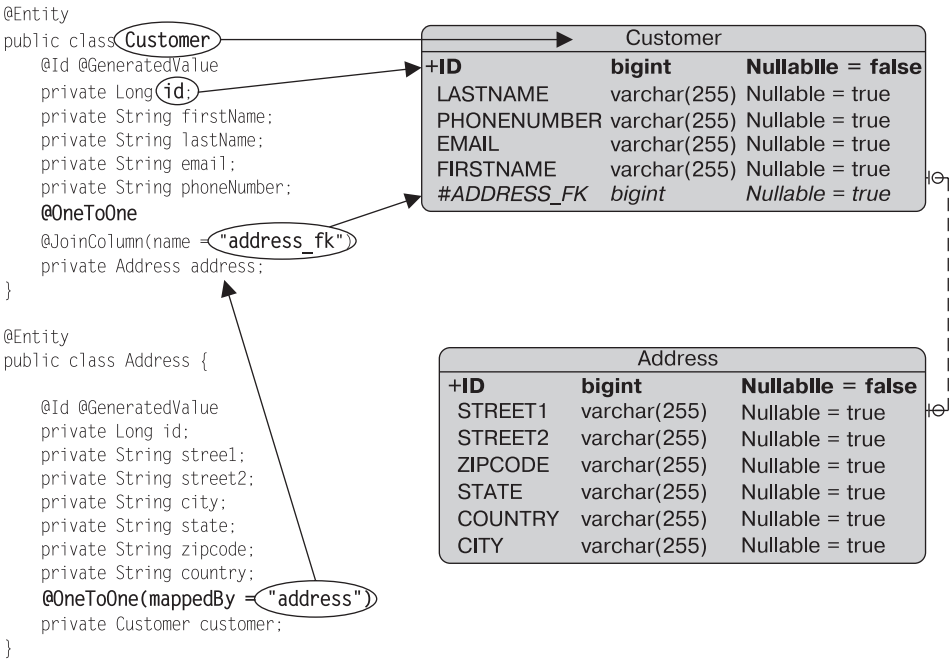


Рис. 5.13. Код Customer и Address н ряду с отобр жением б зы д нных

На рис. 5.13 в аннотации `@OneToOne` сущности `Address` используется элемент `mappedBy`. `Address` в данном случае именуется противоположным владельцем связи, поскольку обладает элементом `mappedBy`. Элемент `mappedBy` говорит о том, что столбец соединения (`address`) указан на другом конце связи. Кроме того, на другом конце связи сущность `Customer` определяет столбец соединения путем использования аннотации `@JoinColumn` и переименовывает внешний ключ в `address_fk`. Сущность `Customer` представляет собой владеющую сторону связи и, как владелец, должна определять отображение столбца соединения. `Address` выступает в качестве противоположной стороны связи, где таблица владеющей сущности содержит внешний ключ (таблица `CUSTOMER` включает столбец `ADDRESS_FK`).

Элемент `mappedBy` может присутствовать в аннотациях `@OneToOne`, `@OneToMany` и `@ManyToMany`, но не в `@ManyToOne`. Атрибут `mappedBy` не может быть сразу на обеих сторонах двунаправленной ассоциации. Было бы также неправильно, если бы его не было ни на одной из сторон, поскольку поставщик воспринимал бы все это как две независимые однонаправленные связи. Это подразумевало бы, что каждая из сторон является владельцем и может определять столбец соединения.

ПРИМЕЧАНИЕ

Если вы знакомы с более ранними версиями Hibernate, то можете представлять себе JPA-параметр mappedBy как эквивалент Hibernate-атрибута inverse.

Однон пр вленн я связь с использов нием @OneToOne

Однонаправленная связь «один к одному» между сущностями имеет ссылку кардинальности 1, которая досягаема только в одном направлении. Обратившись к примеру клиента и его домашнего адреса, предположим, что у него имеется только один домашний адрес (кардинальность 1). Важно выполнить навигацию от Customer (источник) по направлению к Address (цель), чтобы узнать, где клиент живет. Однако по некоторым причинам при нашей модели, показанной на рис. 5.14, вам не потребуется возможность навигации в противоположном направлении (например, вам не будет нужно знать, какой клиент живет по определенному адресу).

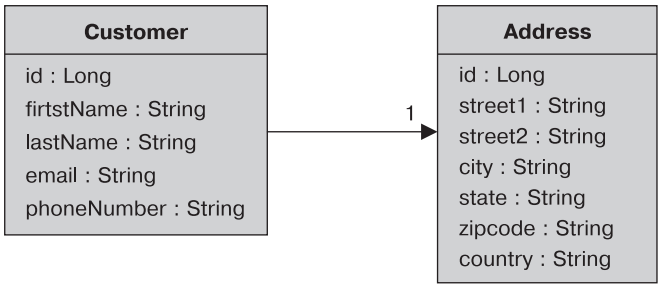


Рис. 5.14. У одного клиент имеется один дом шний дрес

В Java однонаправленная связь означает, что у Customer будет иметься атрибут Address (листинг 5.35), однако у Address не будет атрибута Customer (листинг 5.36).

Листинг 5.35. Customer с одним Address

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.36. Сущность Address

```
@Entity
public class Address {
  @Id @GeneratedValue
  private Long id;
  private String street1;
  private String street2;
  private String city;
  private String state;
  private String zipcode;
```

```
  private String country;
  // Конструкторы, геттеры, сеттеры
}
```

Как вы можете видеть в листингах 5.35 и 5.36, эти две сущности снабжены минимально необходимыми нотациями: `@Entity` с `@Id` и `@GeneratedValue` для первичного ключа и все. При использовании конфигурации в порядке исключения поставщик постоянства отобразит эти две сущности в две таблицы, а также внешний ключ для связи (из `customer`, указывающего на `address`). Отображение «один к одному» инициируется тем фактом, что `Address` является объявленной сущностью и включает сущность `Customer` как атрибут. Мы автоматически предполагаем связь, используя одну сущность как свойство другой, поэтому нам не нужна аннотация `@OneToOne`, поскольку она опирается на правила по умолчанию (листинги 5.37 и 5.38).

Листинг 5.37. Т блиц CUSTOMER с внешним ключом к ADDRESS

```
create table CUSTOMER (
  ID BIGINT not null,
  FIRSTNAME VARCHAR(255),
  LASTNAME VARCHAR(255),
  EMAIL VARCHAR(255),
  PHONENUMBER VARCHAR(255),
  ADDRESS_ID BIGINT,
  primary key (ID),
  foreign key (ADDRESS_ID) references ADDRESS(ID)
);
```

Листинг 5.38. Т блиц ADDRESS

```
create table ADDRESS (
  ID BIGINT not null,
  STREET1 VARCHAR(255),
  STREET2 VARCHAR(255),
  CITY VARCHAR(255),
  STATE VARCHAR(255),
  ZIPCODE VARCHAR(255),
  COUNTRY VARCHAR(255),
  primary key (ID)
);
```

Как вы уже знаете, при использовании JPA, если не аннотировать атрибут, будут применены правила отображения по умолчанию. Таким образом, изначально внешний ключ будет носить имя `ADDRESS_ID` (листинг 5.37), которое представляет собой конкатенацию имени атрибута связи (здесь это `address`), символа «_» и имени столбца первичного ключа целевой таблицы (здесь им будет идентификатор столбца таблицы `ADDRESS`). Обратите также внимание, что в DDL-коде столбец `ADDRESS_ID` по умолчанию является `nullable`, а это означает, что ассоциация «один к одному» отображается в нуль (значение `null`) или единицу.

Для настройки отображения вы можете использовать две аннотации. Первая — `@OneToOne` (поскольку кардинальность отношения равна единице). Она позволяет модифицировать отдельные атрибуты ассоциации как таковой, например, подход,

посредством которого должна осуществляться ее выборка. В листинге 5.39 определен API-интерфейс аннотации @OneToOne.

Листинг 5.39. API-интерфейс ннот ции @OneToOne

```
@Target({METHOD, FIELD}) @Retention(RUNTIME)
public @interface OneToOne {
  Class targetEntity() default void.class;
  CascadeType[] cascade() default {};
  FetchType fetch() default EAGER;
  boolean optional() default true;
  String mappedBy() default "";
  boolean orphanRemoval() default false;
}
```

Второй аннотацией является @JoinColumn (ее API-интерфейс очень схож с API-интерфейсом аннотации @Column из листинга 5.12). Она используется для настройки столбца соединения, то есть внешнего ключа владеющей стороны. В листинге 5.40 показано, как вы применили бы две эти аннотации.

Листинг 5.40. Сущность Customer с н строенным отобр жением связей

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @OneToOne (fetch = FetchType.LAZY)
  @JoinColumn(name = "add_fk", nullable = false)
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

В JPA внешний ключ называется столбцом соединения. Аннотация @JoinColumn позволяет вам настраивать отображение внешнего ключа. В листинге 5.40 она используется для переименования столбца внешнего ключа в ADD_FK, а также для того, чтобы сделать связь обязательной, отклонив значение null (nullable=false). Аннотация @OneToOne подсказывает поставщику постоянства о том, что выборка связи должна быть отложенной (подробнее об этом мы поговорим позднее).

Однон пр вленн я связь с использов нием @OneToMany

Связь «один ко многим» наблюдается, когда один объект-источник ссылается на множество объектов-целей. Например, заказ на покупку включает несколько строк заказа (рис. 5.15). Строка заказа могла бы ссылаться на заказ на покупку с использованием аннотации @ManyToOne, однако это не тот случай, поскольку связь является однонаправленной. Order — это сторона «одного» и источник связи, а OrderLine — сторона «многих» и цель.

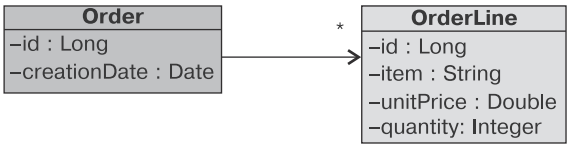


Рис. 5.15. Один з к з включ ет несколько строк

Отношения являются множественными, а навигация осуществляется только от `Order` по направлению к `OrderLine`. В Java эта множественность характеризуется интерфейсами `Collection`, `List` и `Set` из пакета `java.util`. В листинге 5.41 приведен код сущности `Order` с однонаправленной связью «один ко многим» относительно `OrderLine` (листинг 5.42).

Листинг 5.41. Order содержит orderLines

```
@Entity
public class Order {
  @Id @GeneratedValue
  private Long id;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  private List<OrderLine> orderLines;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.42. OrderLine

```
@Entity
@Table(name = "order_line")
public class OrderLine {
  @Id @GeneratedValue
  private Long id;
  private String item;
  private Double unitPrice;
  private Integer quantity;
  // Конструкторы, геттеры, сеттеры
}
```

`Order` из листинга 5.41 не имеет каких-либо специальных аннотаций и опирается на парадигму «конфигурация в порядке исключения». Тот факт, что коллекция типа сущности используется как атрибут для этой сущности, по умолчанию инициирует отображение связи «один ко многим». Изначально однонаправленные связи «один ко многим» задействуют таблицу соединения с двумя столбцами внешнего ключа для сохранения информации о связях. Один столбец внешнего ключа будет ссылаться на таблицу `ORDER` и иметь тот же тип, что и ее первичный ключ, а другой будет ссылаться на таблицу `ORDER_LINE`. Имя этой таблицы соединения будет состоять из имен обеих сущностей, разделенных символом «_». Таблица соединения получит имя `ORDER_ORDER_LINE`, а в результате у нас будет схематическая структура, показанная на рис. 5.16.

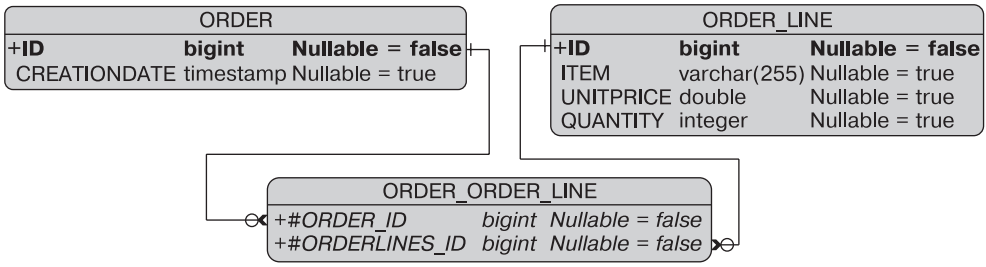


Рис. 5.16. Т блиц соединения между ORDER и ORDER_LINE

Если вам не нравятся имена таблицы соединения и внешнего ключа либо если вы выполняете отображение в уже существующую таблицу, то можете воспользоваться аннотациями JPA для переопределения этих применяемых по умолчанию имен. Именем по умолчанию для столбца соединения является конкатенация имени сущности, символа «_» и имени первичного ключа, на который происходит ссылка. В то время как аннотация `@JoinColumn` может быть использована для изменения столбцов внешнего ключа, аннотация `@JoinTable` (листинг 5.43) позволяет сделать то же самое, если речь идет об отображении таблицы соединения. Вы также можете воспользоваться аннотацией `@OneToMany` (листинг 5.44), которая, как и `@OneToOne`, дает возможность настраивать саму связь (применение режима `fetch` и т. д.).

Листинг 5.43. API-интерфейс ннот ции @JoinTable

```
@Target({METHOD, FIELD}) @Retention(RUNTIME)
public @interface JoinTable {
  String name() default "";
  String catalog() default "";
  String schema() default "";
  JoinColumn[] joinColumns() default {};
  JoinColumn[] inverseJoinColumns() default {};
  UniqueConstraint[] uniqueConstraints() default {};
  Index[] indexes() default {};
}
```

Листинг 5.44. Сущность Order с ннотиров нной связью «один ко многим»

```
@Entity
public class Order {
  @Id @GeneratedValue
  private Long id;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  @OneToMany
  @JoinTable(name = "jnd_ord_line",
    joinColumns = @JoinColumn(name = "order_fk"),
    inverseJoinColumns = @JoinColumn(name = "order_line_fk") )
  private List<OrderLine> orderLines;
  // Конструкторы, геттеры, сеттеры
}
```

В случае с API-интерфейсом аннотации @JoinTable в листинге 5.42 вы можете видеть два атрибута типа @JoinColumn: joinColumns и inverseJoinColumns. Они различаются владеющей и противоположной сторонами. Элемент joinColumns характеризует владеющую сторону (владельца связи) и в нашем примере ссылается на таблицу ORDER. Элемент inverseJoinColumns определяет противоположную сторону, то есть цель связи, и ссылается на ORDER_LINE.

При использовании сущности Order (листинг 5.44) вы можете добавить аннотации @OneToMany и @JoinTable для атрибута orderLines, переименовав таблицу соединения в JND_ORD_LINE (вместо ORDER_ORDER_LINE), а также два столбца внешнего ключа.

Сущность Order из листинга 5.44 будет отображена в таблицу соединения, описанную в листинге 5.45.

Листинг 5.45. Структура таблицы соединения между ORDER и ORDER_LINE

```
create table JND_ORD_LINE (
  ORDER_FK BIGINT not null,
  ORDER_LINE_FK BIGINT not null,
  primary key (ORDER_FK, ORDER_LINE_FK),
  foreign key (ORDER_LINE_FK) references ORDER_LINE(ID),
  foreign key (ORDER_FK) references ORDER(ID)
);
```

Правило по умолчанию для однонаправленной связи «один ко многим» — использование таблицы соединения, однако его очень легко (и целесообразно, если речь идет об унаследованных базах данных) изменить таким образом, чтобы применялись внешние ключи. Для сущности Order необходимо предусмотреть аннотацию @JoinColumn вместо @JoinTable, что позволит изменить код, как показано в листинге 5.46.

Листинг 5.46. Сущность Order наряду со столбцом соединения

```
@Entity
public class Order {
  @Id @GeneratedValue
  private Long id;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  @OneToMany(fetch = FetchType.EAGER)
  @JoinColumn(name = "order_fk")
  private List<OrderLine> orderLines;
  // Конструкторы, геттеры, сеттеры
}
```

Код сущности OrderLine (показанной в листинге 5.46) не изменится. Обратите внимание, что в листинге 5.46 аннотация @OneToMany переключает режим по умолчанию fetch (поменяв LAZY на EAGER). При использовании вами @JoinColumn стратегия внешнего ключа затем обеспечивает отображение однонаправленной связи. Внешний ключ переименовывается с помощью аннотации в ORDER_FK и располагается в целевой таблице (ORDER_LINE). В результате получается структура базы данных, показанная на рис. 5.17. Таблица соединения отсутствует, а ссылка между обеими таблицами осуществляется по внешнему ключу ORDER_FK.

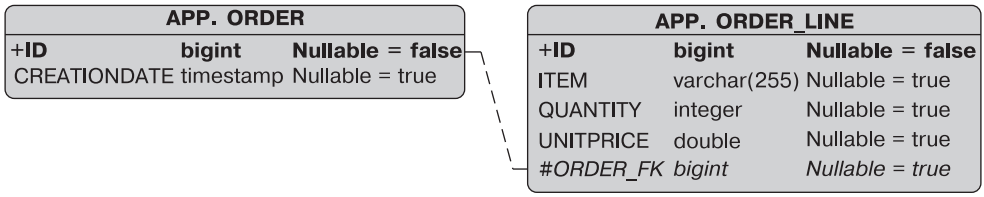


Рис. 5.17. Столбец соединения между ORDER и ORDER_LINE

Двун пр вленн я связь с использов нием @ManyToMany

Двунаправленная связь «многие ко многим» имеет место, когда один объект-источник ссылается на много целей и когда цель ссылается на много источников. Например, CD-альбом создается несколькими артистами, а один артист принимает участие в создании нескольких CD-альбомов. В мире Java у каждой сущности будет коллекция целевых сущностей. В реляционном мире единственный способ отобразить связь «многие ко многим» — использовать таблицу соединения (столбец соединения не поможет). Кроме того, как вы уже видели ранее, при двунаправленной связи вам потребуется явным образом определить владельца (с помощью элемента `mappedBy`).

Если исходить из того, что сущность `Artist` является владельцем связи, то это будет означать, что противоположным владельцем (листинг 5.47) выступает сущность `CD`, которой необходимо, чтобы элемент `mappedBy` был использован в сочетании с ее аннотацией `@ManyToMany`. Элемент `mappedBy` сообщит поставщику постоянства о том, что `appearsOnCDs` — это имя соответствующего атрибута владеющей сущности.

Листинг 5.47. Несколько ртистов приняли уч стие в созд нии одного CD- льбом

```
@Entity
public class CD {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  @ManyToMany(mappedBy = "appearsOnCDs")
  private List<Artist> createdByArtists;
  // Конструкторы, геттеры, сеттеры
}
```

Таким образом, если сущность `Artist` является владельцем связи, как показано в листинге 5.48, то она будет использоваться для настройки отображения таблицы соединения посредством аннотаций `@JoinTable` и `@JoinColumn`.

Листинг 5.48. Один ртист принял уч стие в созд нии нескольких CD- льбомов

```
@Entity
public class Artist {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
```

```
    private String lastName;
    @ManyToMany
    @JoinTable(name = "jnd_art_cd", ➥
      joinColumns = @JoinColumn(name = "artist_fk"), ➥
      inverseJoinColumns = @JoinColumn(name = "cd_fk"))
    private List<CD>appearsOnCDs;
    // Конструкторы, геттеры, сеттеры
}
```

Как вы можете видеть в листинге 5.48, таблица соединения между `Artist` и `CD` переименовывается в `JND_ART_CD`, как и каждый столбец соединения (благодаря аннотации `@JoinTable`). Элемент `joinColumns` ссылается на владеющую сторону (`Artist`), а `inverseJoinColumns` — на противоположную владеющую сторону (`CD`). Соответствующая структура базы данных показана на рис. 5.18.

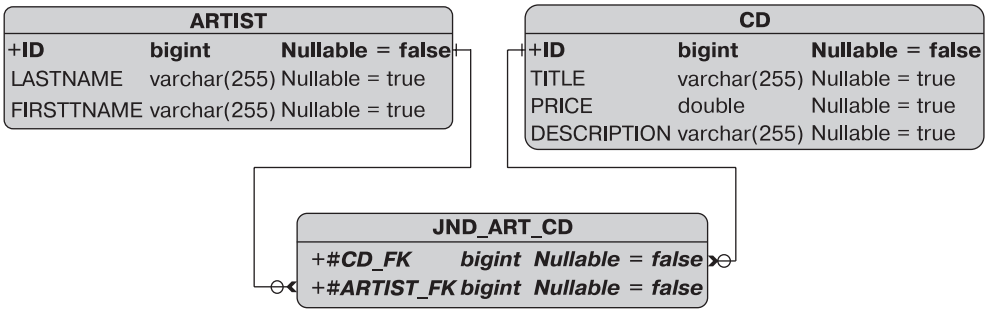


Рис. 5.18. ARTIST, CD и т блиц соединения

Следует отметить, что при двунаправленной связи «многие ко многим» и «один к одному» любая из сторон может быть обозначена как владеющая. Независимо от того, какая из сторон будет обозначена как владелец, владеющая сторона должна включать элемент `mappedBy`. В противном случае поставщик будет считать, что обе стороны являются владельцами, и воспринимать все это как две отдельные однонаправленные связи «один ко многим». В результате этого могло бы получиться четыре таблицы: `ARTIST` и `CD` плюс две таблицы соединения с именами `ARTIST_CD` и `CD_ARTIST`. И недопустимым было бы наличие `mappedBy` на обеих сторонах.

Выборк связей

Все аннотации, которые вы видели ранее (`@OneToOne`, `@OneToMany`, `@ManyToOne` и `@ManyToMany`), определяют атрибут выборки, указывающий, что загрузка ассоциированных объектов должна быть незамедлительной или отложенной с результирующим влиянием на производительность. В зависимости от вашего приложения доступ к одним связям осуществляется чаще, чем к другим. В таких ситуациях вы можете оптимизировать производительность, загружая информацию из базы данных, когда сущность подвергается первоначальному чтению (незамедлительная загрузка) или когда к ней осуществляется доступ (отложенная загрузка). В качестве примера взглянем на некоторые крайние случаи.

Представим себе четыре сущности, которые все связаны между собой и имеют разные отношения («один к одному», «один ко многим»). В первом случае (рис. 5.19) между всеми сущностями будут связи EAGER. Это означает, что, как только вы загрузите Class1 (произведя поиск по идентификатору или выполнив запрос), все зависимые объекты будут автоматически загружены в память. Это может отразиться на производительности вашей системы.

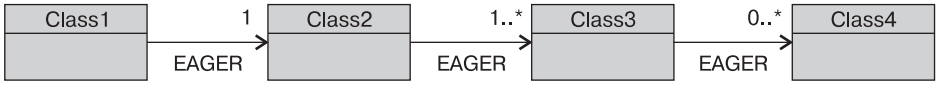


Рис. 5.19. Четыре сущности со связями EAGER

Если взять противоположный сценарий, то все связи будут задействовать режим fetch, обеспечивающий отложенную выборку (рис. 5.20). При загрузке Class1 ничего больше загружаться не будет (за исключением, конечно же, непосредственных атрибутов Class1). Вам потребуется явным образом получить доступ к Class2 (например, с помощью метода-геттера), чтобы дать команду поставщику постоянства на загрузку информации из базы данных и т. д. Если вы захотите управлять всем графом объекта, то вам потребуется явным образом вызывать каждую сущность.

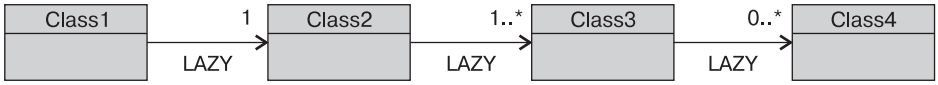


Рис. 5.20. Четыре сущности со связями LAZY

Однако не следует думать, что EAGER — это плохо, а LAZY — хорошо. EAGER поместит все данные в память с помощью небольшого количества операций доступа к базе данных (поставщик постоянства, вероятно, будет использовать запросы с соединением для связи таблиц и извлечения данных). В случае с LAZY вы не рискуете заполнить всю используемую вами память, поскольку будете контролировать, какой объект будет загружаться. Однако вам придется каждый раз осуществлять доступ к базе данных.

Параметр fetch очень важен, поскольку, если его неправильно использовать, это может привести к проблемам с производительностью. У каждой аннотации есть значение fetch по умолчанию, которое вам необходимо знать, и, если оно окажется неподходящим, изменить его (табл. 5.2).

Т блиц 5.2. Стр тегии выборки по умолч нию

Аннот ция	Стр тегия выборки по умолч нию
@OneToOne	EAGER
@ManyToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

Если при разгрузке заказа вам постоянно будет нужен доступ к его строкам в вашем приложении, то, возможно, будет целесообразно изменить режим fetch по умолчанию аннотации @OneToMany на EAGER (листинг 5.49).

Листинг 5.49. Order со связью EAGER с OrderLine

```
@Entity
public class Order {
  @Id @GeneratedValue
  private Long id;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  @OneToMany(fetch = FetchType.EAGER)
  private List<OrderLine> orderLines;
  // Конструкторы, геттеры, сеттеры
}
```

Упорядочение связей

При связях «один ко многим» или «многие ко многим» ваши сущности имеют дело с коллекциями объектов. На стороне Java эти коллекции обычно неупорядочены. В таблицах реляционных баз данных тоже не соблюдается какой-либо порядок. Следовательно, если у вас возникнет необходимость в упорядоченном списке, то вам придется либо отсортировать свою коллекцию программным путем, либо воспользоваться JPQL-запросом с предложением ORDER BY. У JPA имеются более простые механизмы, основанные на аннотациях, которые могут помочь в упорядочении связей.

@OrderBy

Динамическое упорядочение может быть обеспечено благодаря аннотации @OrderBy. «Динамическое» оно потому, что вы упорядочиваете элементы коллекции при извлечении ассоциации.

В примере приложения CD-BookStore пользователям предоставляется возможность писать новости о музыке и книгах. Эти новости затем выкладываются на сайте, а после их публикации люди могут добавлять к ним комментарии (листинг 5.50). Вам необходимо, чтобы комментарии выводились на сайте в хронологическом порядке.

Листинг 5.50. Сущность Comment с postedDate

```
@Entity
public class Comment {
  @Id @GeneratedValue
  private Long id;
  private String nickname;
  private String content;
  private Integer note;
  @Column(name = "posted_date")
  @Temporal(TemporalType.TIMESTAMP)
  private Date postedDate;
  // Конструкторы, геттеры, сеттеры
}
```

Комментарии моделируются с использованием сущности Comment, показанной в листинге 5.50. У нее имеется content, она размещается пользователем (идентифицируется параметром nickname), оставляющим примечания к новостям, кроме того, она располагает postedDate типа TIMESTAMP, который автоматически создается системой. В случае с сущностью News, показанной в листинге 5.51, вы захотите иметь возможность упорядочивать список комментариев согласно дате их размещения в убывающем порядке. Для этого вам потребуется прибегнуть к аннотации @OrderBy в сочетании с аннотацией @OneToMany.

Листинг 5.51. Коммент рии, которые относятся к сущности News, упорядочив ются согл сно д те р змещения в убыв ющем порядке

```
@Entity
public class News {
  @Id @GeneratedValue
  private Long id;
  @Column(nullable = false)
  private String content;
  @OneToMany(fetch = FetchType.EAGER)
  @OrderBy("postedDate DESC")
  private List<Comment> comments;
  // Конструкторы, геттеры, сеттеры
}
```

Аннотация @OrderBy принимает имена атрибутов, которые должны быть отсортированы (атрибут postedDate), а также метод (сортировка в возрастающем или убывающем порядке). Строка ASC или DESC может быть использована для обеспечения сортировки соответственно либо в возрастающем, либо в убывающем порядке. Аннотация @OrderBy может охватывать несколько столбцов. Если вам потребуется выполнить упорядочение согласно дате размещения и примечаниям, то вы сможете воспользоваться OrderBy("postedDate DESC, note ASC").

Аннотация @OrderBy никак не влияет на отображение базы данных. Поставщик постоянства просто информируется о необходимости использовать предложение ORDER BY при извлечении коллекции во время выполнения.

@OrderColumn

Версия JPA 1.0 позволяла осуществлять динамическое упорядочение с использованием аннотации @OrderBy, однако не предусматривала возможности поддерживать постоянное упорядочение. С выходом JPA 2.0 это стало возможным благодаря добавлению аннотации @OrderColumn (ее API-интерфейс схож с API-интерфейсом аннотации @Column из листинга 5.12). Эта аннотация информирует поставщика постоянства о том, что он должен обеспечить поддержку упорядоченного списка с использованием отдельного столбца, в котором располагается индекс. @OrderColumn определяет этот отдельный столбец.

Воспользуемся примером сущностей News и Comment и немного изменим его. На этот раз у сущности Comment, показанной в листинге 5.52, не будет атрибута postedDate и, следовательно, сортировка комментариев в хронологическом порядке окажется невозможной.

Листинг 5.52. Сущность Comment без postedDate

```
@Entity
public class Comment {
  @Id @GeneratedValue
  private Long id;
  private String nickname;
  private String content;
  private Integer note;
  // Конструкторы, геттеры, сеттеры
}
```

Для сохранения сортировки без даты размещения в случае с сущностью News (показанной в листинге 5.53) можно аннотировать связь с помощью @OrderColumn. Тогда поставщик постоянства отобразит сущность News в таблицу с дополнительным столбцом для сохранения сортировки.

Листинг 5.53. Обеспечение постоянств сортировки коммент риев

```
@Entity
public class News {
  @Id @GeneratedValue
  private Long id;
  @Column(nullable = false)
  private String content;
  @OneToMany(fetch = FetchType.EAGER)
  @OrderColumn(name = "posted_index")
  private List<Comment> comments;
  // Конструкторы, геттеры, сеттеры
}
```

В листинге 5.53 @OrderColumn переименовывает дополнительный столбец в POSTED_INDEX. Если это имя не будет изменено, то по умолчанию имя столбца будет представлять собой конкатенацию имени атрибута сущности и строки _ORDER (COMMENTS_ORDER в нашем примере). Этот столбец должен иметь числовой тип. Соответствующая упорядоченная связь будет отображена в отдельную таблицу соединения, как показано далее:

```
create table NEWS_COMMENT (
    NEWS_ID BIGINT not null,
    COMMENTS_ID BIGINT not null,
    POSTED_INDEX INTEGER
);
```

Есть особенности, влияющие на производительность, о которых вам следует знать; как и в случае с аннотацией @OrderColumn, поставщик постоянства должен отслеживать изменения индекса. Он отвечает за поддержку порядка при вставке, удалении и переупорядочении. Если информация окажется вставлена в середину уже существующего отсортированного списка данных, то поставщику постоянства придется переупорядочить весь индекс.

Переносимым приложениям не следует ожидать, что список будет упорядочен базой данных, предполагая, что движки некоторых баз данных автоматически

оптимизируют их индексы, благодаря чему таблица данных выглядит отсортированной. Вместо этого для них следует использовать конструкцию `@OrderColumn` либо `@OrderBy`. Нужно отметить, что вы не сможете одновременно задействовать обе эти аннотации.

Отобр жение н следов ния

Объектно-ориентированные языки задействуют парадигму наследования с момента своего появления. C++ допускает множественное наследование, а Java поддерживает наследование от одного класса. Применяя объектно-ориентированные языки, разработчики обычно повторно используют код, наследуя атрибуты и поведения корневых классов.

Вы только что изучили связи, а связи между сущностями очень просто отображаются в реляционную базу данных. Однако с наследованием дело обстоит подругому. В реляционном мире наследование — неизвестная концепция, которая изначально не реализована там. Концепция наследования предполагает использование ряда трюков при сохранении объектов в реляционную базу данных.

Как превратить иерархическую модель в плоскую реляционную модель? JPA предлагает на выбор три разные стратегии.

- *Иерархическая стратегия «одна таблица на класс»* — совокупность атрибутов всей иерархии сущностей отображается в одну таблицу (применяется по умолчанию).
- *Стратегия «соединенный подкласс»* — при этом подходе каждая сущность в иерархии, конкретная или абстрактная, отображается в свою специально отведенную для этого таблицу.
- *Стратегия «таблица на конкретный класс»* — при использовании этой стратегии каждая иерархия конкретных сущностей отображается в свою отдельную таблицу.

ПРИМЕЧАНИЕ

Поддержка стратегии отображения «таблица на конкретный класс» в версии JPA 2.1 все еще необязательна. Лучше не применять ее для переносимых приложений до тех пор, пока ее поддержка не станет обязательной официально.

Выгодно используя легкость применения аннотаций, JPA 2.1 обеспечивает поддержку определения и отображения иерархий наследования, включая сущности, абстрактные сущности, отображаемые классы и временные классы. Аннотация `@Inheritance` применяется в отношении корневой сущности с целью продиктовать стратегию отображения для нее самой и для листовых классов. JPA также переносит объектное понятие переопределения в сферу отображения, которое позволяет дочерним классам переопределять атрибуты корневого класса. В следующем разделе вы также увидите, как тип доступа может быть использован при наследовании для смешения доступа к полям с доступом к свойствам.

Стр тегии н следов ния

В том, что касается отображения наследования, JPA поддерживает три разные стратегии. При наличии иерархии сущностей одна из них всегда выступает в каче-

стве корневой. Класс корневой сущности может определить стратегию наследования, используя элемент `strategy` аннотации `@Inheritance`, согласно одному из вариантов, определенных в перечислимом типе `javax.persistence.InheritanceType`. В противном случае будет задействована иерархическая стратегия по умолчанию «одна таблица на класс». Чтобы исследовать каждую стратегию, я расскажу вам о том, как отобразить сущности `CD` и `Book`, которые наследуют от сущности `Item` (рис. 5.21).

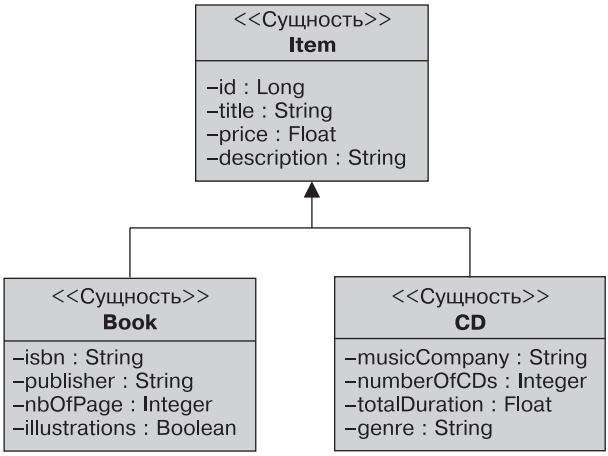


Рис. 5.21. Иерархия наследования между CD, Book и Item

Сущность `Item` является корневой и содержит атрибуты `id`, `title`, `description` и `price`. Обе сущности — `CD` и `Book` — наследуют от `Item`. Каждый из этих листовых классов привносит дополнительные атрибуты, например `isbn` в случае с сущностью `Book` или `totalDuration`, если вести речь о сущности `CD`.

Иерархическая стратегия «одна таблица на класс»

По умолчанию используется стратегия отображения наследования «одна таблица на класс». При ней все сущности в иерархии отображаются в одну таблицу. Поскольку она применяется по умолчанию, вы можете вообще не использовать аннотацию `@Inheritance` в сочетании с корневой сущностью (благодаря конфигурации в порядке исключения), что и было сделано с сущностью `Item` (листинг 5.54).

Листинг 5.54. Сущность Item задействует стратегию по умолчанию «одна таблица на класс»

```
@Entity
public class Item {
  @Id @GeneratedValue
  protected Long id;
  protected String title;
  protected Float price;
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

Item (см. листинг 5.54) является корневой по отношению к сущностям Book (листинг 5.55) и CD (листинг 5.56). Эти сущности наследуют атрибуты от Item, а также используемую по умолчанию стратегию наследования, поэтому нет нужды в аннотации @Inheritance.

Листинг 5.55. Book р сширяет Item

```
@Entity
public class Book extends Item {
  private String isbn;
  private String publisher;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.56. CD р сширяет Item

```
@Entity
public class CD extends Item {
  private String musicCompany;
  private Integer numberOfCDs;
  private Float totalDuration;
  private String genre;
  // Конструкторы, геттеры, сеттеры
}
```

Учитывая то, что вы видели до настоящего момента, без наследования эти три сущности были бы отображены в их собственные отдельные таблицы, однако с наследованием все будет по-другому. При использовании стратегии «одна таблица на класс» все они окажутся в одной и той же таблице базы данных, имя которой по умолчанию будет совпадать с именем корневого класса — ITEM. Ее структура показана на рис. 5.22.

ITEM		
+ID	**bigint**	**Nullable = false**
DTYPE	varchar(31)	Nullable = true
TITLE	varchar(255)	Nullable = false
PRICE	double	Nullable = false
DESCRIPTION	varchar(255)	Nullable = true
ILLUSTRATIONS	smallint	Nullable = true
ISBN	varchar(255)	Nullable = true
NBOFPAGE	integer	Nullable = true
PUBLISHER	varchar(255)	Nullable = true
MUSICCOMPANY	varchar(255)	Nullable = true
NUMBEROFCDS	integer	Nullable = true
TOTALDURATION	double	Nullable = true
GENRE	varchar(255)	Nullable = true

Рис. 5.22. Структур т блицы ITEM

Как вы можете видеть на рис. 5.22, в таблице ITEM собраны все атрибуты сущностей Item, Book и CD. Однако есть дополнительный столбец, который не относится к атрибутам какой-либо из этих сущностей — это столбец дискриминатора DTYPE.

Таблица ITEM будет наполнена информацией, касающейся элементов, книг и CD-альбомов. При доступе к данным поставщику постоянства потребуется знать, какая строка к какой сущности относится. Таким образом, поставщик создаст экземпляр соответствующего типа объекта (Item, Book или CD) при чтении таблицы ITEM. Вот почему столбец дискриминатора используется для того, чтобы явным образом указать тип в каждой строке.

На рис. 5.23 показан фрагмент таблицы ITEM с данными. Как вы можете видеть, в стратегии «одна таблица на класс» имеются некоторые бреши; не каждый столбец подходит для любой из сущностей. В первой строке располагаются данные, касающиеся сущности Item (имя этой сущности содержится в столбце DTYPE). В случае с Item в таблице имеется только название, цена и описание (см. листинг 5.53), при этом отсутствует название компании звукозаписи, ISBN-номер и т. д. Поэтому соответствующие столбцы всегда будут оставаться пустыми.

ID	DTYPE	НАЗВАНИЕ	ЦЕНА	ОПИСАНИЕ	КОМПАНИЯ ЗВУКОЗАПИСИ	ISBN	...
1	Item	Ручка	2.10	Красивая черная ручка			...
2	CD	«Соултрейн»	23.50	Потрясающий джазовый альбом	Prestige		...
3	CD	«Прелести Зут»	18	Один из лучших альбомов Заппы	Warner		...
4	Book	«Роботы зари»	22.30	Роботы повсюду		0-554-456	...
5	Book	«Автостопом по Галактике»	17.50	Интересная книга на тему высоких технологий		1-278-983	...

Рис. 5.23. Фрагмент таблицы ITEM, наполненной данными

Столбец дискриминатора по умолчанию имеет имя DTYPE, тип String (отображаемый в VARCHAR) и содержит имя сущности. Если используемые по умолчанию значения окажутся неподходящими, то аннотация @DiscriminatorColumn позволит вам изменить имя и тип данных. По умолчанию значением этого столбца является имя сущности, на которую он ссылается, однако сущность может переопределить это значение благодаря аннотации @DiscriminatorValue.

В листинге 5.57 столбец дискриминатора переименовывается в DISC (вместо DTYPE), а также изменяется его тип данных с String на Char. Тогда значение дискриминатора каждой из сущностей должно измениться соответственно с Item на I, с Book на B (листинг 5.58) и с CD на C (листинг 5.59).

Листинг 5.57. Item переопределяет столбец дискриминатор

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn (name="disc", ➥
                      discriminatorType = DiscriminatorType.CHAR)
```

```
@DiscriminatorValue("I")
public class Item {
  @Id @GeneratedValue
  protected Long id;
  protected String title;
  protected Float price;
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.58. Book переопределяет зн чение дискримин тор н B @Entity

```
@DiscriminatorValue("B")
public class Book extends Item {
  private String isbn;
  private String publisher;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.59. CD переопределяет зн чение дискримин тор н C @Entity

```
@DiscriminatorValue("C")
public class CD extends Item {
  private String musicCompany;
  private Integer numberOfCDs;
  private Float totalDuration;
  private String genre;
  // Конструкторы, геттеры, сеттеры
}
```

Корневая сущность `Item` один раз определяет столбец дискриминатора для иерархии сущностей с использованием `@DiscriminatorColumn`. Затем она изменяет свое значение по умолчанию на `I` благодаря `@DiscriminatorValue`. Дочерним сущностям требуется переопределить только собственное значение дискриминатора.

Результат показан на рис. 5.24. Столбец дискриминатора и его значения отличаются от тех, что были приведены ранее на рис. 5.23.

ID	DISC	НАЗВАНИЕ	ЦЕНА	ОПИСАНИЕ	КОМПАНИЯ ЗВУКОЗАПИСИ	ISBN	...
1	I	Ручка	2.10	Красивая черная ручка			···
2	C	«Соултрейн»	23.50	Потрясающий джазовый альбом	Prestige		···
3	C	«Прелести Зут»	18	Один из лучших альбомов Заппы	Warner		···
4	B	«Роботы зари»	22.30	Роботы повсюду		0-554-456	···
5	B	«Автостопом по Галактике»	17.50	Интересная книга на тему высоких технологий		1-278-983	···

Рис. 5.24. Т блиц ITEM, включ ющ я другое имя и зн чения дискримин тор

Стратегия «одна таблица на класс» является самой легкой для понимания и хорошо работает, когда иерархия относительно проста и стабильна. Однако у нее имеются кое-какие недостатки: добавление новых сущностей в иерархию или атрибутов в уже существующие сущности влечет добавление новых столбцов в таблицу, миграцию данных и изменение индексов. Эта стратегия также требует, чтобы столбцы дочерних сущностей допускали значение `null`. Если столбец сущности `Book`, содержащий ISBN-номер, не окажется таковым, то вы больше не сможете вставлять данные, которые относятся к CD-альбомам, поскольку для сущности `CD` отсутствует значение такого столбца.

Стратегия «соединенный подкласс»

При использовании стратегии «соединенный подкласс» каждая сущность в иерархии отображается в свою таблицу. Корневая сущность отображается в таблицу, которая определяет первичный ключ, подлежащий использованию всеми таблицами в иерархии, а также столбец дискриминатора. Каждый подкласс представляется с помощью отдельной таблицы, содержащей его атрибуты (не унаследованные от корневого класса) и первичный ключ, который ссылается на первичный ключ корневой таблицы. Таблицы, не являющиеся корневыми, не содержат столбец дискриминатора.

Вы можете реализовать стратегию «соединенный подкласс», снабдив корневую сущность аннотацией `@Inheritance`, как показано в листинге 5.60 (код `CD` и `Book` останется таким же, как и раньше).

Листинг 5.60. Сущность Item с использованием стратегии «соединенный подкласс»

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Item {
  @Id @GeneratedValue
  protected Long id;
  protected String title;
  protected Float price;
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

С точки зрения разработчика, стратегия «соединенный подкласс» естественна, поскольку состояния всех сущностей, абстрактных или конкретных, будут отображаться в разные таблицы. На рис. 5.25 показано, как будут отображены сущности `Item`, `Book` и `CD`.

Вы по-прежнему сможете использовать аннотации `@DiscriminatorColumn` и `@DiscriminatorValue` в случае с корневой сущностью для настройки столбца дискриминатора и изменения значений (столбец `DTYPE` располагается в таблице `ITEM`).

Стратегия «соединенный подкласс» интуитивно понятна и близка к тому, что вы знаете, исходя из механизма объектного наследования. Однако выполнение запросов может влиять на производительность. В названии этой стратегии присутствует слово «соединенный», так как для повторной сборки экземпляра подкласса таблицу подкласса необходимо соединить с таблицей корневого класса.

Чем глубже иерархия, тем больше соединений потребуется для сборки листовой сущности. Эта стратегия хорошо поддерживает полиморфные связи, однако требует, чтобы при создании экземпляров подклассов сущностей была проведена одна или несколько операций соединения. Это может привести к низкой производительности в случае с обширными иерархиями классов. Аналогичным образом запросы, которые охватывают всю иерархию классов, требуют проведения операций соединения между таблицами подклассов, приводящих к снижению производительности.

Стр тегия «т блиц н конкретный кл сс»

Если задействуется стратегия «таблица на класс» (или «таблица на конкретный класс»), то каждая сущность отображается в свою специально отведенную для этого таблицу, как при использовании стратегии «соединенный подкласс». Отличие состоит в том, что все атрибуты корневой сущности также будут отображены в столбцы таблицы дочерней сущности. С позиции базы данных эта стратегия денормализует модель и приводит к тому, что все атрибуты корневой сущности переопределяются в таблицах всех листовых сущностей, которые наследуют от нее. При стратегии «таблица на конкретный класс» нет совместно используемой таблицы, совместно используемых столбцов и столбца дискриминатора. Единственное требование состоит в том, что все таблицы должны совместно пользоваться общим первичным ключом — одинаковым для всех таблиц в иерархии.

Для отображения нашего примера с применением этой стратегии потребуется указать `TABLE_PER_CLASS` в аннотации `@Inheritance` (листинг 5.61) корневой сущности (`Item`).

Листинг 5.61. Сущность Item с использов нием стр тегии «т блиц н конкретный кл сс»

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Item {
  @Id @GeneratedValue
  protected Long id;
  protected String title;
  protected Float price;
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

На рис. 5.26 показаны таблицы `ITEM`, `BOOK` и `CD`. Как вы можете видеть, в `BOOK` и `CD` имеются дубликаты столбцов `ID`, `TITLE`, `PRICE` и `DESCRIPTION` таблицы `ITEM`. Обратите внимание, что показанные таблицы не связаны.

Разумеется, помните, что каждая таблица может быть переопределена, если снабдить каждую сущность аннотацией `@Table`.

Стратегия «таблица на конкретный класс» хорошо работает при выполнении запросов к экземплярам одной сущности, поскольку ее применение схоже с использованием стратегии «одна таблица на класс»: запрос ограничивается одной

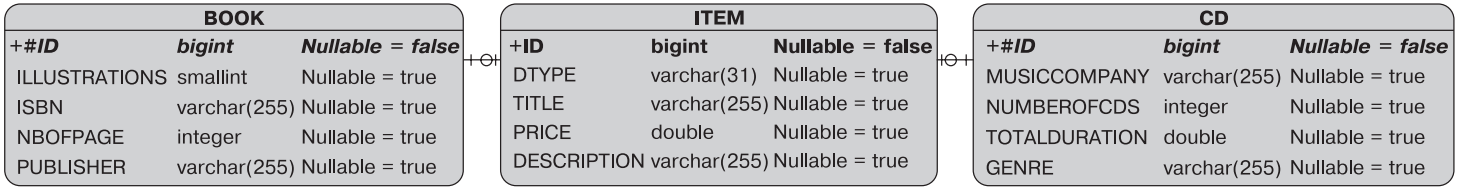


Рис. 5.25. Отобр жение н следов ния с применением стр тегии «соединенный подкл сс»

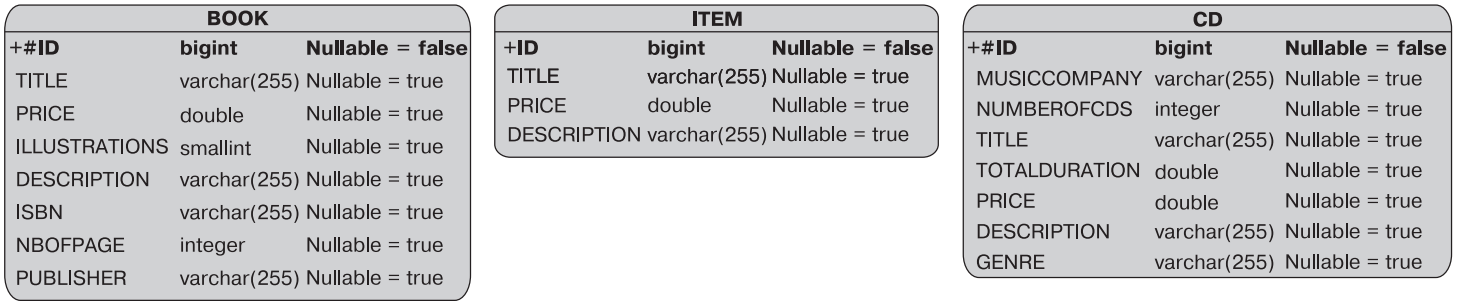


Рис. 5.26. Н личие дублик тов столбцов т блицы ITEM в т блиц х BOOK и CD

таблицей. Недостаток этой стратегии заключается в том, что она делает полиморфные запросы в иерархии классов более затратными, чем другие стратегии (например, поиск всех элементов, включая CD-альбомы и книги). При применении этой стратегии запросы ко всем таблицам подклассов должны выполняться с использованием операции UNION, которая оказывается затратной, если охватывается большой объем данных. Поддержка этой стратегии в JPA 2.1 все еще необязательна.

Переопределение трибутов

При использовании стратегии «таблица на конкретный класс» столбцы таблицы корневого класса дублируются в листовых таблицах. Они будут иметь одинаковые имена. Но что, если применять унаследованную базу данных, а столбцы будут обладать другими именами? JPA задействует аннотацию @AttributeOverride для переопределения отображения одного столбца и @AttributeOverrides, если речь идет о переопределении нескольких.

Чтобы переименовать столбцы ID, TITLE и DESCRIPTION в таблицах BOOK и CD, код сущности Item не потребуется изменять, однако придется задействовать аннотацию @AttributeOverride для сущностей Book (листинг 5.62) и CD (листинг 5.63).

Листинг 5.62. Book переопределяет некоторые столбцы, связ нные с Item

```
@Entity
@AttributeOverrides({
  @AttributeOverride(name = "id", ➥
                     column = @Column(name = "book_id")),
  @AttributeOverride(name = "title", ➥
                     column = @Column(name = "book_title")),
  @AttributeOverride(name = "description", ➥
                     column = @Column(name = "book_description"))
})
public class Book extends Item {
  private String isbn;
  private String publisher;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 5.63. CD переопределяет некоторые столбцы, связ нные с Item

```
@Entity
@AttributeOverrides({
  @AttributeOverride(name = "id", ➥
                     column = @Column(name = "cd_id")),
  @AttributeOverride(name = "title", ➥
                     column = @Column(name = "cd_title")),
  @AttributeOverride(name = "description", ➥
                     column = @Column(name = "cd_description"))
})
public class CD extends Item {
  private String musicCompany;
```

```
    private Integer numberOfCDs;
    private Float totalDuration;
    private String genre;
    // Конструкторы, геттеры, сеттеры
}
```

Поскольку требуется переопределить несколько атрибутов, вам необходимо использовать аннотацию `@AttributeOverrides`, которая принимает массив аннотаций `@AttributeOverride`. После этого каждая аннотация указывает на атрибут сущности `Item` и переопределяет отображение столбца с помощью аннотации `@Column`. Таким образом, `name = "title"` ссылается на атрибут `title` сущности `Item`, а `@Column(name = "cd_title")` информирует поставщика постоянства о том, что `title` надлежит отобразить в столбец `CD_TITLE`. Результат показан на рис. 5.27.

ПРИМЕЧАНИЕ

Ранее в разделе «Встраиваемые объекты» этой главы вы видели, что встраиваемый объект может совместно использоваться несколькими сущностями (Address был встроен в Customer и Order). Поскольку встраиваемые объекты — это внутренняя часть владеющей сущности, в таблице каждой сущности также будут иметься дубликаты столбцов, связанных с этими объектами. Кроме того, @AttributeOverrides можно применять, если вам необходимо переопределить встраиваемые столбцы.

Типы классов в иерархии наследования

В примерах, приводившихся ранее для объяснения стратегий отображения, были задействованы только сущности. `Item`, как и `Book` и `CD`, является сущностью. Однако сущностям не всегда приходится наследовать от сущностей. В иерархии классов могут быть смешаны всевозможные разные классы: сущности, а также классы, которые не являются сущностями (или временные классы), абстрактные сущности и отображенные суперклассы. Наследование от этих классов разных типов будет влиять на отображение.

Абстрактная сущность

В приведенных ранее примерах сущность `Item` представляла собой конкретный класс. Она была снабжена аннотацией `@Entity` и не имела ключевого слова `abstract`, однако абстрактный класс тоже может быть определен как сущность. Абстрактная сущность отличается от конкретной только тем, что нельзя непосредственно создать ее экземпляр с помощью ключевого слова `new`. Она обеспечивает общую структуру данных для своих листовых сущностей (`Book` и `CD`) и придерживается соответствующих стратегий отображения. Для поставщика постоянства абстрактная сущность отображается как обычная сущность. Единственное отличие заключается в пространстве Java, а не в отображении.

Класс, который не является сущностью

Классы, которые не являются сущностями, также называют *временными*, а это означает, что они представляют собой POJO. Сущность может выступать подклассом по отношению к классу, который не является сущностью, либо такой класс

BOOK		
+BOOK_ID	**bigint**	**Nullable = false**
BOOK_TITLE	varchar(255)	Nullable = true
BOOK_DESCRIPTION	varchar(255)	Nullable = true
PRICE	double	Nullable = true
ILLUSTRATIONS	smallint	Nullable = true
ISBN	varchar(255)	Nullable = true
NBOFPAGE	integer	Nullable = true
PUBLISHER	varchar(255)	Nullable = true

ITEM		
+ID	**bigint**	**Nullable = false**
TITLE	varchar(255)	Nullable = true
PRICE	double	Nullable = true
DESCRIPTION	varchar(255)	Nullable = true

CD		
+CD_ID	**bigint**	**Nullable = false**
CD_TITLE	varchar(255)	Nullable = true
CD_DESCRIPTION	varchar(255)	Nullable = true
PRICE	double	Nullable = true
MUSICCOMPANY	varchar(255)	Nullable = true
NUMBEROFCDS	integer	Nullable = true
TOTALDURATION	double	Nullable = true
GENRE	varchar(255)	Nullable = true

Рис. 5.27. Т блицы BOOK и CD переопределяют столбцы т блицы ITEM

может расширять ее. Зачем вам могут понадобиться в иерархии классы, которые не являются сущностями? Объектное моделирование и наследование — это инструменты, с помощью которых совместно используются состояния и поведения. Классы, которые не являются сущностями, могут применяться для обеспечения общей структуры данных для листовых сущностей. Состояние суперкласса, не являющегося сущностью, непостоянно, поскольку не управляется поставщиком постоянства (помните, что условием для того, чтобы класс управлялся поставщиком постоянства, является наличие аннотации `@Entity`).

Например, `Book` представляет собой сущность (листинг 5.65) и расширяет `Item`, который не является сущностью (у `Item` нет аннотаций), как показано в листинге 5.64.

Листинг 5.64. Item — простой POJO без @Entity

```
public class Item {
  protected String title;
  protected Float price;
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

Сущность `Book` (см. листинг 5.65) наследует от `Item`, поэтому Java-код может получить доступ к атрибутам `title`, `price` и `description`, а также к любому другому методу, который определен обычным, объектно-ориентированным, путем. `Item` может быть конкретным или абстрактным и не влияет на финальное отображение.

Листинг 5.65. Сущность Book расширяет POJO

```
@Entity
public class Book extends Item {
  @Id @GeneratedValue
  private Long id;
  private String isbn;
  private String publisher;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Класс `Book` является сущностью и наследует от `Item`. Однако в таблице были бы отображены только атрибуты `Book`. Атрибуты `Item` в структуре таблицы, определенной в листинге 5.66, отсутствуют. Чтобы обеспечить постоянство `Book`, вам потребуется создать экземпляр `Book`, задать значения для любых атрибутов по вашему желанию (`title`, `price`, `isbn`, `publisher` и т. д.), однако будет обеспечено постоянство только атрибутов `Book` (`id`, `isbn` и т. д.).

Листинг 5.66. В таблице BOOK атрибуты Item отсутствуют

```
create table BOOK (
  ID BIGINT not null,
  ILLUSTRATIONS SMALLINT,
```

```
  ISBN VARCHAR(255),
  NBOFPAGE INTEGER,
  PUBLISHER VARCHAR(255),
  primary key (ID)
);
```

Отобр женный суперкл сс

JPA определяет особый тип классов, называемых *отображенными суперклассами*, для совместного использования состояний и поведений, а также информации об отображении, которые от них наследуют сущности. Однако отображенные суперклассы не являются сущностями. Они не управляются поставщиком постоянства, у них нет какой-либо таблицы для отображения в нее, к ним нельзя выполнять запросы и они не могут состоять в связях, однако должны обеспечивать постоянные свойства для любых сущностей, которые расширяют их. Они аналогичны встраиваемым классам за исключением того, что могут быть использованы в сочетании с наследованием. Чтобы показать, что класс является отображенным суперклассом, нужно снабдить его аннотацией `@MappedSuperclass`.

При использовании корневого класса `Item` снабжается аннотацией `@MappedSuperclass`, а не `@Entity`, как показано в листинге 5.67. В данном случае определяется стратегия наследования (`JOINED`), а также аннотируются некоторые из его атрибутов с использованием `@Column`. Однако, поскольку отображенные суперклассы не отображаются в таблицах, не допускается применять аннотацию `@Table`.

Листинг 5.67. Item — отобр женный суперкл сс

```
@MappedSuperclass
@Inheritance(strategy = InheritanceType.JOINED)
public class Item {
  @Id @GeneratedValue
  protected Long id;
  @Column(length = 50, nullable = false)
  protected String title;
  protected Float price;
  @Column(length = 2000)
  protected String description;
  // Конструкторы, геттеры, сеттеры
}
```

Как вы можете видеть в листинге 5.67, атрибуты `title` и `description` снабжены аннотацией `@Column`. В листинге 5.68 показана сущность `Book`, расширяющая `Item`.

Листинг 5.68. Сущность Book р сширяет Item

```
@Entity
public class Book extends Item {
  private String isbn;
  private String publisher;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Эта иерархия будет отображена только в одну таблицу. Item не является сущностью и не содержит таблиц. Атрибуты Item и Book были бы отображены в столбцы таблицы BOOK, кроме того, отображенные суперклассы также совместно используют свою информацию об отображении. Аннотации @Column отображенного суперкласса Item будут унаследованы. Однако, поскольку такие суперклассы не являются сущностями, которые находятся под управлением, вы не смогли бы, к примеру, обеспечить их постоянство или выполнять к ним запросы. В листинге 5.69 показана структура таблицы BOOK с настроенными столбцами TITLE и DESCRIPTION.

Листинг 5.69. Таблица BOOK включает все атрибуты Item

```
create table BOOK (
  ID BIGINT not null,
  TITLE VARCHAR(50) not null,
  PRICE DOUBLE(52, 0),
  DESCRIPTION VARCHAR(2000),
  ILLUSTRATIONS SMALLINT,
  ISBN VARCHAR(255),
  NBOFPAGE INTEGER,
  PUBLISHER VARCHAR(255),
  primary key (ID)
);
```

Резюме

Благодаря конфигурации в порядке исключения немногое требуется для того, чтобы отобразить сущности в таблицах. Проинформируйте поставщика постоянства о том, что класс на самом деле является сущностью (посредством @Entity), атрибут — его идентификатором (посредством @Id), а JPA сделает все остальное. Эта глава могла бы быть намного короче, если бы мы придерживались в ней только того, что применяется по умолчанию. JPA обладает очень богатым набором аннотаций для настройки всех мелких деталей объектно-реляционного отображения (а также эквивалентного XML-отображения).

Элементарные аннотации могут быть использованы в отношении атрибутов (@Basic, @Temporal и т. д.) или классов для настройки отображения. Вы можете изменить имя таблицы либо тип первичного ключа или даже избегать отображения с помощью аннотации @Transient. Применяя JPA, вы можете отображать коллекции базовых типов или встраиваемых объектов. В зависимости от своей бизнес-модели вы можете отображать связи (@OneToOne, @ManyToMany и т. д.) с разными направлениями и множественностью. То же самое касается и наследования (@Inheritance, @MappedSuperclass и т. д.), при котором допустимо использовать разные стратегии для отображения иерархий, где смешаны сущности и классы, не являющиеся сущностями.

В этой главе внимание было сосредоточено на статической части JPA, или на том, как отображать сущности в таблицах. В следующей главе рассматриваются динамические темы: как управлять этими сущностями и выполнять к ним запросы.

Гл в 6
Упр вление постоянными объект ми

У Java Persistence API имеется две стороны. Первая — это способность отображать объекты в реляционные базы данных. Конфигурация в порядке исключения дает поставщикам постоянства возможность выполнять бóльшую часть работы с использованием малого количества кода, а функционал JPA также позволяет осуществлять настроенное отображение из объектов в таблицы с помощью аннотаций или XML-дескрипторов. JPA предлагает широкий спектр настроек, начиная с простого отображения (изменения имени столбца) и заканчивая более сложным (наследованием). Благодаря этому вы сможете отобразить почти любую объектную модель в унаследованной базе данных.

Другая сторона JPA — это способность выполнять запросы к этим отображенным объектам. В JPA централизованной службой для манипулирования экземплярами сущностей является менеджер сущностей. Это API-интерфейс для создания, поиска, удаления и синхронизации объектов с базой данных. Он также позволяет выполнять разнообразные JPQL-запросы, например динамические, статические или «родные» запросы к сущностям. При использовании менеджера сущностей также возможно применение механизмов блокировки.

Мир баз данных опирается на язык структурированных запросов. Этот язык программирования предназначен для управления реляционными данными (извлечение, вставка, обновление и удаление), а его синтаксис является табличноориентированным. Вы можете осуществлять выборку столбцов из таблицы, состоящей из строк, соединять таблицы, комбинировать результаты двух SQL-запросов посредством объединений и т. д. Здесь нет объектов, а есть только строки, столбцы и таблицы. В мире Java, где мы манипулируем объектами, язык, созданный для работы с таблицами (SQL), необходимо «изогнуть» таким образом, чтобы он сочетался с языком, который базируется на объектах (Java). Именно здесь в дело вступает язык запросов Java Persistence Query Language.

JPQL — это язык, определенный в JPA для выполнения запросов к сущностям, которые располагаются в реляционных базах данных. Синтаксис JPQL похож на синтаксис SQL, однако используется в отношении объектов-сущностей, а не взаимодействует непосредственно с таблицами баз данных. JPQL не видит структуры основной базы данных и не имеет дела с таблицами или столбцами, а работает с объектами и атрибутами. Для этого он задействует точечную (.) нотацию, которая знакома Java-разработчикам.

Из этой главы вы узнаете, как управлять постоянными объектами. Это означает, что вы научитесь проводить операции создания, чтения, обновления и удаления

(CRUD) с помощью менеджера сущностей, а также выполнять комплексные запросы с использованием JPQL. В этой главе также рассказывается о том, как JPA справляется с конкурентным доступом и работает с кэшем второго уровня. Она заканчивается объяснением жизненного цикла сущности и того, как JPA позволяет вам добавлять собственную бизнес-логику, когда в случае с сущностью имеют место определенные события.

Менеджер сущностей

Менеджер сущностей — центральный элемент JPA. Он управляет состоянием и жизненным циклом сущностей, а также позволяет выполнять запросы к сущностям в контексте постоянства. Менеджер сущностей отвечает за создание и удаление экземпляров постоянных сущностей и поиск сущностей по их первичному ключу. Он может блокировать сущности для защиты от конкурентного доступа, используя оптимистическую или пессимистическую блокировку, а также способен задействовать JPQL-запросы для извлечения сущностей согласно определенным критериям.

Когда менеджер сущностей получает ссылку на сущность, считается, что он управляет ею. До этого момента сущность рассматривается как обычный POJO-объект (то есть отсоединенный). Мощь JPA заключается в том, что сущности могут использоваться как обычные объекты на разных уровнях приложения и стать управляемыми менеджером сущностей, когда вам необходимо загрузить или вставить информацию в базу данных. Когда сущность находится под управлением, вы можете проводить операции, направленные на обеспечение постоянства, а менеджер сущностей будет автоматически синхронизировать состояние сущности с базой данных. Когда сущность оказывается отсоединенной (то есть не находится под управлением), она снова становится простым Java-объектом, который затем может быть использован на других уровнях (например, JavaServer Faces или JSF на уровне представления) без синхронизации его состояния с базой данных.

Что касается постоянства, то реальная работа здесь начинается с помощью менеджера сущностей. Он является интерфейсом, реализуемым поставщиком постоянства, который будет генерировать и выполнять SQL-операторы. Интерфейс `javax.persistence.EntityManager` представляет собой API-интерфейс для манипулирования сущностями (соответствующее подмножество приведено в листинге 6.1).

Листинг 6.1. Подмножество API-интерфейс EntityManager

```
public interface EntityManager {

  // EntityManagerFactory для создания EntityManager,
  // его закрытия и проверки того, открыт ли он
  EntityManagerFactory getEntityManagerFactory();
  void close();
  boolean isOpen();

  // Возвращает EntityTransaction
  EntityTransaction getTransaction();

  // Обеспечивает постоянство, слияние сущности в базе данных,
```

```
// а также ее удаление оттуда
void persist(Object entity);
<T> T merge(T entity);
void remove(Object entity);

// Обеспечивает поиск сущности на основе ее первичного ключа
// (с разными механизмами блокировки)
<T> T find(Class<T> entityClass, Object primaryKey);
<T> T find(Class<T> entityClass, Object primaryKey, LockModeType lockMode);
<T> T getReference(Class<T> entityClass, Object primaryKey);

// Блокирует сущность, применяя указанный тип режима блокировки
// (оптимистическая, пессимистическая...)
void lock(Object entity, LockModeType lockMode);

// Синхронизирует контекст постоянства с основной базой данных
void flush();
void setFlushMode(FlushModeType flushMode);
FlushModeType getFlushMode();

// Обновляет состояние сущности из базы данных,
// перезаписывая любые внесенные изменения
void refresh(Object entity);
void refresh(Object entity, LockModeType lockMode);

// Очищает контекст постоянства, а также проверяет, содержит ли он сущность
void clear();
void detach(Object entity);
boolean contains(Object entity);

// Задает и извлекает значение свойства EntityManager или подсказку
void setProperty(String propertyName, Object value);
Map<String, Object> getProperties();

// Создает экземпляр Query или TypedQuery для выполнения JPQL-оператора
Query createQuery(String qlString);
<T> TypedQuery<T> createQuery(CriteriaQuery<T> criteriaQuery);
<T> TypedQuery<T> createQuery(String qlString, Class<T> resultClass);

// Создает экземпляр Query или TypedQuery для выполнения именованного запроса
Query createNamedQuery(String name);
<T> TypedQuery<T> createNamedQuery(String name, Class<T> resultClass);

// Создает экземпляр TypedQuery для выполнения «родного» SQL-запроса
Query createNativeQuery(String sqlString);
Query createNativeQuery(String sqlString, Class resultClass);
Query createNativeQuery(String sqlString, String resultSetMapping);

// Создает StoredProcedureQuery для выполнения хранимой процедуры в базе данных
StoredProcedureQuery createStoredProcedureQuery(String procedureName);
```

```
    StoredProcedureQuery createNamedStoredProcedureQuery(String name);

    // Metamodel и CriteriaBuilder для запросов с использованием критериев
    // (выборка, обновление и удаление)
    CriteriaBuilder getCriteriaBuilder();
    Metamodel getMetamodel();
    Query createQuery(CriteriaUpdate updateQuery);
    Query createQuery(CriteriaDelete deleteQuery);

    // Указывает на то, что JTA-транзакция активна,
    // и соединяет с ней контекст постоянства
    void joinTransaction();
    boolean isJoinedToTransaction();

    // Возвращает объект базового поставщика для EntityManager
    <T> T unwrap(Class<T> cls);
    Object getDelegate();

    // Возвращает EntityGraph
    <T> EntityGraph<T> createEntityGraph(Class<T> rootType);
    EntityGraph<?> createEntityGraph(String graphName);
    <T> EntityGraph<T> getEntityGraph(String graphName);
    <T> List<EntityGraph<? super T>> getEntityGraphs(Class<T> entityClass);
}
```

Не стоит пугаться API-интерфейса из листинга 6.1, поскольку в этой главе рассматривается большинство соответствующих методов. В следующем разделе я объясню, как получить экземпляр `EntityManager`.

Получение менеджер сущностей

Менеджер сущностей — центральный интерфейс, используемый для взаимодействия с сущностями, однако приложению нужно сначала получить его. В зависимости от того, является ли среда управляемой контейнером (как вы увидите в главе 7 при работе с EJB-компонентами) или же управляемой приложением, код может быть совершенно другим. Например, в среде, управляемой контейнером, транзакциями управляет контейнер. Это означает, что вам не потребуется явным образом указывать `commit` или `rollback`, что вам пришлось бы сделать в среде, управляемой приложением.

Словосочетание «управляемая приложением» означает, что приложение отвечает за явное получение экземпляра `EntityManager` и управление его жизненным циклом (например, оно закрывает менеджер сущностей по окончании работы). В коде, приведенном в листинге 6.2, показано, как класс, функционирующий в среде Java SE, получает экземпляр менеджера сущностей. Он задействует класс `Persistence` для начальной загрузки экземпляра `EntityManagerFactory`, который ассоциирован с единицей сохраняемости (`chapter06PU`), используемой затем для создания `EntityManager`. Следует отметить, что в среде, управляемой приложением, за создание и закрытие менеджера сущностей (то есть за управление его жизненным циклом) отвечает разработчик.

Листинг 6.2. Кл сс Main, созд ющий менеджер сущностей с помощью ф брики EntityManagerFactory

```
public class Main {
  public static void main(String[] args) {
    // Создает экземпляр Book
    Book book = new Book ("H2G2", "Автостопом по Галактике", ➥
    12.5F, "1-84023-742-2", 354, false);

    // Получает менеджер сущностей и транзакцию
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("chap-
ter06PU");
    EntityManager em = emf.createEntityManager();

    // Обеспечивает постоянство Book в базе данных
    EntityTransaction tx = em.getTransaction();
    tx.begin();
    em.persist(book);
    tx.commit();

    // Закрывает менеджер сущностей и фабрику
    em.close();
    emf.close();
  }
}
```

Создание управляемого приложением менеджера сущностей осуществляется довольно просто с помощью фабрики, однако управляемую приложением среду отличает от управляемой контейнером среды то, каким образом происходит получение `EntityManagerFactory`. Среда, управляемая контейнером, имеет место, когда приложение эволюционирует в сервлет или EJB-контейнер. В среде Java EE самый распространенный способ получения менеджера сущностей заключается в использовании аннотации `@PersistenceContext` или в JNDI-поиске. Компоненту, работающему в контейнере (это может быть сервлет, EJB-компонент, веб-служба и т. д.), не нужно создавать или закрывать `EntityManager`, поскольку его жизненный цикл управляется контейнером. В листинге 6.3 показан код сессионного EJB-компонента без сохранения состояния, в который мы внедряем ссылку на единицу сохраняемости `chapter06PU`.

Листинг 6.3. EJB-компонент без сохр нения состояния с внедренной в него ссылкой н менеджер сущностей

```
@Stateless
public class BookEJB {
  @PersistenceContext(unitName = "chapter06PU")
  private EntityManager em;
  public void createBook() {
    // Создает экземпляр Book
    Book book = new Book("H2G2", "Автостопом по Галактике", ➥
    12.5F, "1-84023-742-2", 354, false);

    // Обеспечивает постоянство Book в базе данных
    em.persist(book);
```

```
    }
}
```

По сравнению с кодом из листинга 6.2 код, приведенный в листинге 6.3, намного проще. Во-первых, в нем нет `Persistence` или `EntityManagerFactory`, поскольку контейнер внедряет экземпляр `EntityManager`. Приложение не отвечает за управление жизненным циклом менеджера сущностей (создание или закрытие). Во-вторых, поскольку EJB-компоненты без сохранения состояния управляют транзакциями, в этом коде не указан явным образом параметр `commit` или `rollback`. Такой стиль `EntityManager` демонстрируется в главе 7.

ПРИМЕЧАНИЕ

Если вы заглянете в подраздел «Производители данных» главы 2, то поймете, что также можете использовать @Inject в сочетании с EntityManager, если будете генерировать его (с применением аннотации @Produces).

Контекст постоянств

Перед тем как подробно исследовать API-интерфейс менеджера сущностей, вам необходимо понять крайне важную концепцию: *контекст постоянства*. Это набор экземпляров сущностей, находящихся под управлением. Он наблюдается в определенный момент времени для транзакции определенного пользователя: в контексте постоянства может существовать только один экземпляр сущности с одним и тем же постоянным идентификатором. Например, если экземпляр `Book` с идентификатором `12` существует в контексте постоянства, то никакой другой экземпляр `Book` с этим идентификатором не может существовать в том же самом контексте постоянства. Менеджер сущностей управляет лишь теми сущностями, которые содержатся в контексте постоянства, а это означает, что изменения будут отражаться в базе данных.

Менеджер сущностей обновляет контекст постоянства или обращается к нему при каждом вызове метода интерфейса `javax.persistence.EntityManager`. Например, когда произойдет вызов метода `persist()`, сущность, передаваемая как аргумент, будет добавлена в контекст постоянства, если ее там еще нет. Аналогичным образом при поиске сущности по ее первичному ключу менеджер сущностей сначала проверяет, не содержится ли уже в контексте постоянства запрашиваемая сущность. Контекст постоянства можно рассматривать как кэш первого уровня. Это небольшое жизненное пространство, в котором менеджер сущностей размещает сущности перед тем, как сбрасывать содержимое в базу данных. По умолчанию объекты располагаются в контексте постоянства столько времени, сколько длится соответствующая транзакция.

Чтобы резюмировать все это, взглянем на рис. 6.1, где показано, что двум пользователям требуется доступ к сущностям, данные которых хранятся в базе данных. У каждого пользователя имеется собственный контекст постоянства, в котором все сохраняется, пока длится его транзакция. Пользователь 1 получает из базы данных сущности с идентификаторами `12` и `56`, поэтому оба размещаются в его контексте постоянства. Пользователь 2 получает сущности с идентификаторами `12` и `34`. Как вы можете видеть, сущность с идентификатором `12` располагается в контексте постоянства каждого из пользователей. Пока длится транзакция, контекст постоянства

выступает в роли кэша первого уровня, где находятся сущности, которыми может управлять менеджер сущностей. Когда транзакция завершается, контекст постоянства очищается от сущностей.

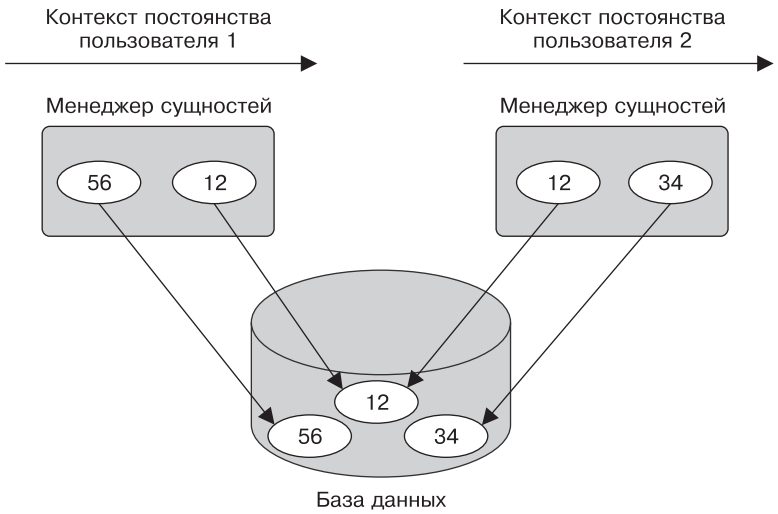


Рис. 6.1. Сущности, которые р спол г ются в контексте постоянств р зных пользов телей

Конфигурация для менеджера сущностей привязывается к экземпляру `EntityManagerFactory`, который применяется для его создания. Независимо от того, является ли среда управляемой приложением или же контейнером, эта фабрика необходима как единица сохраняемости, с использованием которой будет создаваться менеджер сущностей. Единица сохраняемости обуславливает параметры для подключения к базе данных и список сущностей, которыми можно управлять в контексте постоянства. Файл `persistence.xml` (листинг 6.4), располагающийся в каталоге META-INF, определяет единицу сохраняемости, у которой есть имя (`chapter06PU`) и набор атрибутов.

Листинг 6.4. Единиц сохр няемости с н бором сущностей, которые подд ются упр влению

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
             http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
             version="2.1">

  <persistence-unit name="chapter06PU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <class>org.agoncal.book.javaee7.chapter06.Book</class>
    <class>org.agoncal.book.javaee7.chapter06.Customer</class>
    <class>org.agoncal.book.javaee7.chapter06.Address</class>
```

```
    <properties>
      <property name="javax.persistence.schema-generation.database.action" ➥
                value="drop-and-create"/>
      <property name="javax.persistence.jdbc.driver" ➥
                value="org.apache.derby.jdbc.EmbeddedDriver"/>
      <property name="javax.persistence.jdbc.url" ➥
                value="jdbc:derby:memory:chapter06DB;create=true"/>
      <property name="eclipselink.logging.level" value="INFO"/>
    </properties>
  </persistence-unit>
</persistence>
```

Единица сохраняемости — это мост между контекстом постоянства и базой данных. С одной стороны, в теге `<class>` указываются все сущности, которыми можно было бы управлять в контексте постоянства, и, с другой стороны, он предоставляет всю информацию, необходимую для физического подключения к базе данных (с использованием свойств). Это происходит потому, что вы находитесь в среде, управляемой приложением (`transaction-type="RESOURCE_LOCAL"`). Как вы увидите в главе 7, в среде, управляемой контейнером, `persistence.xml` определял бы источник данных вместо свойств для подключения к базе данных и задал бы тип транзакций как JTA (`transaction-type="JTA"`).

В JPA 2.1 некоторые свойства в файле `persistence.xml` были стандартизированы (табл. 6.1). Все они начинаются с `javax.persistence`, например `javax.persistence.jdbc.url`. Требуется, чтобы поставщики JPA поддерживали эти стандартные свойства, однако они могут обеспечивать собственные свойства вроде `eclipselink` в приведенном чуть выше примере (например, `eclipselink.logging.level`).

Таблица 6.1. Стандартные свойства JPA

Свойство	Описание
javax.persistence.jdbc.driver	Полностью уточненное имя класса драйвера
javax.persistence.jdbc.url	Специфичный для драйвера URL-адрес
javax.persistence.jdbc.user	Имя пользователя, применяемое при подключении к базе данных
javax.persistence.jdbc.password	Пароль, применяемый при подключении к базе данных
javax.persistence.database-product-name	Имя целевой базы данных (например, Derby)
javax.persistence.database-major-version	Номер версии целевой базы данных
javax.persistence.database-minor-version	Дополнительный номер версии целевой базы данных
javax.persistence.ddl-create-script-source	Имя сценария, создающего базу данных
javax.persistence.ddl-drop-script-source	Имя сценария, удаляющего базу данных
javax.persistence.sql-load-script-source	Имя сценария, загружающего информацию в базу данных

Продолжение ⇩

Т блиц 6.1 *(продолжение)*

Свойство	Опис ние
javax.persistence.schema-generation.database.action	Определяет действие, которое должно предприниматься в отношении базы данных (none, create, drop-and-create, drop)
javax.persistence.schema-generation.scripts.action	Определяет действие, которое должно предприниматься в отношении DDL-сценариев (none, create, drop-and-create, drop)
javax.persistence.lock.timeout	Значение времени ожидания в миллисекундах при пессимистической блокировке
javax.persistence.query.timeout	Значение времени ожидания в миллисекундах при запросах
javax.persistence.validation.group.pre-persist	Группы, намеченные для валидации при наступлении события pre-persist
javax.persistence.validation.group.pre-update	Группы, намеченные для валидации при наступлении события pre-update
javax.persistence.validation.group.pre-remove	Группы, намеченные для валидации при наступлении события pre-remove

М нипулиров ние сущностями

Мы используем менеджер сущностей как для простого манипулирования сущностями, так и для выполнения комплексных JPQL-запросов. При манипулировании одиночными сущностями интерфейс менеджера можно рассматривать как обобщенный объект доступа к данным (Data Access Object — DAO), который позволяет выполнять CRUD-операции в отношении любой сущности (табл. 6.2).

Т блиц 6.2. Методы интерфейс менеджер сущностей для м нипулиров ния сущностями

Метод	Опис ние
void persist(Object entity)	Делает так, что экземпляр помещается под управление, а также обеспечивает постоянство экземпляра
<T> T find(Class<T> entityClass, Object primaryKey)	Выполняет поиск сущности указанного класса и первичного ключа
<T> T getReference(Class<T> entityClass, Object primaryKey)	Извлекает экземпляр, выборка состояния которого может быть отложенной
void remove(Object entity)	Удаляет экземпляр сущности из контекста постоянства и основной базы данных
<T> T merge(T entity)	Обеспечивает слияние состояния определенной сущности с текущим контекстом постоянства
void refresh(Object entity)	Обновляет состояние экземпляра из базы данных, перезаписывая все изменения, внесенные в сущность, если таковые имеются

Метод	Опис ние
void flush()	Синхронизирует контекст постоянства с основной базой данных
void clear()	Очищает контекст постоянства, приводя к тому, что все сущности, которые находятся под управлением, оказываются отсоединенными
void detach(Object entity)	Убирает определенную сущность из контекста постоянства, приводя к тому, что сущность, которая находится под управлением, оказывается отсоединенной
boolean contains(Object entity)	Проверяет, является ли экземпляр сущностью, находящейся под управлением, которая относится к текущему контексту постоянства

Чтобы помочь вам лучше понять эти методы, я воспользуюсь простым примером однонаправленной связи «один к одному» между `Customer` и `Address`. Обе сущности обладают автоматически генерируемыми идентификаторами (благодаря аннотации `@GeneratedValue`), а в случае с `Customer` (листинг 6.5) имеет место отложенная выборка по отношению к `Address` (листинг 6.6).

Листинг 6.5. Сущность Customer с односторонней связью «один к одному» с Address

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  @OneToOne (fetch = FetchType.LAZY)
  @JoinColumn(name = "address_fk")
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Листинг 6.6. Сущность Address

```
@Entity
public class Address {
  @Id @GeneratedValue
  private Long id;
  private String street1;
  private String city;
  private String zipcode;
  private String country;
  // Конструкторы, геттеры, сеттеры
}
```

Эти две сущности будут отображены в структуру базы данных, которая показана на рис. 6.2. Обратите внимание, что столбец ADDRESS_FK является внешним ключом, ссылающимся на ADDRESS.

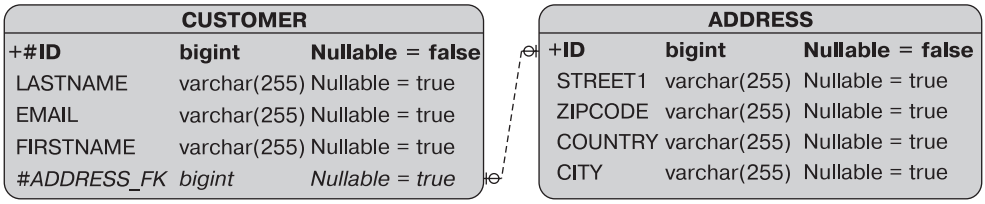


Рис. 6.2. T блицы CUSTOMER и ADDRESS

Для лучшей удобочитаемости фрагментов кода, приведенных в следующем подразделе, предполагается, что атрибут em имеет тип EntityManager, а tx — тип EntityTransaction.

Обеспечение постоянств сущности

Обеспечение постоянства сущности подразумевает вставку в базу данных информации, которой там еще нет (в противном случае будет сгенерировано исключение EntityExistsException). Для этого необходимо создать новый экземпляр сущности с использованием оператора new, задать значения для атрибутов, привязать одну сущность к другой, если есть ассоциации, и, наконец, вызвать метод EntityManager.persist(), как показано в варианте тестирования JUnit в листинге 6.7.

Листинг 6.7. Обеспечение постоянств Customer и Address

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
Address address = new Address("Ризердаун Роуд", "Лондон", "8QE", "Англия");
customer.setAddress(address);

tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();

assertNotNull(customer.getId());
assertNotNull(address.getId());
```

В листинге 6.7 customer и address — это всего лишь два объекта, которые располагаются в памяти виртуальной машины Java. Они оба станут сущностями, которые находятся под управлением, когда менеджер сущностей (переменная em) примет их в расчет, обеспечив постоянство (em.persist(customer)). На данном этапе оба объекта окажутся подходящими для вставки в базу данных. Когда произойдет фиксация транзакции (tx.commit()), информация будет сброшена в базу данных, строка, касающаяся адреса, будет вставлена в таблицу ADDRESS, а строка, которая касается клиента, — в таблицу CUSTOMER. Поскольку Customer является владельцем связи, его таблица содержит внешний ключ, ссылающийся

на ADDRESS. Выражения assertNotNull обеспечат проверку того, что обе сущности получили сгенерированные идентификаторы (благодаря поставщику постоянства, а также аннотациям @Id и @GeneratedValue).

Обратите внимание на порядок методов persist(): сначала обеспечивается постоянство Customer, а затем — постоянство Address. Если бы этот порядок был иным, то результат все равно оказался бы тем же. Ранее менеджер сущностей был охарактеризован как кэш первого уровня. Пока не произойдет фиксации транзакции, данные остаются в памяти, а доступ к базе данных отсутствует. Менеджер сущностей кэширует данные и, когда готов, сбрасывает их в том порядке, в каком ожидает основная база данных (с соблюдением ограничений целостности). Поскольку внешний ключ располагается в таблице CUSTOMER, сначала будет выполнен оператор insert для ADDRESS, а затем — для CUSTOMER.

ПРИМЕЧАНИЕ

Большинство сущностей в этой главе не реализуют интерфейс Serializable. Причина этого заключается в том, что сущностям не требуется быть сериализуемыми для того, чтобы оказалось возможным обеспечение их постоянства в базе данных. Они передаются по ссылке от одного метода к другому, и, когда необходимо обеспечить их постоянство, вызывается метод EntityManager.persist(). Но если вам потребуется передать сущности по значению (удаленный вызов, внешний EJB-контейнер и т. д.), то они должны будут реализовывать маркерный (не содержащий методов) интерфейс java.io.Serializable. Этот интерфейс говорит компилятору, что он должен позаботиться о том, чтобы все поля, связанные с классом-сущностью, обязательно были сериализуемыми. Благодаря этому любой экземпляр можно будет сериализовать в байтовый поток и передать с использованием удаленного вызова методов (Remote Method Invocation — RMI).

Поиск по идентифик тору

Для поиска сущности по ее идентификатору вы можете использовать два метода. Первый — EntityManager.find(), имеющий два параметра: класс-сущность и уникальный идентификатор (листинг 6.8). Если сущность удастся найти, то она будет возвращена; если сущность не удастся найти, то будет возвращено значение null.

Листинг 6.8. Поиск Customer по идентифик тору

```
Customer customer = em.find(Customer.class, 1234L)
if (customer!= null) {
  // Обработать объект
}
```

Второй метод — getReference() (листинг 6.9). Он очень схож с операцией find, поскольку имеет те же параметры, однако извлекает ссылку на сущность (с помощью ее первичного ключа), но не извлекает ее данных. Считайте его прокси для сущности, но не самой сущностью. Он предназначен для ситуаций, в которых требуется экземпляр сущности, находящейся под управлением, но не требуется никаких данных, кроме тех, что потенциально относятся к первичному ключу сущности, к которой осуществляется доступ. При использовании getReference() выборка данных о состоянии является отложенной, а это означает, что, если вы не осуществите доступ к состоянию до того, как сущность окажется отсоединенной, данных

уже может не быть там. Если сущность не удастся найти, то будет сгенерировано исключение EntityNotFoundException.

Листинг 6.9. Поиск Customer по ссылке

```
try {
  Customer customer = em.getReference(Customer.class, 1234L)
  // Обработать объект
} catch(EntityNotFoundException ex) {
  // Сущность не найдена
}
```

Уд ление сущности

Сущность можно удалить методом EntityManager.remove(). Как только сущность окажется удалена, она будет убрана из базы данных, отсоединена от менеджера сущностей и ее больше нельзя будет синхронизировать с базой данных. В плане Java-объектов эта сущность будет по-прежнему доступна, пока не окажется вне области видимости и сборщик мусора не уберет ее. В коде, приведенном в листинге 6.10, показано, как удалить объект после того, как он был создан.

Листинг 6.10. Созд ние и уд ление сущностей Customer и Address

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
Address address = new Address("Ризердаун Роуд", "Лондон", "8QE", "Англия");
customer.setAddress(address);
tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();

tx.begin();
em.remove(customer);
tx.commit();

// Данные удаляются из базы данных, но объект по-прежнему доступен
assertNotNull(customer);
```

Код из листинга 6.10 создает экземпляр Customer и Address, связывает их (customer.setAddress(address)) и обеспечивает их постоянство. В базе данных строка, касающаяся клиента, связывается со строкой, которая касается адреса, с помощью внешнего ключа; далее в коде удаляется только Customer. В зависимости от того, как сконфигурировано каскадирование (о нем мы поговорим позднее в этой главе), Address может остаться без какой-либо другой сущности, которая ссылается на нее, а строка, касающаяся адреса, станет изолированной.

Уд ление сущностей-сирот

Изолированные сущности нежелательны, поскольку они приводят к тому, что в базе данных есть строки, на которые не ссылается какая-либо другая таблица, при этом они лишены средств доступа. При использовании JPA вы можете проинформировать поставщика постоянства о необходимости автоматически удалять такие сущ-

ности или каскадировать операцию удаления. Если целевая сущность (Address) находится в частном владении источника (Customer), подразумевая, что целью никогда не может владеть несколько источников, а этот источник окажется удален приложением, то поставщик должен будет удалить и цель.

Ассоциации, которые определены как «один к одному» или «один ко многим», поддерживают использование параметра orphanRemoval. Чтобы включить этот параметр в пример, взглянем на то, как добавить элемент orphanRemoval=true в аннотацию @OneToOne (листинг 6.11).

Листинг 6.11. Сущность Customer и уд ление изолиров нной сущности Address

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  @OneToOne (fetch = FetchType.LAZY, orphanRemoval=true)
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

При таком отображении код, приведенный в листинге 6.10, автоматически удалит сущность Address, если окажется удалена сущность Customer либо если связь будет разорвана (при присвоении атрибуту address значения null или удалении дочерней сущности из коллекции при связи «один ко многим»). Операция удаления выполняется во время операции сброса (когда имеет место фиксация транзакции).

Синхрониз ция с б зой д нных

До настоящего момента синхронизация с базой данных осуществлялась во время фиксации. Менеджер сущностей является кэшем первого уровня, ожидающим фиксации транзакции, чтобы сбросить информацию в базу данных. Но что будет, если потребуется вставить Customer и Address?

```
tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();
```

Всем ожидаемым изменениям требуется SQL-оператор. В данном случае два оператора insert генерируются и становятся постоянными, только когда происходит фиксация транзакции в базе данных. Для большинства приложений этой автоматической синхронизации данных будет достаточно. Хотя неизвестно, в какой именно момент времени поставщик на самом деле сбрасывает изменения в базу данных, вы можете быть уверены, что это случится, когда произойдет фиксация транзакции. База данных синхронизируется с сущностями в контексте постоянства, однако данные могут быть сброшены (flush) явным образом в базу либо сущности могут быть обновлены с использованием информации из базы

(refresh). Если данные будут сброшены в базу данных в определенный момент, а позднее в коде приложение вызовет метод rollback(), то сброшенные данные будут изъяты из базы.

Сброс сущности. С помощью метода EntityManager.flush() поставщика постоянства можно явным образом заставить сбрасывать данные в базу, но не будет фиксации транзакции. Это позволяет разработчику вручную инициировать тот же самый процесс, который менеджер сущностей использует внутренне для сброса контекста постоянства.

```
tx.begin();
em.persist(customer);
em.flush();
em.persist(address);
tx.commit();
```

В приведенном коде можно отметить две любопытные вещи. Первая заключается в том, что em.flush() не станет дожидаться фиксации транзакции и заставит поставщика сбросить контекст постоянства. Оператор insert будет сгенерирован и выполнен при сбросе. Вторая вещь состоит в том, что этот код не будет работать из-за ограничения целостности. Без явного сброса менеджер сущностей станет кэшировать все изменения, а также упорядочивать и последовательно вносить их в базу данных. При явном сбросе будет выполнен оператор insert в отношении CUSTOMER, однако ограничение целостности в случае с внешним ключом ADDRESS окажется нарушено (столбец ADDRESS_FK в CUSTOMER). Это приведет к откату транзакции. Откат также произойдет в случае с уже сброшенными данными. Явные сбросы следует использовать осторожно и только при необходимости.

Обновление сущности. Метод refresh() применяется для синхронизации данных в направлении, противоположном сбросу, то есть он перезаписывает текущее состояние сущности, которая находится под управлением, с использованием данных в таком виде, в каком они присутствуют в базе данных. Типичный случай — когда вы используете метод EntityManager.refresh() для отмены изменений, внесенных в сущность только в памяти. Фрагмент варианта тестирования, который приведен в листинге 6.12, обеспечивает поиск Customer по идентификатору, изменение значения его firstName и отмену этого изменения с помощью метода refresh().

Листинг 6.12. Обновление сущности Customer из б зы д нных

```
Customer customer = em.find(Customer.class, 1234L)
assertEquals(customer.getFirstName(), "Энтони");
customer.setFirstName("Уильям");
em.refresh(customer);
assertEquals(customer.getFirstName(), "Энтони");");
```

Содержимое контекст постоянств

В контексте постоянства содержатся сущности, которые находятся под управлением. Используя интерфейс EntityManager, вы можете проверить, находится ли сущность под управлением, отсоединить ее или очистить контекст постоянства от всех сущностей.

Contains. Сущности либо управляются менеджером сущностей, либо нет. Метод `EntityManager.contains()` возвращает логическое значение и позволяет вам проверить, находится ли экземпляр определенной сущности в настоящее время под управлением менеджера в контексте постоянства. В варианте тестирования, приведенном в листинге 6.13, показано, что обеспечивается постоянство `Customer`, и вы можете незамедлительно проверить, находится ли эта сущность под управлением (`em.contains(customer)`). Ответ в данном случае звучит как «да». Далее происходит вызов метода `remove()` и сущность удаляется из базы данных, а также из контекста постоянства (`em.contains(customer)` возвращает `false`).

Листинг 6.13. В ри нт тестиров ния для проверки н предмет того, присутствует ли сущность Customer в контексте постоянств

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
tx.begin();
em.persist(customer);
tx.commit();
assertTrue(em.contains(customer));

tx.begin();
em.remove(customer);
tx.commit();
assertFalse(em.contains(customer));
```

Очистка и отсоединение. Метод `clear()` прост: он очищает контекст постоянства, приводя к тому, что все сущности, которые находятся под управлением, оказываются отсоединенными. Метод `detach(Object entity)` убирает определенную сущность из контекста постоянства. После такого «выселения», изменения, внесенные в эту сущность, не будут синхронизированы с базой данных. Код, приведенный в листинге 6.14, создает сущность, проверяет, находится ли она под управлением, отсоединяет ее от контекста постоянства, а также проверяет, была ли она отсоединена.

Листинг 6.14. Проверк , присутствует ли сущность Customer в контексте постоянств

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
tx.begin();
em.persist(customer);
tx.commit();
assertTrue(em.contains(customer));

em.detach(customer);
assertFalse(em.contains(customer));
```

Слияние сущности

Отсоединенная сущность больше не ассоциирована с контекстом постоянства. Если вы хотите управлять ею, то вам потребуется снова присоединить эту сущность (то есть обеспечить ее слияние). Обратимся к примеру сущности, которую необходимо вывести на JSF-странице. Сущность, сначала загружаемая из базы данных на постоянный уровень (находящаяся под управлением), возвращается из вызова локального EJB-компонента (она является отсоединенной, поскольку контекст

транзакции перестает существовать), выводится на уровне представления (все еще являясь отсоединенной), а затем возвращается для обновления в базу данных. Однако в тот момент сущность является отсоединенной, и необходимо присоединить ее снова или обеспечить слияние, чтобы синхронизировать ее состояние с базой данных.

Эта ситуация симулируется в листинге 6.15 путем очистки контекста постоянства (`em.clear()`), что приводит к отсоединению сущности.

Листинг 6.15. Очистк контекст постоянств и слияние сущности

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
tx.begin();
em.persist(customer);
tx.commit();
em.clear();
// Задает новое значение для отсоединенной сущности
customer.setFirstName("Уильям");

tx.begin();
em.merge(customer);
tx.commit();
```

Код, приведенный в листинге 6.15, создает и обеспечивает постоянство `Customer`. Вызов `em.clear()` форсирует отсоединение сущности `Customer`, однако отсоединенные сущности продолжают присутствовать, но уже вне контекста постоянства, в котором они находились, а синхронизация их состояния с состоянием базы данных больше не обеспечивается. Именно это происходит при использовании `customer.setFirstName("William")`. Код выполняется в отношении отсоединенной сущности, и данные не обновляются в базе данных. Для репликации этого изменения в базу данных вам потребуется снова присоединить сущность (то есть обеспечить ее слияние) с помощью `em.merge(customer)` в рамках транзакции.

Обновление сущности

Обновление сущности является простой операцией, но в то же время может оказаться сложной для понимания. Как вы только что видели, `EntityManager.merge()` можно использовать для присоединения сущности и синхронизации ее состояния с базой данных. Однако если сущность находится в данный момент под управлением, внесенные в нее изменения не будут автоматически отражены в базе данных. В противном случае вам потребуется явным образом вызвать `merge()`.

В листинге 6.16 демонстрируется обеспечение постоянства `Customer` с `setFirstName`, для которого задано значение `Antony`. Когда вы вызываете метод `em.persist()`, сущность находится под управлением, поэтому любые изменения, внесенные в эту сущность, будут синхронизированы с базой данных. Когда вы вызываете метод `setFirstName()`, состояние сущности изменяется. Менеджер сущностей кэширует все действия, начиная с `tx.begin()`, и обеспечивает соответствующую синхронизацию при фиксации.

Листинг 6.16. Обновление setFirstName сущности Customer

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
tx.begin();
em.persist(customer);

customer.setFirstName("Уильям");
tx.commit();
```

К ск диров ние событий

По умолчанию любая операция выполняется менеджером сущностей только в отношении сущности, которая предусмотрена в качестве аргумента при этой операции. Но порой, когда операция выполняется в отношении сущности, возникает необходимость распространить ее на соответствующие ассоциации. Это называется *каскадированием события*. Приводившиеся до сих пор примеры опирались на поведение каскадирования по умолчанию, а не на настроенное поведение. В листинге 6.17 показано, что для создания Customer нужно сгенерировать экземпляры сущностей Customer и Address, связать их (customer.setAddress(address)), а затем обеспечить постоянство этих двух экземпляров.

Листинг 6.17. Обеспечение постоянств Customer и Address

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
Address address = new Address("Ризердаун Роуд", "Лондон", "8QE", "Англия");
customer.setAddress(address);

tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();
```

Поскольку между Customer и Address есть связь, вы могли бы каскадировать действие persist от Customer к Address. Это означало бы, что вызов em.persist(customer) привел бы к каскадированию события PERSIST к сущности Address, если она допускает распространение события такого типа. Тогда вы могли бы сократить код и избавиться от em.persist(address), как показано в листинге 6.18.

Листинг 6.18. К ск диров ние события PERSIST к Address

```
Customer customer = new Customer("Энтони", "Балла", "tballa@mail.com");
Address address = new Address("Ризердаун Роуд", "Лондон", "8QE", "Англия");
customer.setAddress(address);

tx.begin();
em.persist(customer);
tx.commit();
```

Без каскадирования было бы обеспечено постоянство Customer, но не Address. Каскадирование события возможно при изменении отображения связи. У аннотаций @OneToOne, @OneToMany, @ManyToOne и @ManyToMany есть атрибут cascade, который принимает массив событий для каскадирования, а также событие PERSIST, которое можно каскадировать, как и событие REMOVE (широко используемое для выполнения

каскадных удалений). Для этого вам потребуется изменить отображение сущности Customer (листинг 6.19) и добавить атрибут cascade в аннотацию @OneToOne в случае с Address.

Листинг 6.19. Сущность Customer и к ск диров ние событий PERSIST и REMOVE

```
@Entity
public class Customer {
@Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  @OneToOne (fetch = FetchType.LAZY, cascade = {CascadeType.PERSIST,
CascadeType.REMOVE})
  @JoinColumn(name = "address_fk")
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Вы можете выбирать из нескольких событий для каскадирования к цели ассоциации (эти события приведены в табл. 6.3) и даже каскадировать их все, используя тип CascadeType.ALL.

Т блиц 6.3. Возможные события для к ск диров ния

CascadeType	Опис ние
PERSIST	Каскадирует операции по обеспечению постоянства к цели ассоциации
REMOVE	Каскадирует операции удаления к цели ассоциации
MERGE	Каскадирует операции объединения к цели ассоциации
REFRESH	Каскадирует операции обновления к цели ассоциации
DETACH	Каскадирует операции отсоединения к цели ассоциации
ALL	Объявляет, что должны быть каскадированы все предыдущие операции

JPQL

Вы только что видели, как манипулировать сущностями по отдельности, используя API-интерфейс EntityManager. Вы уже знаете, как искать сущность по идентификатору, удалять ее, обновлять ее атрибуты и т. д. Однако поиск сущности по идентификатору довольно сильно ограничивает вас, поскольку вы можете извлечь только одну сущность, используя ее уникальный идентификатор. На практике вам может потребоваться извлечь сущность, исходя из иных критериев (имени, ISBN-номера и т. д.), либо извлечь набор сущностей на основе других критериев (например, все сущности, связанные с клиентами, проживающими в США). Эта возможность присуща реляционным базам данных, а у JPA есть язык, который обеспечивает это взаимодействие, — JPQL.

JPQL предназначен для определения поисков постоянных сущностей, которые не зависят от основной базы данных. JPQL — это язык запросов, укоренившийся в синтаксисе SQL. Он является стандартным языком для выполнения запросов к базам данных. Однако основное отличие состоит в том, что при использовании SQL вы получаете результаты в виде строк и столбцов (таблиц), а в случае применения JPQL — в виде сущности или коллекции сущностей. Синтаксис JPQL является объектно-ориентированным и, следовательно, более легким для понимания разработчиками, чей опыт ограничивается объектно-ориентированными языками. Разработчики управляют своей доменной моделью сущностей, а не структурой таблицы, используя точечную нотацию (например, `myClass.myAttribute`).

«За кулисами» JPQL применяет механизм отображения для преобразования JPQL-запросов в такие, которые будут понятны базам данных SQL. Запрос выполняется в отношении основной базы данных с использованием SQL- и JDBC-вызовов, после чего следует присваивание значений атрибутам экземпляров сущностей и их возврат приложению — все происходит очень простым и эффективным образом с применением богатого синтаксиса запросов.

Самый простой JPQL-запрос обеспечивает выборку всех экземпляров одной сущности:

```
SELECT b
FROM Book b
```

Если вы знаете SQL, то этот код должен показаться вам знакомым. Вместо выборки из таблицы JPQL производит выборку сущностей, в данном случае той, что носит имя `Book`. Кроме того, используется оператор `FROM` для наделения сущности псевдонимом: `b` является псевдонимом `Book`. Оператор `SELECT` запроса указывает на то, что типом результата запроса является сущность `b` (`Book`). Выполнение этого оператора приведет к получению списка, в который будет входить нуль или более экземпляров `Book`.

Чтобы ограничить результаты, добавьте критерии поиска. Вы можете воспользоваться оператором `WHERE`, как показано далее:

```
SELECT b
FROM Book b
WHERE b.title = 'H2G2'
```

Псевдоним предназначен для навигации по атрибутам сущности с применением оператора-точки. Поскольку сущность `Book` обладает постоянным атрибутом, носящим имя `title` и тип `String`, `b.title` ссылается на атрибут `title` сущности `Book`. Выполнение этого оператора приведет к получению списка, в который будет входить нуль или более экземпляров `Book` со значением `title` в виде `H2G2`.

Самый простой запрос на выборку состоит из двух обязательных частей — операторов `SELECT` и `FROM`. `SELECT` определяет формат результатов запроса. Оператор `FROM` определяет сущность или сущности, из которых будут получаться результаты, а необязательные операторы `WHERE`, `ORDER BY`, `GROUP BY` и `HAVING` могут быть использованы для ограничения или упорядочения результатов запроса. Листинг 6.20 демонстрирует упрощенный синтаксис JPQL-оператора.

Листинг 6.20. Упрощенный синт ксис JPQL-опер тор

```
SELECT <оператор SELECT>
FROM <оператор FROM>
[WHERE <оператор WHERE>]
[ORDER BY <оператор ORDER BY>]
[GROUP BY <оператор GROUP BY>]
[HAVING <оператор HAVING>]
```

Листинг 6.20 определяет оператор SELECT, а операторы DELETE и UPDATE также могут быть использованы для выполнения операций удаления и обновления в отношении множественных экземпляров заданного класса-сущности.

SELECT

Оператор SELECT придерживается синтаксиса выражений путей и приводит к результату в одной из следующих форм: сущность, атрибут сущности, выражение-конструктор, агрегатная функция или их последовательность. Выражения путей являются строительными блоками запросов и используются для перемещения по атрибутам сущности или связям сущностей (либо коллекции сущностей) с помощью точечной (.) навигации с применением следующего синтаксиса:

```
SELECT [DISTINCT] <выражение> [[AS] <идентификационная переменная>]
expression ::= { NEW | TREAT | AVG | MAX | MIN | SUM | COUNT }
```

Простой оператор SELECT возвращает сущность. Например, если сущность Customer содержит псевдоним c, то SELECT возвратит сущность или список сущностей:

```
SELECT c
FROM Customer c
```

Однако оператор SELECT также может возвращать атрибуты. Если у Customer имеется firstName, то SELECT c.firstName возвратит строку или коллекцию строк с firstName.

Чтобы извлечь firstName и lastName сущности Customer, вам потребуется сгенерировать список, который будет включать два следующих атрибута:

```
SELECT c.firstName, c.lastName
FROM Customer c
```

С выходом версии JPA 2.0 появилась возможность извлекать атрибуты в зависимости от условий (с использованием выражения CASE WHEN ... THEN ... ELSE ... END). Например, вместо извлечения значения, говорящего о цене книги, оператор может возвратить расчет цены (например, с 50%-ной скидкой) в зависимости от издателя (например, с 50%-ной скидкой на книги от «Apress» и 20%-ной скидкой на все прочие книги).

```
SELECT CASE b.editor WHEN 'Apress'
                    THEN b.price * 0.5
                    ELSE b.price * 0.8
       END
FROM Book b
```

Если сущность Customer связана отношением «один к одному» с Address, то c.address будет ссылаться на адрес клиента, а в результате выполнения приведенного далее запроса будет возвращен не список клиентов, а список адресов:

```
SELECT c.address
FROM Customer c
```

Навигационные выражения можно объединять в цепочку для обхода комплексных EntityGraph. Благодаря этой методике можно создавать выражения путей, например c.address.country.code, ссылающиеся на код страны в адресе клиента:

```
SELECT c.address.country.code
FROM Customer c
```

В выражении SELECT можно применять конструктор для возврата экземпляра Java-класса, который инициализируется с использованием результата запроса. Класс не обязан быть сущностью, однако конструктор должен быть полностью уточненным и сочетаться с атрибутами.

```
SELECT NEW org.agoncal.javaee7.CustomerDTO(c.firstName, c.lastName, c.address.
street1)
FROM Customer c
```

Результатом этого запроса будет список объектов CustomerDTO, экземпляры которых были созданы с применением оператора new и инициализированы с использованием имен, фамилий и улиц проживания клиентов.

Выполнение показанных далее запросов приведет к возврату одного значения или коллекции, в которую будет входить нуль и более сущностей (или атрибутов), включая дубликаты. Для удаления дубликатов необходимо воспользоваться оператором DISTINCT.

```
SELECT DISTINCT c
FROM Customer c

SELECT DISTINCT c.firstName
FROM Customer c
```

Результатом запроса может оказаться результат выполнения агрегатной функции, примененной в выражении пути. В операторе SELECT могут быть использованы следующие агрегатные функции: AVG, COUNT, MAX, MIN, SUM. Результаты можно сгруппировать оператором GROUP BY и профильтровать оператором HAVING.

```
SELECT COUNT(c)
FROM Customer c
```

Скалярные выражения тоже могут быть использованы в операторе SELECT запроса, равно как и в операторах WHERE и HAVING. Эти выражения можно применять в отношении числовых (ABS, SQRT, MOD, SIZE, INDEX) и строковых значений (CONCAT, SUBSTRING, TRIM, LOWER, UPPER, LENGTH, LOCATE), а также значений даты/времени (CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP).

FROM

Оператор FROM определяет сущности путем объявления идентификационных переменных. *Идентификационная переменная*, или *псевдоним*, — это идентификатор, который может быть использован в других операторах (SELECT, WHERE и т. д.). Синтаксис оператора FROM включает сущность и псевдоним. В приведенном далее примере Customer является сущностью, а c — идентификационной переменной:

```
SELECT c
FROM Customer c
```

WHERE

Оператор WHERE состоит из условного выражения, используемого для ограничения результатов выполнения оператора SELECT, UPDATE или DELETE. Оператор WHERE может быть простым выражением либо набором условных выражений, применяемых для фильтрации результатов запроса.

Самый простой способ ограничить результаты запроса — использовать атрибут сущности. Например, приведенный далее запрос обеспечивает выборку всех клиентов, для которых firstName имеет значение Винсент:

```
SELECT c
FROM Customer c
WHERE c.firstName = 'Винсент'
```

Вы можете еще больше ограничить результаты запросов, используя логические операторы AND и OR. В приведенном далее примере AND задействуется для выборки всех клиентов, для которых firstName имеет значение Винсент, а country — значение Франция:

```
SELECT c
FROM Customer c
WHERE c.firstName = 'Винсент' AND c.address.country = 'Франция'
```

В операторе WHERE также могут применяться операторы сравнения: =, >, >=, <, <=, <>, [NOT] BETWEEN, [NOT] LIKE, [NOT] IN, IS [NOT] NULL, IS [NOT] EMPTY, [NOT] MEMBER [OF]. Далее приведен пример использования двух из этих операторов:

```
SELECT c
FROM Customer c
WHERE c.age > 18

SELECT c
FROM Customer c
WHERE c.age NOT BETWEEN 40 AND 50

SELECT c
FROM Customer c
WHERE c.address.country IN ('США', 'Португалия')
```

Выражение LIKE состоит из строки и опциональных знаков переключения кода, которые определяют условия соответствия: знак подчеркивания (_) для метасим-

волов, состоящих из одного знака, и знак процента (%), если речь идет о метасимволах, состоящих из множества знаков.

```
SELECT c
FROM Customer c
WHERE c.email LIKE '%mail.com'
```

Привязк п р метров

Операторы WHERE, приводившиеся в этой книге до настоящего момента, задействовали только фиксированные значения. В приложениях запросы часто зависят от параметров. JPQL поддерживает синтаксис для привязки параметров двух типов, делая возможным внесение динамических изменений в ограничительный оператор запроса. Это позиционные и именованные параметры.

Позиционные параметры обозначаются знаком вопроса (?), за которым следует целочисленное значение (например, ?1). При выполнении запроса необходимо указать номера параметров, которые нужно заменить.

```
SELECT c
FROM Customer c
WHERE c.firstName = ?1 AND c.address.country = ?2
```

Именованные параметры обозначаются строковым идентификатором с префиксом в виде знака двоеточия (:). При выполнении запроса необходимо указать имена параметров, которые следует заменить.

```
SELECT c
FROM Customer c
WHERE c.firstName = :fname AND c.address.country = :country
```

В разделе «Запросы», приведенном далее в этой главе, вы увидите, как приложение осуществляет привязку параметров.

Подз просы

Подзапрос — это запрос SELECT, который вложен в условное выражение WHERE или HAVING. Результаты подзапроса оцениваются и интерпретируются в условном выражении главного запроса. Для извлечения информации о самых молодых клиентах из базы данных сначала нужно выполнить подзапрос с использованием MIN(age), а затем его результаты будут оценены в главном запросе.

```
SELECT c
FROM Customer c
WHERE c.age = (SELECT MIN(cust. age) FROM Customer cust ))
```

ORDER BY

Оператор ORDER BY позволяет упорядочивать сущности или значения, возвращаемые в результате выполнения запроса SELECT. Упорядочение применяется для атрибута сущности, который указан в этом предложении и за которым следует ключевое слово ASC или DESC. Ключевое слово ASC определяет, что будет применяться упорядочение по возрастанию; ключевое слово DESC, которое является его противоположностью,

определяет, что применяется упорядочение по убыванию. Сортировка по возрастанию задействуется по умолчанию, и ее можно не указывать.

```
SELECT c
FROM Customer c
WHERE c.age > 18
ORDER BY c.age DESC
```

Для уточнения порядка сортировки также можно использовать множественные выражения:

```
SELECT c
FROM Customer c
WHERE c.age > 18
ORDER BY c.age DESC, c.address.country ASC
```

GROUP BY и HAVING

Конструкция GROUP BY делает возможной агрегацию результирующих значений согласно набору свойств. Сущности делятся на группы в зависимости от значений связанных с ними полей, которые указаны в предложении GROUP BY. Чтобы сгруппировать клиентов в соответствии со страной их проживания и сосчитать их, используйте приведенный далее запрос:

```
SELECT c.address.country, count(c)
FROM Customer c
GROUP BY c.address.country
```

GROUP BY определяет выражения группировки (c.address.country), с использованием которых будет осуществляться агрегация и подсчет результатов (count(c)). Следует отметить, что выражения, присутствующие в операторе GROUP BY, также должны быть в операторе SELECT.

Оператор HAVING определяет соответствующий фильтр, который применяется после того, как будут сгруппированы результаты запроса, подобно вторичному оператору WHERE, обеспечивая фильтрацию результатов GROUP BY. Если воспользоваться предыдущим запросом, добавив HAVING, то могут быть возвращены результаты, в случае с которыми country будет иметь значение, отличное от Англия.

```
SELECT c.address.country, count(c)
FROM Customer c
GROUP BY c.address.country
HAVING c.address.country <> 'Англия'
```

GROUP BY и HAVING могут быть использованы только в операторе SELECT (а не в DELETE или UPDATE).

M ссовое уд ление

Вы уже знаете, как удалить сущность с помощью метода EntityManager.remove() и выполнить запрос к базе данных для извлечения списка сущностей, которые соответствуют определенным критериям. Чтобы удалить список сущностей, вы можете выполнить запрос, произвести итерацию по этому списку и удалить каждую сущ-

ность по отдельности. Хотя это допустимый алгоритм, он ужасен в плане производительности (слишком много раз придется получать доступ к базе данных). Есть более подходящий способ решить эту задачу — произвести массовое удаление.

JPQL выполняет операции массового удаления в отношении множественных экземпляров определенного класса-сущности. Такой подход используется для удаления большого количества сущностей в рамках одной операции. Оператор DELETE похож на оператор SELECT, поскольку может включать ограничительное выражение WHERE и задействовать параметры. В результате возвращается ряд экземпляров сущности, затронутых операцией. Синтаксис оператора DELETE выглядит следующим образом:

```
DELETE FROM <имя сущности> [[AS] <идентификационная переменная>]
[WHERE <выражение WHERE>]
```

Чтобы в качестве примера удалить всех клиентов моложе 18 лет, вы можете прибегнуть к массовому удалению с помощью оператора DELETE.

```
DELETE FROM Customer c
WHERE c.age < 18
```

Массовое обновление

Для массового обновления сущностей необходимо обратиться к оператору UPDATE, задав значение для одного или нескольких атрибутов субъекта сущности согласно условиям в выражении WHERE. Синтаксис оператора UPDATE выглядит следующим образом:

```
UPDATE <имя сущности> [[AS] <идентификационная переменная>]
SET <оператор UPDATE> {, <оператор UPDATE>}*
[WHERE <выражение WHERE>]
```

Вместо того чтобы удалять всех молодых клиентов, для них можно изменить значение firstName на TOO YOUNG с помощью приведенного далее оператора:

```
UPDATE Customer c
SET c.firstName = 'TOO YOUNG'
WHERE c.age < 18
```

Запросы

Вы уже видели синтаксис JPQL и поняли, как описывать задачи с использованием разных операторов (SELECT, FROM, WHERE и т. д.). Но как включить JPQL-оператор в свое приложение? Ответ: с помощью запросов. В JPA 2.1 имеется пять отличающихся типов запросов, которые могут быть использованы в коде, причем каждый из них — с разным назначением.

- *Динамические запросы* — это самая простая форма, куда входит всего лишь строка JPQL-запроса, динамически генерируемого во время выполнения.
- *Именованные запросы* — являются статическими и неизменяемыми.
- *Criteria API* — в JPA 2.0 была представлена концепция объектно-ориентированного Query API.

- *«Родные» запросы* — запросы этого типа пригодны для выполнения «родных» SQL-операторов вместо JPQL-операторов.
- *Запросы к хранимым процедурам* — JPA 2.1 привносит новый API для вызова хранимых процедур.

Центральной точкой, обуславливающей выбор из этих пяти типов запросов, является интерфейс менеджера сущностей, который обладает несколькими фабричными методами, приведенными в табл. 6.4, возвращая либо интерфейс `Query`, либо `TypedQuery`, либо `StoredProcedureQuery` (`TypedQuery` и `StoredProcedureQuery` расширяют `Query`). Интерфейс `Query` используется в тех случаях, когда типом результата является `Object`, а `TypedQuery` применяется, когда предпочтителен типизированный результат. `StoredProcedureQuery` задействуется для контроля выполнения запросов к хранимым процедурам.

Т блиц 6.4. Методы менеджер сущностей для генериров ния з просов

Метод	Опис ние
Query createQuery(String jpqlString)	Создает экземпляр Query для выполнения JPQL-оператора для динамических запросов
Query createNamedQuery(String name)	Создает экземпляр Query для выполнения именованного запроса (с использованием JPQL или «родного» SQL)
Query createNativeQuery(String sqlString)	Создает экземпляр Query для выполнения «родного» SQL-оператора
Query createNativeQuery(String sqlString, Class resultClass)	«Родной» запрос, передающий класс ожидаемых результатов
Query createNativeQuery(String sqlString, String resultSetMapping)	«Родной» запрос, передающий отображение результирующего набора
<T> TypedQuery<T> createQuery(CriteriaQuery<T> criteriaQuery)	Создает экземпляр TypedQuery для выполнения запроса с использованием критериев
<T> TypedQuery<T> createQuery(String jpqlString, Class<T> resultClass)	Типизированный запрос, передающий класс ожидаемых результатов
<T> TypedQuery<T> createNamedQuery(String name, Class<T> resultClass)	Типизированный запрос, передающий класс ожидаемых результатов
StoredProcedureQuery createStored-ProcedureQuery(String procedureName)	Создает StoredProcedureQuery для выполнения хранимой процедуры в базе данных
StoredProcedureQuery createStored-ProcedureQuery(String procedureName, Class... resultClasses)	Запрос к хранимой процедуре, передающий классы, в которые будут отображаться результирующие наборы
StoredProcedureQuery createStored-ProcedureQuery(String procedureName, String... resultSetMappings)	Запрос к хранимой процедуре, передающий отображение результирующих наборов
StoredProcedureQuery createNamedStored-ProcedureQuery(String name)	Генерирует запрос к именованной хранимой процедуре

При получении реализации интерфейса Query, TypedQuery или StoredProcedureQuery с помощью одного из фабричных методов в интерфейсе менеджера сущностей она будет контролироваться богатым API. API Query, показанный в листинге 6.21, задействуется для выполнения статических (то есть именованных) и динамических запросов с применением JPQL, а также «родных» запросов с использованием SQL. Кроме того, API Query поддерживает привязку параметров и управление разбиением на страницы.

Листинг 6.21. Query API

```
public interface Query {
  // Выполняет запрос и возвращает результат
  List getResultList();
  Object getSingleResult();
  int executeUpdate();

  // Задает параметры для запроса
  Query setParameter(String name, Object value);
  Query setParameter(String name, Date value, TemporalType temporalType);
  Query setParameter(String name, Calendar value, TemporalType temporalType);
  Query setParameter(int position, Object value);
  Query setParameter(int position, Date value, TemporalType temporalType);
  Query setParameter(int position, Calendar value, TemporalType temporalType);
  <T> Query setParameter(Parameter<T> param, T value);
  Query setParameter(Parameter<Date> param, Date value, TemporalType
temporalType);
  Query setParameter(Parameter<Calendar> param, Calendar value, TemporalType
temporalType);

  // Извлекает параметры посредством запроса
  Set<Parameter<?>> getParameters();
  Parameter<?> getParameter(String name);
  Parameter<?> getParameter(int position);
  <T> Parameter<T> getParameter(String name, Class<T> type);
  <T> Parameter<T> getParameter(int position, Class<T> type);
  boolean isBound(Parameter<?> param);
  <T> T getParameterValue(Parameter<T> param);
  Object getParameterValue(String name);
  Object getParameterValue(int position);

  // Ограничивает количество результатов, возвращаемых запросом
  Query setMaxResults(int maxResult);
  int getMaxResults();
  Query setFirstResult(int startPosition);
  int getFirstResult();

  // Задает и извлекает подсказки в запросах
  Query setHint(String hintName, Object value);
  Map<String, Object> getHints();

  // Задает тип режима сброса для использования при выполнении запроса
```

```
    Query setFlushMode(FlushModeType flushMode);
    FlushModeType getFlushMode();

    // Задает тип режима блокировки для использования при выполнении запроса
    Query setLockMode(LockModeType lockMode);
    LockModeType getLockMode();

    // Разрешает доступ к API, специфичному для поставщика
    <T> T unwrap(Class<T> cls);
}
```

Методы, которые главным образом используются в этом API, обеспечивают выполнение запроса как такового. Чтобы выполнить запрос SELECT, вам придется сделать выбор между двумя методами в зависимости от требуемого результата.

- Метод getResultList() выполняет запрос и возвращает список результатов (сущностей, атрибутов, выражений и т. д.).
- Метод getSingleResult() выполняет запрос и возвращает одиночный результат (генерирует исключение NonUniqueResultException при обнаружении нескольких результатов).

Для осуществления операции обновления или удаления метод executeUpdate() выполняет массовый запрос и возвращает несколько сущностей, затронутых при выполнении запроса.

Как вы уже видели в разделе «JPQL» ранее, при запросе могут использоваться параметры, которые являются либо именованными (например, :myParam), либо позиционными (например, ?1). API Query определяет несколько методов setParameter для задания параметров перед выполнением запроса.

Когда вы выполняете запрос, он может возвратить большое количество результатов. В зависимости от приложения они могут быть обработаны все вместе либо порциями (например, веб-приложение выводит только десять строк за один раз). Для управления разбиением на страницы интерфейс Query определяет методы setFirstResult() и setMaxResults(), позволяющие указывать соответственно первый получаемый результат (с нумерацией, начинающейся с нуля) и максимальное количество результатов для возврата относительно этой точки.

Режим сброса является для поставщика постоянства индикатором того, как следует поступать с ожидаемыми изменениями и запросами. Есть две возможные настройки режима сброса: AUTO и COMMIT. AUTO (используемая по умолчанию) означает, что поставщик постоянства обеспечивает, что ожидаемые изменения будут видимыми при обработке запроса. COMMIT используется, когда эффект от обновления сущностей не перекрывается измененными данными в контексте постоянства.

Запросы можно блокировать с помощью метода setLockMode(LockModeType).

В последующих разделах демонстрируется пять разных типов запросов с использованием описанных здесь методов.

Дин мические з просы

Динамические запросы генерируются на лету по мере того, как это требуется приложению. Для создания динамического запроса используйте метод EntityMana-

ger.createQuery(), принимающий в качестве параметра строку, которая представляет JPQL-запрос.

В приведенном далее коде JPQL-запрос обеспечивает выборку всех клиентов из базы данных. Результатом этого запроса является список, так что, когда вы вызываете метод getResultList(), он возвращает список экземпляров сущности Customer (List<Customer>). Однако если вы знаете, что ваш запрос возвращает только одну сущность, используйте метод getSingleResult(). Он возвращает одну сущность и избегает работы по извлечению данных в виде списка.

```
Query query = em.createQuery("SELECT c FROM Customer c");
List<Customer> customers = query.getResultList();
```

Этот JPQL-запрос возвращает объект Query. Когда вы вызываете метод query.getResultList(), он возвращает список нетипизированных объектов. Если вы хотите, чтобы этот же запрос возвратил список типа Customer, то вам нужно использовать TypedQuery следующим образом:

```
TypedQuery<Customer> query =
    em.createQuery("SELECT c FROM Customer c", Customer.class);
List<Customer> customers = query.getResultList();
```

Эта строка запроса также может динамически создаваться приложением, которое затем способно сгенерировать заранее не известный комплексный запрос во время выполнения. Конкатенация строк используется для динамического генерирования запроса в зависимости от соответствующих критериев.

```
String jpqlQuery = "SELECT c FROM Customer c";
if (someCriteria)
    jpqlQuery += "WHERE c.firstName = 'Бетти'";
query = em.createQuery(jpqlQuery);
List<Customer> customers = query.getResultList();
```

Предыдущий запрос извлекает клиентов, для которых firstName имеет значение Бетти, однако вы можете захотеть передать параметр для firstName. Есть два возможных варианта для этого: с использованием имен или позиций. В приведенном далее примере я использую именованный параметр :fname (обратите внимание на символ «:») в запросе и привязываю его с помощью метода setParameter:

```
query = em.createQuery("SELECT c FROM Customer c where c.firstName = :fname");
query.setParameter("fname", "Бетти");
List<Customer> customers = query.getResultList();
```

Обратите внимание, что имя параметра fname не содержит двоеточия, используемого в запросе. Код, включающий в себя позиционный параметр, выглядел бы следующим образом:

```
query = em.createQuery("SELECT c FROM Customer c where c.firstName = ?1");
query.setParameter(1, "Бетти");
List<Customer> customers = query.getResultList();
```

Если вам потребуется прибегнуть к разбиению на страницы для вывода списка клиентов частями по десять человек, то можете воспользоваться методом setMaxResults, как показано далее:

```
query = em.createQuery("SELECT c FROM Customer c", Customer.class);
query.setMaxResults(10);
List<Customer> customers = query.getResultList();
```

В случае с динамическими запросами необходимо принимать во внимание, во что обходится преобразование JPQL-строк в SQL-операторы во время выполнения. Поскольку запрос генерируется динамически и его нельзя предсказать, поставщику постоянства приходится разбирать JPQL-строку, извлекать метаданные объектно-реляционного отображения и генерировать эквивалентный SQL-оператор. Обработка каждого из таких динамических запросов может отрицательно сказаться на производительности. Если вам потребуется выполнить статические запросы, которые являются неизменяемыми, и вы захотите избежать этих накладных расходов, то можете вместо этого обратиться к именованным запросам.

Именов нные з просы

Именованные запросы отличаются от динамических тем, что являются статическими и неизменяемыми. В дополнение к их статической природе, которая не обеспечивает гибкости динамических запросов, выполнение именованных запросов может быть более эффективным, поскольку поставщик постоянства получает возможность преобразовать JPQL-строки в SQL-операторы, как только запустится приложение, а не делать это каждый раз при выполнении запроса.

Именованные запросы — это статические запросы, выражаемые метаданными внутри либо аннотации `@NamedQuery`, либо XML-эквивалента. Для определения этих запросов, пригодных для повторного использования, снабдите сущность аннотацией `@NamedQuery`, которая принимает два элемента: имя запроса и его содержимое. Итак, внесем изменения для сущности `Customer` и статически определим три запроса с использованием аннотаций (листинг 6.22).

Листинг 6.22. Определение именов нных з просов для сущности Customer

```
@Entity
@NamedQueries({
  @NamedQuery(name = "findAll", query="select c from Customer c"),
  @NamedQuery(name = "findVincent",
              query="select c from Customer c where c.firstName = 'Винсент'"),
  @NamedQuery(name = "findWithParam",
              query="select c from Customer c where c.firstName = :fname")
})
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private Integer age;
  private String email;
  @OneToOne
  @JoinColumn(name = "address_fk")
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Поскольку для сущности Customer определяется более одного именованного запроса, здесь применяется аннотация @NamedQueries, которая принимает массив @NamedQuery. Первый запрос findAll обеспечивает выборку всех клиентов из базы данных без каких-либо ограничений (оператор WHERE отсутствует). Запрос findWithParam задействует параметр :fname для ограничения выборки клиентов согласно значению firstName. В листинге 6.22 показан массив @NamedQuery, однако если бы в случае с Customer имел место один запрос, то он выглядел бы следующим образом:

```
@Entity
@NamedQuery(name = "findAll ", query="select c from Customer c")
public class Customer {...}
```

Способ выполнения таких именованных запросов схож с тем, что применяется для выполнения динамических запросов. Вызывается метод EntityManager.createNamedQuery(), которому передается имя запроса, определяемое аннотациями. Этот метод возвращает Query или TypedQuery, который может быть использован для задания параметров, максимального количества результатов, режимов выборки и т. д. Чтобы выполнить запрос findAll, напишите следующий код:

```
Query query = em.createNamedQuery("findAll");
```

Опять-таки если у вас возникнет необходимость ввести запрос для возврата списка объектов Customer, то придется воспользоваться TypedQuery, как показано далее:

```
TypedQuery<Customer> query = em.createNamedQuery("findAll", Customer.class);
```

Следующий фрагмент кода вызывает именованный запрос findWithParam с передачей параметра :fname и присваиванием setMaxResults значения 3:

```
Query query = em.createNamedQuery("findWithParam");
query.setParameter("fname", "Винсент");
query.setMaxResults(3);
List<Customer> customers = query.getResultList();
```

Поскольку большинство методов Query API возвращает объект Query, вы можете использовать элегантное сокращение при написании запросов. Вызывайте методы один за другим (setParameter().setMaxResults() и т. д.):

```
Query query = em.createNamedQuery("findWithParam").setParameter("fname",
                                                    "Винсент")
                                                    .setMaxResults(3);
```

Именованные запросы пригодны для организации определений запросов и являются эффективным средством повышения производительности приложений. Это происходит благодаря тому, что именованные запросы для сущностей определяются статически и обычно располагаются в классе-сущности, который соответствует непосредственно результату запроса (здесь запрос findAll возвращает всех клиентов, поэтому он должен быть определен в сущности Customer).

Есть ограничение: областью видимости для имени запроса является та, что имеет место в случае с единицей сохраняемости. При этом имя должно быть уникальным в этой области видимости, то есть может присутствовать только один метод findAll.

Запрос findAll, который касается клиентов, и запрос findAll, касающийся адресов, должны именоваться по-разному. Общепринятая практика — снабжение имени запроса префиксом в виде имени сущности. Например, запрос findAll к сущности Customer имел бы имя Customer.findAll.

Другая проблема заключается в том, что имя запроса, являющееся строкой, подвергается манипуляциям, и, если вы допустите опечатку или выполните реорганизацию своего кода, то могут быть сгенерированы исключения, говорящие о том, что запрос не существует. Чтобы ограничить риски, вы можете заменить имя запроса константой. В листинге 6.23 показано, как произвести реорганизацию сущности Customer.

Листинг 6.23. Сущность Customer и определение именов нного з прос с использов нием конст нты

```
@Entity
@NamedQuery(name = Customer.FIND_ALL, query="select c from Customer c"),
public class Customer {
  public static final String FIND_ALL = "Customer.findAll";
  // Атрибуты, конструкторы, геттеры, сеттеры
}
```

Константа FIND_ALL однозначно идентифицирует запрос findAll путем снабжения его имени префиксом в виде имени сущности. Та же константа затем используется в аннотации @NamedQuery, и вы можете задействовать ее для выполнения запроса, как показано далее:

```
TypedQuery<Customer> query = em.createNamedQuery(Customer.FIND_ALL,
      Customer.class);
```

Criteria API (или объектно-ориентиров нные з просы)

До сих пор при написании JPQL-операторов (для выполнения динамических или именованных запросов) я использовал строки. Такой подход удобен тем, что позволяет писать краткие запросы к базам данных. Однако он обладает и недостатками, к которым относятся предрасположенность к ошибкам и сложность манипулирования с помощью внешнего фреймворка. Это строка, а в результате вы прибегаете к конкатенации строк, из-за чего можете допустить множество опечаток. Например, вы можете допустить опечатки в ключевых словах JPQL (SLECT вместо SELECT), именах классов (Custmer вместо Customer) или атрибутов (firstname вместо firstName). Вы также можете написать синтаксически неправильный оператор (SELECT c WHERE c.firstName = 'John' FROM Customer). Любая из этих ошибок будет обнаружена во время выполнения, а иногда может быть сложно выяснить, откуда происходит определенный дефект.

С выходом JPA 2.0 появился новый API-интерфейс под названием *Criteria API*, определенный в пакете javax.persistence.criteria. Он позволяет вам писать любые запросы объектно-ориентированным и синтаксически правильным образом. Большинство ошибок, которые разработчики могут допустить при написании операторов, выявляется во время компиляции, а не во время выполнения. Идея заключается

в том, что все ключевые слова JPQL (SELECT, UPDATE, DELETE, WHERE, LIKE, GROUP BY...) определены в этом API-интерфейсе. Другими словами, Criteria API поддерживает все, что может сделать JPQL, но с использованием основанного на объектах синтаксиса. Впервые взглянем на запрос, который извлекает всех клиентов, для которых firstName имеет значение Vincent. На JPQL он выглядел бы следующим образом:

```
SELECT c FROM Customer c WHERE c.firstName = 'Винсент'
```

Этот JPQL-оператор был переписан в листинге 6.24 объектно-ориентированным образом с использованием Criteria API.

Листинг 6.24. З прос с использов нием критериев, обеспечив ющий выборку всех клиентов, для которых firstName имеет зн чение Vincent

```
CriteriaBuilder builder = em.getCriteriaBuilder();
CriteriaQuery<Customer> criteriaQuery = builder.createQuery(Customer.class);
Root<Customer> c = criteriaQuery.from(Customer.class);
criteriaQuery.select(c).where(builder.equal(c.get("firstName"), "Винсент"));
Query query = em.createQuery(criteriaQuery).getResultList();
List<Customer> customers = query.getResultList();
```

Не вдаваясь в подробности, вы можете видеть, что у ключевых слов SELECT, FROM и WHERE имеется API-представление с помощью методов select(), from() и where(). Это правило относится ко всем ключевым словам JPQL. Запросы с использованием критериев генерируются благодаря интерфейсу CriteriaBuilder, который получает менеджер сущностей (атрибут em в листингах 6.24 и 6.25). Он включает методы для генерирования определения запроса (этот интерфейс определяет такие ключевые слова, как desc(), asc(), avg(), sum(), max(), min(), count(), and(), or(), greaterThan(), lowerThan()...). Еще одна роль CriteriaBuilder — выступать в качестве главной фабрики запросов с использованием критериев (CriteriaQuery) и элементов запросов с применением критериев. Этот интерфейс определяет такие методы, как select(), from(), where(), orderBy(), groupBy() и having(), которые имеют эквивалентное значение в JPQL. В листинге 6.24 получение псевдонима c (как в SELECT c FROM Customer) осуществляется с помощью интерфейса Root (Root<Customer> c). Таким образом, для того чтобы написать любой необходимый SQL-оператор, вам потребуется лишь воспользоваться CriteriaBuilder, CriteriaQuery и Root: от самого простого (выборка всех сущностей из базы данных) до самого сложного (соединения, подзапросы, выражения CASE, функции...).

Листинг 6.25. З прос с использов нием критериев, обеспечив ющий выборку всех клиентов ст рше 40 лет

```
CriteriaBuilder builder = em.getCriteriaBuilder();
CriteriaQuery<Customer> criteriaQuery = builder.createQuery(Customer.class);
Root<Customer> c = criteriaQuery.from(Customer.class);
criteriaQuery.select(c).where(builder.greaterThan(c.get("age").as(Integer.
class), 40));
Query query = em.createQuery(criteriaQuery).getResultList();
List<Customer> customers = query.getResultList();
```

Обратимся к другому примеру. В листинге 6.25 приведен запрос, который извлекает всех клиентов старше 40 лет. Выражение c.get("age") извлекает атрибут age из сущности Customer и проверяет, больше ли 40 его значение.

Я начал этот раздел, сказав, что Criteria API позволяет вам писать операторы без ошибок. Однако это не совсем так. Взглянув на листинги 6.24 и 6.25, вы тем не менее сможете увидеть строки (`"firstName"` и `"age"`), которые представляют атрибуты сущности `Customer`. Таким образом, опечатки все равно возможны. В листинге 6.25 нам даже потребовалось преобразовать `age` в `Integer` (`((c.get("age").as(Integer.class))`), поскольку по-другому никак нельзя было бы понять, что атрибут `age` имеет тип `Integer`. Для решения этих проблем Criteria API предусматривает класс статической метамодели для каждой сущности, что обеспечивает безопасность типов в данном API-интерфейсе.

Типобезоп сный Criteria API. Листинги 6.24 и 6.25 почти типобезопасны: каждое ключевое слово JPQL может быть представлено в методе интерфейсов `CriteriaBuilder` и `CriteriaQuery`. Единственной отсутствующей частью являются атрибуты сущности, которые основаны на строках: способ ссылки на атрибут `firstName` сущности `Customer` заключается в вызове `c.get("firstName")`. Метод `get` принимает строку в качестве параметра. Типобезопасный Criteria API решает проблему, переопределяя этот метод с использованием выражения пути из классов API метамодели, что обеспечивает безопасность типов.

В листинге 6.26 показана сущность `Customer` с несколькими атрибутами разных типов (`Long`, `String`, `Integer`, `Address`).

Листинг 6.26. Сущность Customer с несколькими трибут ми р зных типов

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private Integer age;
  private String email;
  private Address address;
  // Конструкторы, геттеры, сеттеры
}
```

Для обеспечения безопасности типов JPA 2.1 может генерировать класс статической метамодели для каждой сущности. В соответствии с соглашением каждая сущность `X` будет обладать классом метаданных с именем `X_` (со знаком подчеркивания). Таким образом, у сущности `Customer` будет свое представление метамодели, описанное в классе `Customer_`, который приведен в листинге 6.27.

Листинг 6.27. Кл сс Customer_, описыв ющий мет модель Customer

```
@Generated("EclipseLink")
@StaticMetamodel(Customer.class)
public class Customer_ {
  public static volatile SingularAttribute<Customer, Long> id;
  public static volatile SingularAttribute<Customer, String> firstName;
  public static volatile SingularAttribute<Customer, String> lastName;
  public static volatile SingularAttribute<Customer, Integer> age;
```

```
    public static volatile SingularAttribute<Customer, String> email;
    public static volatile SingularAttribute<Customer, Address> address;
}
```

В классе статической метамодели каждый атрибут сущности `Customer` определяется подклассом `javax.persistence.metamodel.Attribute` (`CollectionAttribute`, `ListAttribute`, `MapAttribute`, `SetAttribute` или `SingularAttribute`). Каждый из этих атрибутов использует обобщения и является строго типизированным (например, `SingularAttribute<Customer, Integer>, age`). В листинге 6.28 приведен точно такой же код, что и в листинге 6.25, но пересмотренный с использованием класса статической метамодели (`c.get("age")` превратилось в `c.get(Customer_.age)`). Еще одно преимущество безопасности типов заключается в том, что метамодель определяет атрибут `age` как имеющий тип `Integer`, поэтому нет необходимости преобразовывать его в `Integer` с помощью `as(Integer.class)`.

Листинг 6.28. Типобезоп сный з прос с использов нием критериев, обеспечив ющий выборку всех клиентов ст рше 40 лет

```
CriteriaBuilder builder = em.getCriteriaBuilder();
CriteriaQuery<Customer> criteriaQuery = builder.createQuery(Customer.class);
Root<Customer> c = criteriaQuery.from(Customer.class);
criteriaQuery.select(c).where(builder.greaterThan(c.get(Customer_.age), 40));
Query query = em.createQuery(criteriaQuery).getResultList();
List<Customer> customers = query.getResultList();
```

Опять-таки это лишь примеры того, что вы можете сделать с помощью Criteria API. Это очень богатый API-интерфейс, который всесторонне охарактеризован в главах 5 и 6 спецификации JPA 2.1.

ПРИМЕЧАНИЕ

Классы, используемые в случае со статической метамоделью, например Attribute или SingularAttribute, являются стандартными и определены в пакете javax.persistence.metamodel. Однако генерирование классов статической метамодели зависит от реализации. EclipseLink использует внутренний класс CanonicalModelProcessor. Этот класс может вызываться вашей интегрированной средой разработки, пока вы разрабатываете Java-команду, Ant-задание или Maven-плагин.

«Родные» з просы

JPQL обладает очень богатым синтаксисом, который позволяет обрабатывать сущности в любой форме и обеспечивает переносимость между базами данных. JPA дает возможность использовать специфические особенности баз данных благодаря «родным» запросам. «Родные» запросы принимают «родной» SQL-оператор (`SELECT`, `UPDATE` или `DELETE`) в качестве параметра и возвращают экземпляр `Query` для выполнения этого оператора. Однако нельзя рассчитывать на переносимость «родных» запросов между базами данных.

Если код не является переносимым, то почему бы не использовать JDBC-вызовы? Главная причина, в силу которой следует задействовать «родные» запросы JPA, состоит в том, что результат запроса будет автоматически преобразован

обратно в сущности. Если вы захотите извлечь все экземпляры сущности Customer из базы данных с использованием SQL, то вам потребуется прибегнуть к методу EntityManager.createNativeQuery(), принимающему в качестве параметров SQL-запрос и класс-сущность, в который должен быть отображен результат.

```
Query query = em.createNativeQuery("SELECT * FROM t_customer", Customer.
class);
List<Customer> customers = query.getResultList();
```

Как вы можете видеть в приведенном фрагменте кода, SQL-запрос является строкой, которая динамически генерируется во время выполнения (точно так же, как динамические JPQL-запросы). Опять-таки запрос может быть комплексным, и, поскольку поставщик не будет заранее знать о нем, он станет каждый раз интерпретировать его. Подобно именованным запросам, «родные» могут задействовать аннотации для определения статических SQL-запросов. Именованные «родные» запросы определяются с помощью аннотации @NamedNativeQuery, которую необходимо поместить в код любой сущности (см. код ниже). Как и в случае с именованными JPQL-запросами, имя запроса должно быть уникальным в единице сохраняемости.

```
@Entity
@NamedNativeQuery(name = "findAll", query="select * from t_customer")
@Table(name = "t_customer")
public class Customer {...}
```

З просы к хр нимым процедур м

До сих пор у всех разных запросов (JPQL или SQL) было одно и то же назначение: отправка запроса от вашего приложения к базе данных, которая выполнит его и отошлет назад результат. Хранимые процедуры отличаются в том смысле, что они фактически хранятся в самой базе данных и выполняются в ее рамках.

Хранимая процедура — это подпрограмма, имеющаяся в распоряжении приложений, которые осуществляют доступ к реляционной базе данных. Хранимые процедуры обычно используются для экстенсивной или комплексной обработки, которая требует выполнения нескольких SQL-операторов либо для решения повторяющихся задач, связанных с работой с большими объемами данных. Как правило, хранимые процедуры пишутся на том или ином языке, близком к SQL, и, следовательно, не являются легко переносимыми между базами данных от разных поставщиков. Однако сохранение кода в базе данных даже в непереносимой форме обеспечивает многие преимущества.

- Лучшую производительность благодаря предварительной компиляции хранимой процедуры, а также повторного использования плана ее выполнения.
- Сохранение статистики, касающейся кода, для поддержания его оптимизированным.
- Снижение количества данных, передаваемых по сети, благодаря сохранению кода на сервере.
- Изменение кода в центральной локации без репликации в нескольких разных программах.

- Хранимые процедуры, которые могут использоваться множеством программ, написанных на разных языках (а не только на Java).
- Скрытие необработанных данных путем предоставления доступа к информации только хранимым процедурам.
- Усиление мер безопасности путем предоставления пользователем разрешения на выполнение той или иной хранимой процедуры независимо от разрешений, связанных с базовой таблицей.

Взглянем на пример из практики — архивирование старых книг и компакт-дисков. После определенной даты книги и компакт-диски должны помещаться в архив на конкретном складе, а это означает, что затем их придется физически перевозить со склада к перекупщику. Архивирование книг и компакт-дисков может отнимать много времени, поскольку потребуется обновлять несколько таблиц (с именами, например, `T_Inventory`, `T_Warehouse`, `T_Book`, `T_CD`, `T_Transport` и т. д.). Таким образом, мы можем написать хранимую процедуру для перегруппировки нескольких SQL-операторов и повышения производительности. Хранимая процедура `sp_archive_books`, определенная в листинге 6.29, принимает `archiveDate` и `warehouseCode` в качестве параметров и обновляет таблицы `T_Inventory` и `T_Transport`.

Листинг 6.29. Абстракция хранимой процедуры, используемой при архивировании книг

```
CREATE PROCEDURE sp_archive_books @archiveDate DATE, @warehouseCode VARCHAR AS
  UPDATE T_Inventory
  SET Number_Of_Books_Left - 1
  WHERE Archive_Date < @archiveDate AND Warehouse_Code = @warehouseCode;

  UPDATE T_Transport
  SET Warehouse_To_Take_Books_From = @warehouseCode;
END
```

Хранимая процедура из листинга 6.29 помещается в базу данных, а затем может вызываться по своему имени (`sp_archive_books`). Как вы можете видеть, хранимая процедура принимает данные в виде входных или выходных параметров. Входные параметры (`@archiveDate` и `@warehouseCode` в нашем примере) используются при выполнении хранимой процедуры, которая, в свою очередь, может генерировать выходной результат. Этот результат возвращается приложению при использовании результирующего набора.

В JPA 2.1 интерфейс `StoredProcedureQuery` (который расширяет `Query`) поддерживает хранимые процедуры. В отличие от динамических, именованных или «родных» запросов, этот API-интерфейс позволяет вам вызывать только ту хранимую процедуру, которая уже присутствует в базе данных, но не определять ее. Вы можете вызывать хранимую процедуру с помощью аннотаций (`@NamedStoredProcedureQuery`) либо динамически.

В листинге 6.30 показана сущность `Book`, для которой объявляется хранимая процедура `sp_archive_books` с использованием аннотаций именованных запросов. Аннотация `@NamedStoredProcedureQuery` определяет имя хранимой процедуры для вызова, типы всех параметров (`Date.class` и `String.class`), их соответствующие направления параметров (`IN`, `OUT`, `INOUT`, `REF_CURSOR`), а также то, как должны быть отображены

результирующие наборы (при наличии таковых). Аннотацией @StoredProcedureParameter необходимо снабдить каждый параметр.

Листинг 6.30. Сущность, для которой объявляется именов нн я хр ним я процедур

```
@Entity
@NamedStoredProcedureQuery(name = "archiveOldBooks", procedureName =
                           "sp_archive_books",
  parameters = {
    @StoredProcedureParameter(name = "archiveDate", mode = IN, type = Date.class),
    @StoredProcedureParameter(name = "warehouse", mode = IN,
                              type = String.class)
  }
)
public class Book {
  @Id @GeneratedValue
  private Long id;
  private String title;
  private Float price;
  private String description;
  private String isbn;
  private String editor;
  private Integer nbOfPage;
  private Boolean illustrations;
  // Конструкторы, геттеры, сеттеры
}
```

Для вызова хранимой процедуры sp_archive_books вам потребуется прибегнуть к менеджеру сущностей и сгенерировать запрос к именованной хранимой процедуре, передав ее имя (archiveOldBooks). Этот запрос возвратит StoredProcedureQuery, для которого вы сможете задать параметры и выполнить его, как показано в листинге 6.31.

Листинг 6.31. Вызов StoredProcedureQuery

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("archiveOldBooks");
query.setParameter("archiveDate", new Date());
query.setParameter("maxBookArchived", 1000);
query.execute();
```

Если хранимая процедура не определена с использованием метаданных (@NamedStoredProcedureQuery), то вы можете прибегнуть к API-интерфейсу для динамического генерирования запроса к хранимой процедуре. Это означает, что параметры и информацию касаемо результирующего набора потребуется обеспечить программным путем. Это можно сделать с помощью метода registerStoredProcedureParameter интерфейса StoredProcedureQuery, как показано в листинге 6.32.

Листинг 6.32. Регистр ция и вызов StoredProcedureQuery

```
StoredProcedureQuery query =
em.createStoredProcedureQuery("sp_archive_old_books");
query.registerStoredProcedureParameter("archiveDate", Date.class,
ParameterMode.IN);
```

```
query.registerStoredProcedureParameter("maxBookArchived", Integer.class,
ParameterMode.IN);

query.setParameter("archiveDate", new Date());
query.setParameter("maxBookArchived", 1000);
query.execute();
```

Cache API

В большинстве спецификаций (а не только в спецификации Java EE) много внимания уделено функциональным требованиям, а нефункциональным требованиям, таким как производительность, масштабируемость или кластеризация, отводится роль деталей реализации. Реализации должны строго придерживаться спецификации, однако также могут привносить свои особенности. Идеальным примером в случае с JPA будет кэширование.

До выхода версии JPA 2.0 в спецификации кэширование не упоминалось. Менеджер сущностей является кэшем первого уровня, используемым для всесторонней обработки информации для базы данных, а также для кэширования сущностей, которые живут короткий период времени. Этот кэш первого уровня задействуется отдельно в случае с каждой транзакцией для уменьшения количества SQL-запросов в рамках определенной транзакции. Например, если объект будет модифицирован несколько раз в рамках одной и той же транзакции, то менеджер сущностей сгенерирует только один оператор `UPDATE` в конце этой транзакции. Кэш первого уровня не является производительным кэшем.

Однако все реализации JPA используют производительный кэш (также называемый кэшем второго уровня) для оптимизации доступа к базам данных, запросов, соединений и т. д. Как показано на рис. 6.3, кэш второго уровня располагается между менеджером сущностей и базой данных с целью уменьшения трафика путем сохранения объектов загруженными в память и доступными для всего приложения.

Каждая реализация наделяется своим подходом к кэшированию объектов путем либо разработки собственного механизма, либо повторного использования уже существующих решений (с открытым исходным кодом или коммерческих). Кэширование может быть распределено по кластеру либо нет — все возможно, когда спецификация игнорирует соответствующую тему. Создатели JPA 2.0 признали, что кэш второго уровня необходим, и добавили операции кэширования в стандартный API-интерфейс. Приведенный в листинге 6.33 API-интерфейс крайне минималистичен (поскольку цель JPA не заключается в том, чтобы стандартизировать полнофункциональный кэш), однако позволяет коду выполнять запросы к некоторым сущностям и удалять их из кэша второго уровня стандартным образом. Как и менеджер сущностей, `javax.persistence.Cache` является интерфейсом, реализуемым системой кэширования поставщика постоянства.

Листинг 6.33. Cache API

```
public interface Cache {
  // Содержит ли кэш определенную сущность
```

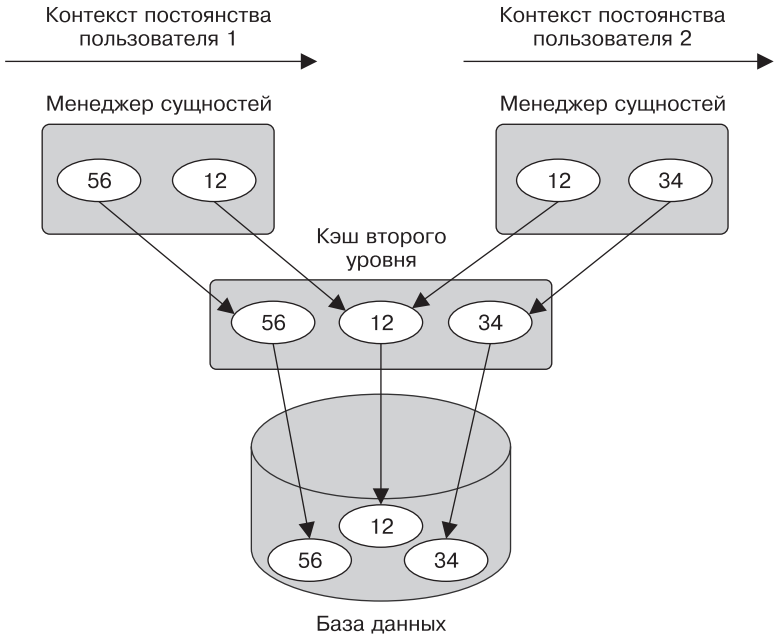


Рис. 6.3. Кэш второго уровня

```
    public boolean contains(Class cls, Object id);

    // Удаляет сущность из кэша
    public void evict(Class cls, Object id);
    // Удаляет сущности указанного класса (и его подклассы) из кэша
    public void evict(Class cls);
    // Очищает кэш
    public void evictAll();

    // Возвращает реализацию кэша, специфичную для поставщика
    public <T> T unwrap(Class<T> cls);
}
```

Вы можете использовать этот API для проверки того, содержится ли определенная сущность в кэше второго уровня, а также для очищения всего кэша. Задействуя этот API, вы можете явным образом проинформировать поставщика постоянства о том, является ли сущность кэшируемой, с помощью аннотации `@Cacheable`, как показано в листинге 6.34. Если у сущности отсутствует аннотация `@Cacheable`, то эта сущность и ее состояние не должны кэшироваться поставщиком.

Листинг 6.34. Сущность Customer является кэшируемой

```
@Entity
@Cacheable(true)
public class Customer {
  @Id @GeneratedValue
  private Long id;
```

```
    private String firstName;
    private String lastName;
    private String email;
    // Конструкторы, геттеры, сеттеры
}
```

Аннотация `@Cacheable` принимает логическое значение. Как только вы решите, какая сущность должна быть кэшируемой, вам придется проинформировать поставщика о том, какой механизм кэширования следует использовать. Это можно сделать с помощью JPA, указав атрибут `shared-cache-mode` в файле `persistence.xml`. Далее приведены возможные значения:

- `ALL` — будут кэшироваться все сущности, а также связанные с ними состояния и данные;
- `DISABLE_SELECTIVE` — будут кэшироваться все сущности за исключением тех, что снабжены аннотацией `@Cacheable(false)`;
- `ENABLE_SELECTIVE` — будут кэшироваться все сущности, снабженные аннотацией `@Cacheable(true)`;
- `NONE` — кэширование будет отключено для единицы сохраняемости;
- `UNSPECIFIED` — поведение кэширования будет неопределенным (могут быть применены правила по умолчанию, специфичные для поставщика).

Если не задать ни одно из этих значений, то поставщик будет сам решать, какой механизм кэширования применять. В коде, приведенном в листинге 6.35, показано, как использовать соответствующий механизм кэширования. Сначала мы создаем объект `Customer` и обеспечиваем его постоянство. Поскольку сущность `Customer` является кэшируемой (см. листинг 6.34), она должна быть размещена в кэше второго уровня (с помощью метода `EntityManagerFactory.getCache().contains()`). Вызов метода `ache.evict(Customer.class)` удаляет эту сущность из кэша.

Листинг 6.35. Сущность Customer является кэшируемой

```
Customer customer = new Customer("Патриция", "Джейн", "plecomte@mail.com");

tx.begin();
em.persist(customer);
tx.commit();

// Использует EntityManagerFactory для получения Cache
Cache cache = emf.getCache();

// Сущность Customer должна располагаться в кэше
assertTrue(cache.contains(Customer.class, customer.getId()));

// Удаляет сущность Customer из кэша
cache.evict(Customer.class);

// Сущность Customer больше не должна располагаться в кэше
assertFalse(cache.contains(Customer.class, customer.getId()));
```

Конкурентный доступ

JPA можно использовать для изменения постоянных данных, а JPQL — для извлечения данных, соответствующих определенным критериям. Все это может происходить в рамках приложения, выполняющегося в кластере с множеством узлов, множеством потоков и одной базой данных, благодаря чему осуществление конкурентного доступа к сущностям стало обычным явлением. Когда это имеет место, синхронизация должна контролироваться приложением с использованием механизма блокировки. Независимо от того, является приложение сложным или простым, есть вероятность, что вы решите применять блокировку где-нибудь в своем коде.

Чтобы проиллюстрировать проблему конкурентного доступа к базе данных, обратимся к примеру приложения с двумя конкурирующими потоками, показанными на рис. 6.4. Один поток ищет книгу по ее идентификатору и повышает цену книги на $2. Другой поток делает то же самое, но повышает цену книги на $5. Если вы выполните эти два потока одновременно в отдельных транзакциях, осуществляя манипуляции с одной и той же книгой, то не сможете предсказать окончательную цену книги. В этом примере начальная цена книги составляет $10. В зависимости от того, какая транзакция финиширует последней, цена может повыситься до $12 или 15.

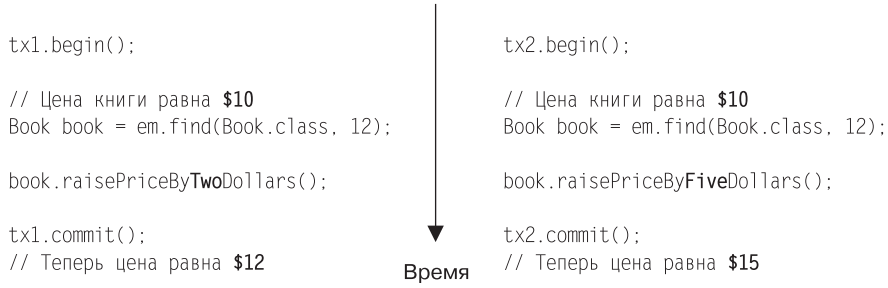


Рис. 6.4. Тр нз кции № 1 (tx1) и № 2 (tx2), одновременно обновляющие цену книги

Эта проблема конкурентного доступа, при которой «победителем» становится транзакция, которая фиксируется последней, неспецифична для JPA. При работе с базами данных эту проблему приходится решать с давних пор, и были найдены разные решения, позволяющие изолировать одну транзакцию от другой. Распространенный механизм, задействуемый базами данных, — это блокировка строки, в отношении которой выполняется SQL-оператор.

JPA 2.1 использует два разных механизма блокировки (версия JPA 1.0 поддерживала только оптимистическую блокировку).

- *Оптимистическая блокировка* основана на предположении, согласно которому большинство транзакций в базах данных не конфликтует с другими транзакциями, обеспечивая как можно более свободный конкурентный доступ, когда разрешается выполнение транзакций.
- *Пессимистическая блокировка* основана на противоположном предположении, в результате блокировка будет применяться к ресурсу до начала операций с ним.

В качества примера из повседневной жизни, подкрепляющего эти концепции, представьте себе «оптимистический и пессимистический переходы через улицу». Там, где движение транспорта совсем неинтенсивно, вы сможете перейти через дорогу, не проверяя, нет ли приближающихся автомобилей. Но вы не сможете так поступить в центре города!

JPA использует разные механизмы блокировки на разных уровнях API-интерфейса. Применение как пессимистической, так и оптимистической блокировки возможно с помощью методов `EntityManager.find` и `EntityManager.refresh` (в дополнение к методу `lock`), а также благодаря JPQL-запросам. Иначе говоря, блокировка может быть обеспечена на уровне менеджера сущностей и на уровне `Query` с помощью методов, приведенных в табл. 6.5 и 6.6.

Таблица 6.5. Методы менеджера сущностей для блокировки сущностей

Метод	Описание
<T> T find(Class<T> entityClass, Object primaryKey, LockModeType lockMode)	Выполняет поиск сущности указанного класса и первичного ключа, а затем блокирует ее согласно заданному типу режима блокировки
void lock(Object entity, LockModeType lockMode)	Блокирует экземпляр сущности, который содержится в контексте постоянства, согласно заданному типу режима блокировки
void refresh(Object entity, LockModeType lockMode)	Обновляет состояние экземпляра из базы данных, перезаписывая изменения, внесенные в сущность, при наличии таковых, и блокирует ее согласно заданному типу режима блокировки
LockModeType getLockMode(Object entity)	Извлекает значение текущего режима блокировки для экземпляра сущности

Таблица 6.6. Методы Query для блокировки JPQL-запросов

Метод	Описание
LockModeType getLockMode()	Извлекает значение текущего режима блокировки для запроса
Query setLockMode(LockModeType lockMode)	Задает тип режима блокировки для использования при выполнении запроса

Каждый из этих методов принимает `LockModeType` в качестве параметра, который, в свою очередь, может хранить разные значения:

- `OPTIMISTIC` — применяет оптимистическую блокировку;
- `OPTIMISTIC_FORCE_INCREMENT` — применяет оптимистическую блокировку и форсирует инкрементирование значения столбца `version`, связанного с сущностью (см. следующий подраздел «Контроль версий»);
- `PESSIMISTIC_READ` — применяет пессимистическую блокировку без необходимости в повторном чтении данных в конце транзакции для обеспечения блокировки;

- PESSIMISTIC_WRITE — применяет пессимистическую блокировку и форсирует сериализацию между транзакциями, которые пытаются обновить сущность;
- PESSIMISTIC_FORCE_INCREMENT — применяет пессимистическую блокировку и форсирует инкрементирование значения столбца version, связанного с сущностью (см. подраздел «Контроль версий» далее);
- NONE — определяет, что не должен использоваться никакой механизм блокировки.

Вы можете задавать эти параметры во многих местах в зависимости от того, как необходимо указать блокировки. Вы можете обеспечить чтение, а *затем* — блокировку.

```
Book book = em.find(Book.class, 12);
// Блокировать, чтобы повысить цену
em.lock(book, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
book.raisePriceByTwoDollars();
```

Либо вы можете обеспечить чтение *и* блокировку.

```
Book book = em.find(Book.class, 12, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
// Блокировка уже применена к сущности Book, повысить цену
book.raisePriceByTwoDollars();
```

Конкурентный доступ и блокировки являются ключевыми мотиваторами для контроля версий.

Контроль версий

В случае со спецификациями Java применяется контроль версий: Java SE 5.0, Java SE 6.0, EJB 3.1, JAX-RS 1.0 и т. д. При выходе новой версии JAX-RS ее номер увеличивается и вы переходите на JAX-RS 1.1. JPA использует этот точный механизм, когда вам необходимо присвоить номера версий сущностям. Таким образом, когда вы в первый раз обеспечите постоянство сущности в базе данных, она получит номер версии, равный 1. Позднее, если вы обновите атрибут и зафиксируете это изменение в базе данных, номер версии сущности будет равен уже 2 и т. д. Номер версии будет модифицироваться при каждом внесении изменений в сущность.

Чтобы это происходило, у сущности должен иметься атрибут для сохранения номера версии, снабженный аннотацией @Version. Он в дальнейшем отображается в столбец в базе данных. К числу типов атрибутов, поддерживающих контроль версий, относятся int, Integer, short, Short, long, Long и Timestamp. В листинге 6.36 показано, как добавить атрибут version в код сущности Book.

Листинг 6.36. Сущность Book с ннот цией @Version в случ е с Integer

```
@Entity
public class Book {
  @Id @GeneratedValue
  private Long id;
  @Version
  private Integer version;
  private String title;
  private Float price;
```

```
    private String description;
    private String isbn;
    private Integer nbOfPage;
    private Boolean illustrations;
    // Конструкторы, геттеры, сеттеры
}
```

Сущность может получить доступ к значению своего свойства `version`, но не имеет возможности модифицировать его. Только поставщику постоянства разрешено задавать или обновлять значение атрибута `version` при записи или обновлении объекта в базе данных. Взглянем на пример, иллюстрирующий поведение, которое наблюдается при этом контроле версий. В листинге 6.37 обеспечивается постоянство новой сущности `Book` в базе данных. Как только происходит фиксация транзакции, поставщик задает для `version` значение `1`. Далее цена книги обновляется и, как только информация сбрасывается в базу данных, номер версии инкрементируется, становясь равным `2`.

Листинг 6.37. Тр нз кции tx1 и tx2, одновременно обновляющие цену книги

```
Book book = new Book("H2G2", 21f, "Лучшая IT-книга", "123-456", 321, false);
tx.begin();
em.persist(book);
tx.commit();
assertEquals(1, book.getVersion());

tx.begin();
book.raisePriceByTwoDollars();
tx.commit();
assertEquals(2, book.getVersion());
```

Атрибут `version` необязателен для использования, но его рекомендуется применять, когда сущность может быть одновременно модифицирована несколькими процессами или потоками. Контроль версий представляет собой ядро оптимистической блокировки и обеспечивает защиту при редких одновременных модификациях сущности. Фактически к сущности может быть автоматически применена оптимистическая блокировка, если у нее имеется свойство, снабженное аннотацией `@Version`.

Оптимистическ я блокировк

Как видно из названия, оптимистическая блокировка основана на том факте, что транзакции в базах данных не конфликтуют друг с другом. Другими словами, высока вероятность того, что транзакция, обновляющая сущность, окажется единственной, которая в действительности будет обновлять сущность в этот промежуток времени. Следовательно, решение о применении блокировки к сущности на самом деле принимается в конце транзакции. Это гарантирует, что обновления сущности будут согласовываться с текущим состоянием базы данных. Результатом транзакций, которые привели бы к нарушению этого ограничения, стало бы генерирование исключения `OptimisticLockException`, а эти транзакции оказались бы помечены как подлежащие откату.

Как можно было бы сгенерировать исключение OptimisticLockException? Это можно сделать, либо применив явным образом блокировку к сущности (с помощью методов lock или find, которые вы видели ранее, передав LockModeType), либо разрешив поставщику постоянства проверить атрибут, снабженный аннотацией @Version. Использование специальной аннотации @Version в случае с сущностью позволяет менеджеру сущностей применить оптимистическую блокировку, просто сравнив значение атрибута version в экземпляре сущности со значением соответствующего столбца в базе данных. Если атрибут не будет аннотирован с использованием @Version, то менеджер сущностей не сможет применять оптимистическую блокировку автоматически (неявно).

Снова взглянем на пример повышения цены книги. Обе транзакции, tx1 и tx2, получают экземпляр одной и той же сущности Book. В тот момент номер версии сущности Book равен 1. Первая транзакция повышает цену книги на $2 и фиксирует это изменение. Когда информация сбрасывается в базу данных, поставщик постоянства увеличивает номер версии, делая его равным 2. В тот момент вторая транзакция поднимает цену на $5 и фиксирует это изменение. Менеджер сущностей для tx2 понимает, что номер версии в базе данных отличается от того, что имеется у сущности. Это означает, что номер версии был изменен в результате выполнения другой транзакции, из-за чего генерируется исключение OptimisticLockException, как показано на рис. 6.5.

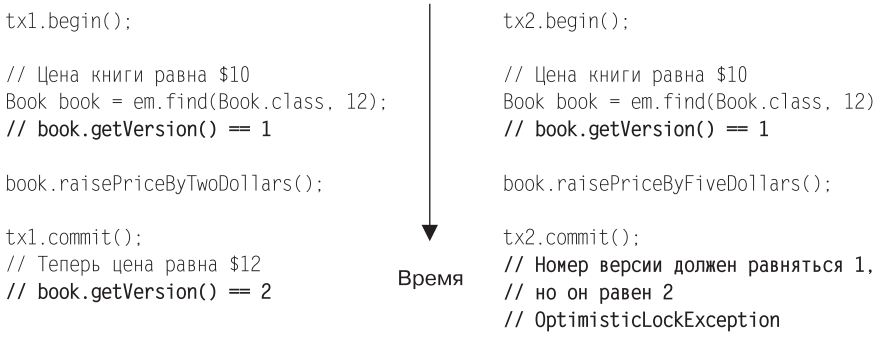


Рис. 6.5. Исключение OptimisticLockException, генерируемое в результ те выполнения тр нз кции tx2

Это поведение по умолчанию, которое наблюдается при использовании аннотации @Version: исключение OptimisticLockException генерируется при сбросе данных (во время фиксации либо с помощью явного вызова метода em.flush()). Вы также можете решать, где вам требуется добавить оптимистическую блокировку, обеспечивая чтение, а затем — блокировку либо чтение и блокировку. Например, код для обеспечения чтения и блокировки выглядел бы следующим образом:

```
Book book = em.find(Book.class, 12);
// Блокировать, чтобы повысить цену
em.lock(book, LockModeType.OPTIMISTIC);
book.raisePriceByTwoDollars();
```

При оптимистической блокировке LockModeType, передаваемый вами в качестве параметра, может принимать два значения: OPTIMISTIC и OPTIMISTIC_FORCE_INCREMENT (или соответственно READ и WRITE, однако эти значения устарели). Единственное отличие заключается в том, что OPTIMISTIC_FORCE_INCREMENT форсирует обновление (инкрементирование) значения столбца version, связанного с сущностью.

При работе с приложениями настоятельно рекомендуется делать возможной оптимистическую блокировку всех сущностей, к которым разрешен конкурентный доступ. Неиспользование механизма блокировки может привести к несогласованному состоянию сущности, потерянным обновлениям и другим нарушениям состояния. Оптимистическая блокировка — полезная оптимизация производительности, освобождающая от работы, которую в ином случае потребовалось бы выполнить базе данных. Она представляет собой альтернативу пессимистической блокировке, которая подразумевает применение низкоуровневой блокировки базы данных.

Пессимистическая блокировка

Пессимистическая блокировка основана на предположении, противоположном тому, которое действует для оптимистической блокировки, так как пессимистическая блокировка быстро применяется к сущности до начала операций с ней. Это очень ограничивает ресурсы и приводит к значительному снижению производительности, так как блокировка базы данных поддерживается с использованием SQL-оператора SELECT ... FOR UPDATE для чтения данных.

Базы данных обычно предлагают службу для обеспечения пессимистической блокировки. Такая служба позволяет менеджеру сущностей блокировать строку таблицы для предотвращения обновления этой же строки другим потоком. Это эффективный механизм, который гарантирует, что два клиента не смогут одновременно модифицировать одну и ту же строку, однако он требует проведения затратных низкоуровневых проверок в базе данных. Результатом транзакций, которые привели бы к нарушению этого ограничения, стало бы генерирование исключения PessimisticLockException, а эти транзакции были бы помечены как подлежащие откату.

Оптимистическая блокировка целесообразна, когда вы сталкиваетесь с умеренным соперничеством между конкурирующими транзакциями. Однако в некоторых приложениях с более высокой степенью риска соперничества уместнее может оказаться пессимистическая блокировка, поскольку блокировка базы данных применяется незамедлительно в противоположность сбоям оптимистических транзакций, случающимся позднее. Например, во времена экономических кризисов на фондовые рынки поступает огромное количество поручений на продажу. Если одновременно 100 миллионам американцев потребуется продать ценные бумаги, то системе придется прибегнуть к пессимистическим блокировкам для обеспечения согласованности данных. Следует отметить, что в настоящее время рынок настроен довольно пессимистично, а не оптимистично, однако это никак не связано с JPA.

Пессимистическая блокировка может применяться к сущностям, которые не включают аннотированный атрибут @Version.

Жизненный цикл сущности

Вам уже известно большинство секретов сущностей, поэтому взглянем на их жизненный цикл. Созданный (с использованием оператора `new`) экземпляр сущности виртуальная машина Java рассматривает как простой Java-объект (то есть отсоединенный), который может быть использован приложением в качестве простого объекта. Затем, когда менеджер сущностей обеспечивает постоянство этой сущности, считается, что она находится под управлением. Когда сущность будет находиться под управлением, менеджер сущностей автоматически синхронизирует значения ее атрибутов с основной базой данных (например, если вы измените значение атрибута с помощью метода `set`, а сущность при этом будет находиться под управлением, то это новое значение окажется автоматически синхронизировано с базой данных).

Чтобы лучше понять этот процесс, взгляните на рис. 6.6, где приведена UML-диаграмма состояний. На ней показаны переходы между всеми состояниями сущности `Customer`.

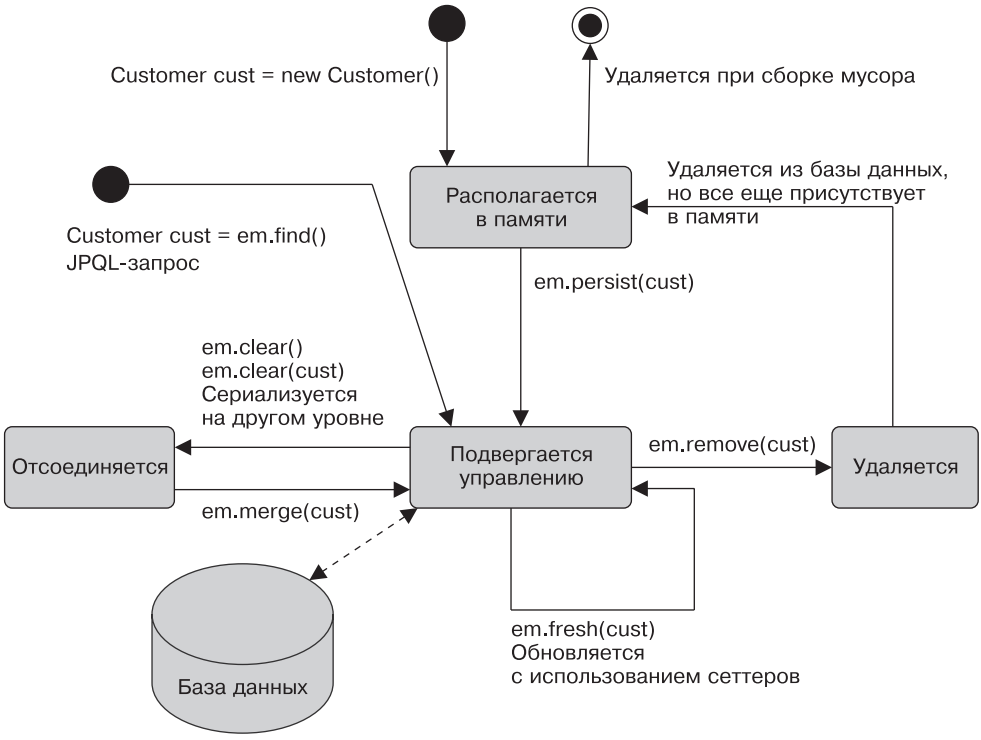


Рис. 6.6. Жизненный цикл сущности

Чтобы создать экземпляр сущности `Customer`, вы используете оператор `new`. Этот объект затем располагается в памяти, хотя JPA ничего о нем не знает. Если вы не станете что-либо делать с объектом, то он окажется вне области видимости и в ито-

ге будет удален при сборке мусора, что станет концом его жизненного цикла. Далее вы можете обеспечить постоянство Customer с помощью метода EntityManager.persist(). В тот момент сущность оказывается под управлением, а ее состояние синхронизируется с базой данных. Пока сущность пребывает в этом состоянии, в котором она подвергается управлению, вы можете обновить атрибуты с использованием методов-сеттеров (например, customer.setFirstName()) или обновить содержимое с помощью метода EntityManager.refresh(). Все эти изменения будут синхронизированы между сущностью и базой данных. Если вы вызовете метод EntityManager.contains(customer), пока сущность пребывает в этом состоянии, то он возвратит true, поскольку сущность Customer содержится в контексте постоянства (то есть находится под управлением).

Сущность также может оказаться под управлением при загрузке из базы данных. Когда вы используете метод EntityManager.find() или генерируете JPQL-запрос для извлечения списка сущностей, все сущности автоматически подвергаются управлению и вы можете начать обновлять или удалять их атрибуты.

Пока сущность пребывает в состоянии, в котором она подвергается управлению, вы можете вызвать метод EntityManager.remove(), в результате чего эта сущность окажется удалена из базы данных и больше не будет находиться под управлением. Однако Java-объект продолжит располагаться в памяти, и вы все еще сможете использовать его до тех пор, пока сборщик мусора не избавится от этого объекта.

Теперь взглянем на состояние, в котором сущность является отсоединенной. Вы уже видели в предыдущей главе, как явный вызов метода EntityManager.clear() или EntityManager.detach(customer) приводит к удалению сущности из контекста постоянства; она оказывается отсоединенной. Однако есть другой, более тонкий способ отсоединить сущность: ее сериализация.

Во многих примерах в этой книге сущности ничего не реализуют, однако если им потребуется пересечь сеть при удаленном вызове или пересечь уровни для вывода на уровне представления, то им потребуется реализовать интерфейс java.io.Serializable. Это не JPA-, а Java-ограничение. Когда находящаяся под управлением сущность сериализуется, пересекает сеть и десериализуется, она рассматривается как отсоединенный объект. Чтобы снова присоединить сущность, вам потребуется вызвать метод EntityManager.merge(). Распространенный сценарий использования — вариант, когда сущность задействуется в JSF-странице. Допустим, сущность Customer выводится в форме на удаленной JSF-странице, подлежащей обновлению. Будучи удаленной, сущность нуждается в сериализации на стороне сервера перед отправкой на уровень представления. В тот момент сущность автоматически отсоединяется. После вывода, если какие-либо данные изменятся и их понадобится обновить, форма будет отправлена, а сущность — отослана обратно на сервер, десериализована. Она потребует слияния, чтобы снова присоединиться.

Методы обратного вызова и слушатели позволяют вам добавлять собственную бизнес-логику, когда с сущностью происходят определенные события жизненного цикла.

Обр тные вызовы

Операции, выполняемые с сущностями во время их жизненного цикла, подпадают под четыре категории: обеспечение постоянства, обновление, удаление и загрузка. При этом они аналогичны категориям операций с базами данных, к которым относятся соответственно вставка, обновление, удаление и выборка. Во время каждого жизненного цикла имеют место события с приставками `pre` и `post`, которые могут быть перехвачены менеджером сущностей для вызова бизнес-метода. Такие бизнес-методы должны быть снабжены одной из аннотаций, описанных в табл. 6.7. Эти аннотации могут применяться к методам класса-сущности, отображенного суперкласса или класса-слушателя обратных вызовов.

Т блиц 6.7. Аннот ции обр тных вызовов жизненного цикл

Аннот ция	Опис ние
@PrePersist	Помечает метод для вызова до выполнения EntityManager.persist()
@PostPersist	Помечает метод для вызова после того, как будет обеспечено постоянство сущности. Если сущность будет автоматически генерировать свой первичный ключ (с использованием @GeneratedValue), то значение окажется доступно в соответствующем методе
@PreUpdate	Помечает метод для вызова до выполнения операции обновления в отношении базы данных (путем вызова сеттеров сущности или метода EntityManager.merge())
@PostUpdate	Помечает метод для вызова после выполнения операции обновления в отношении базы данных
@PreRemove	Помечает метод для вызова до выполнения EntityManager.remove()
@PostRemove	Помечает метод для вызова после того, как сущность будет удалена
@Postload	Помечает метод для вызова после того, как сущность будет загружена (посредством JPQL-запроса или EntityManager.find()) либо обновлена из основной базы данных. Аннотации @Preload не существует, поскольку в предварительной загрузке данных для сущности, которая еще не создана, нет смысла

Результатом добавления аннотаций обратных вызовов в UML-диаграмму состояний, показанную на рис. 6.6, является диаграмма, которую можно увидеть на рис. 6.7.

Перед вставкой сущности в базу данных менеджер сущностей вызывает метод, снабженный аннотацией `@PrePersist`. Если вставка не приведет к генерированию исключения, то будет обеспечено постоянство сущности, инициализирован ее идентификатор, а затем вызван метод, снабженный аннотацией `@PostPersist`. Аналогичное поведение наблюдается при операциях обновления (`@PreUpdate`, `@PostUpdate`) и удаления (`@PreRemove`, `@PostRemove`). Метод, аннотированный с использованием `@PostLoad`, вызывается после загрузки сущности из базы данных (посредством `EntityManager.find()` или JPQL-запроса). Когда сущность отсоединяется и требуется произвести ее слияние, менеджер сущностей сначала должен проверить, имеются ли какие-либо отличия от информации, содержащейся в базе данных (`@PostLoad`). Если да, то ему надлежит обновить данные (`@PreUpdate`, `@PostUpdate`).

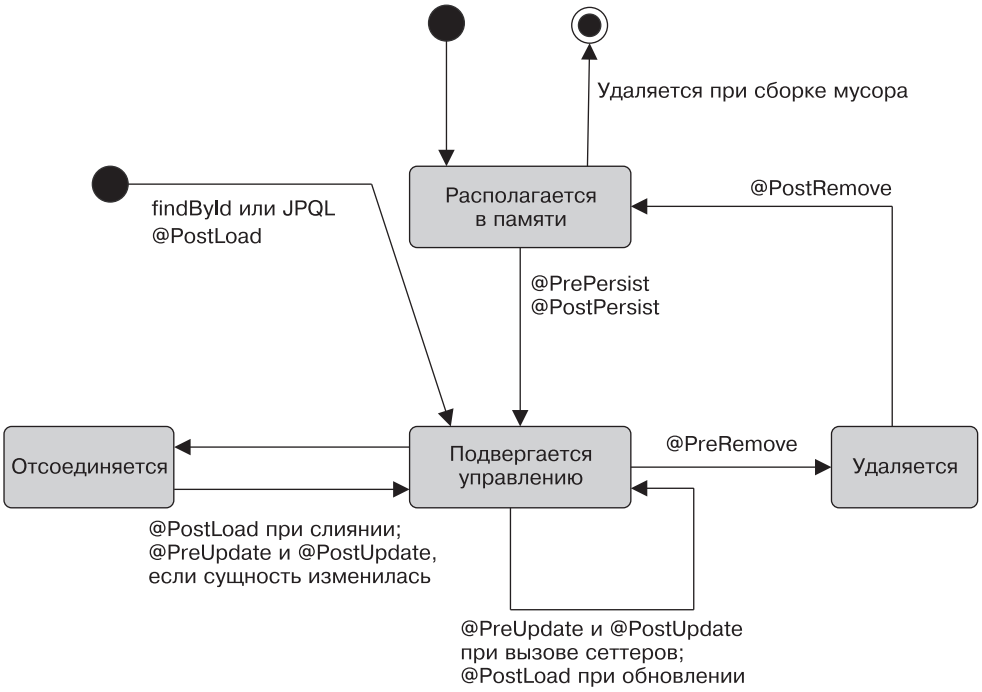


Рис. 6.7. Жизненный цикл сущности с ннот циями обр тных вызовов

Как все это выглядит в коде? Сущности могут включать не только атрибуты, конструкторы, геттеры и сеттеры, но и бизнес-логику, используемую для валидации их состояния или выполнения вычислений для некоторых их атрибутов. Сюда могут входить обычные Java-методы, вызываемые другими классами, или аннотации обратных вызовов (которые также называются методами обратного вызова), как показано в листинге 6.38. Менеджер сущностей вызывает их автоматически в зависимости от инициируемого события.

Листинг 6.38. Сущность Customer с ннот циями обр тных вызовов

```
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Temporal(TemporalType.DATE)
  private Date dateOfBirth;
  @Transient
  private Integer age;
  @Temporal(TemporalType.TIMESTAMP)
```

```
  private Date creationDate;

  @PrePersist
  @PreUpdate
  private void validate() {
    if (firstName == null || "".equals(firstName))
      throw new IllegalArgumentException("Неверное имя");
    if (lastName == null || "".equals(lastName))
      throw new IllegalArgumentException("Неверная фамилия");
  }

  @PostLoad
  @PostPersist
  @PostUpdate
  public void calculateAge() {
    if (dateOfBirth == null) {
      age = null;
      return;
    }

    Calendar birth = new GregorianCalendar();
    birth.setTime(dateOfBirth);
    Calendar now = new GregorianCalendar();
    now.setTime(new Date());
    int adjust = 0;
    if (now.get(DAY_OF_YEAR) - birth.get(DAY_OF_YEAR) < 0) {
      adjust = -1;
    }
    age = now.get(YEAR) - birth.get(YEAR) + adjust;
  }
  // Конструкторы, геттеры, сеттеры
}
```

В листинге 6.38 сущность `Customer` включает метод для валидации ее данных (он проверяет атрибуты `firstName` и `lastName`). Этот метод аннотирован с использованием `@PrePersist` и `@PreUpdate` и будет вызываться до вставки данных или обновления информации в базе данных. Если данные не смогут успешно пройти валидацию, то будет сгенерировано исключение времени выполнения, а вставка или обновление будут подвергнуты откату для гарантии того, что данные, вставленные или обновленные в базе данных, окажутся валидными.

Метод `calculateAge()` вычисляет возраст клиента. Атрибут `age` временный и не отображается в базе данных. После загрузки сущности, обеспечения ее постоянства или обновления метод `calculateAge()` принимает значение даты рождения клиента, вычисляет его возраст и задает значение для соответствующего атрибута.

Приведенные далее правила применяются к методам обратного вызова жизненного цикла.

- Методы могут иметь доступ `public`, `private`, `protected` или доступ на уровне пакета, однако не должны быть `static` или `final`. Обратите внимание в листинге 6.38 на то, что метод `validate()` является `private`.

- Метод может быть снабжен множественными аннотациями событий жизненного цикла (метод `validateData()` аннотирован с использованием `@PrePersist` и `@PreUpdate`). Однако для класса-сущности может присутствовать только одна аннотация жизненного цикла определенного типа (например, для одной и той же сущности не может быть две аннотации `@PrePersist`).
- Метод может генерировать непроверяемые исключения (времени выполнения), но не может — проверяемые. Генерирование исключения времени выполнения приведет к откату транзакции при наличии таковой.
- Метод может вызывать JNDI, JDBC, JMS и EJB-компоненты, но не может осуществлять какие-либо операции `EntityManager` и `Query`.
- При наследовании, если метод указан в суперклассе, он будет вызываться раньше соответствующего метода в дочернем классе. Например, если бы в листинге 6.38 сущность `Customer` наследовала от сущности `Person`, то метод `Person` с аннотацией `@PrePersist` вызывался бы раньше метода `Customer` с аннотацией `@PrePersist`.
- Если при работе со связями используется каскадирование событий, то метод обратного вызова тоже будет вызываться каскадным образом. Допустим, сущность обладает коллекцией адресов, а в случае со связью задано каскадное удаление. При удалении `Customer` произошел бы вызов метода `Address` с аннотацией `@PreRemove`, а также метода `Customer` с аннотацией `@PreRemove`.

Слуш тели

Методы обратного вызова, которыми располагает сущность, хорошо работают при наличии бизнес-логики, связанной только с этой сущностью. Слушатели сущностей применяются для извлечения бизнес-логики в отдельный класс и обеспечения ее совместного использования другими сущностями. Слушатель сущности — это простой Java-объект, в случае с которым вам требуется определить один или несколько методов обратного вызова жизненного цикла. Для регистрации слушателя сущности необходимо задействовать аннотацию `@EntityListeners`.

Используя пример `Customer`, извлечем методы `calculateAge()` и `validate()` в отдельные классы-слушатели: соответственно `AgeCalculationListener` (листинг 6.39) и `DataValidationListener` (листинг 6.40).

Листинг 6.39. Слуш тель, используемый для вычисления зн чения age сущности Customer

```
public class AgeCalculationListener {
  @PostLoad
  @PostPersist
  @PostUpdate
  public void calculateAge(Customer customer) {
    if (customer.getDateOfBirth() == null) {
      customer.setAge(null);
      return;
    }

    Calendar birth = new GregorianCalendar();
```

```
      birth.setTime(customer.getDateOfBirth());
      Calendar now = new GregorianCalendar();
      now.setTime(new Date());
      int adjust = 0; if (now.get(DAY_OF_YEAR) - birth.get(DAY_OF_YEAR) < 0) {
        adjust = -1;
      }
      customer.setAge(now.get(YEAR) - birth.get(YEAR) + adjust);
    }
  }
```

Листинг 6.40. Слуш тель, используемый для в лид ции трибутов сущности Customer

```
public class DataValidationListener {
  @PrePersist
  @PreUpdate
  private void validate(Customer customer) {
    if (customer.getFirstName() == null || "".equals(customer.getFirstName()))
      throw new IllegalArgumentException("Неверное имя");
    if (customer.getLastName() == null || "".equals(customer.getLastName()))
      throw new IllegalArgumentException("Неверная фамилия");
  }
}
```

К классу-слушателю применяются только простые правила. Первое правило состоит в том, что класс должен располагать конструктором `public` без аргументов. Второе правило заключается в том, что подписи методов обратного вызова немного отличаются от тех, что приведены в листинге 6.38. Когда вы вызываете метод обратного вызова в слушателе, этому методу необходимо иметь доступ к состоянию сущности (например, к `firstName` и `lastName` сущности `Customer`, которые необходимо подвергнуть валидации). Методы должны располагать параметром, имеющим тип, который совместим с типом сущности, поскольку сущность, связанная с соответствующим событием, передается в обратный вызов. Метод обратного вызова, определенный в сущности, будет иметь такую подпись без параметров:

```
void <МЕТОД>();
```

Методы обратного вызова, определенные для слушателя сущности, могут иметь подписи двух разных типов. Если метод будет использоваться в нескольких сущностях, то у него должен быть аргумент `Object`:

```
void <МЕТОД>(Object anyEntity)
```

Если он предназначен только для одной сущности или ее подклассов (при наследовании), то параметр может иметь тип сущности:

```
void <МЕТОД>(Customer customerOrSubclasses)
```

Для обозначения того, что эти два слушателя будут уведомляться о событиях жизненного цикла сущности `Customer`, вам необходимо использовать аннотацию `@EntityListeners` (листинг 6.41). Она может принимать в качестве параметра один слушатель сущности либо массив слушателей. Если будет определено несколько

слушателей и произойдет событие жизненного цикла, то поставщик постоянства произведет итерацию по каждому слушателю в том порядке, в котором они указаны, и вызовет метод обратного вызова, передав ссылку на сущность, к которой относится соответствующее событие. Затем он вызовет методы обратного вызова в самой сущности (при наличии таковых).

Листинг 6.41. Сущность Customer, для которой определяется дв слуш теля

```
@EntityListeners({DataValidationListener.class, AgeCalculationListener.class})
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Temporal(TemporalType.DATE)
  private Date dateOfBirth;
  @Transient
  private Integer age;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  // Конструкторы, геттеры, сеттеры
}
```

Результат выполнения этого кода будет точно таким же, что и кода из листинга 6.38. Сущность `Customer` валидирует свои данные перед вставкой или обновлением с использованием метода `DataValidationListener.validate()` и вычислит значение своего `age` с помощью метода слушателя `AgeCalculationListener.calculateAge()`.

Правила, которых должны придерживаться методы слушателя сущности, аналогичны правилам для методов обратного вызова сущности, за исключением нескольких деталей.

- Могут генерироваться только непроверяемые исключения. Это приводит к тому, что остальные слушатели и методы обратного вызова не вызываются, а транзакция подвергается откату (при наличии таковой).
- В иерархии наследования, если слушатели определены для множественных сущностей, слушатели, определенные в суперклассе, вызываются раньше слушателей, определенных в подклассах. Если не требуется, чтобы сущность наследовала слушателей суперкласса, то можно явным образом исключить их, используя аннотацию `@ExcludeSuperclassListeners` (или ее XML-эквивалент).

В листинге 6.41 показан код сущности `Customer`, где определяется два слушателя, однако слушатель может быть определен и для нескольких сущностей. Это может оказаться полезным в ситуациях, где слушатель обеспечивает общую логику, из которой могут извлечь выгоду многие сущности. Например, вы могли бы создать `DebugListener`, который будет выводить имена отдельных инициируемых событий, как показано в листинге 6.42.

Листинг 6.42. DebugListener, который может быть использов н любой сущностью

```
public class DebugListener {
  @PrePersist
  void prePersist(Object object) {
    System.out.println("prePersist");
  }
  @PreUpdate
  void preUpdate(Object object) {
    System.out.println("preUpdate");
  }
  @PreRemove
  void preRemove(Object object) {
    System.out.println("preRemove");
  }
}
```

Обратите внимание, что каждый метод принимает `Object` в качестве параметра, а это означает, что сущность любого типа может использовать этот слушатель при добавлении класса `DebugListener` в свою аннотацию `@EntityListeners`. Чтобы любая сущность вашего приложения применяла этот слушатель, вам пришлось бы пройтись по каждой и добавить их вручную в аннотацию. На этот случай в JPA предусмотрено такое понятие, как слушатели по умолчанию, которые могут охватывать все сущности в контексте постоянства. Поскольку аннотация, нацеленная на всю область видимости единицы сохраняемости, отсутствует, слушатели по умолчанию могут быть объявлены только в файле отображения XML.

В предыдущей главе вы видели, как использовать файлы отображения XML вместо аннотаций. Для определения `DebugListener` в качестве слушателя по умолчанию придется выполнить те же самые шаги. Потребуется создать файл отображения с XML, определенным в листинге 6.43, и произвести его развертывание с использованием приложения.

Листинг 6.43. DebugListener, определенный в к честве слуш теля по умолч нию

```
<?xml version="1.0" encoding="UTF-8"?>
<entity-mappings xmlns="http://xmlns.jcp.org/xml/ns/persistence/orm"
                 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" ➥
                 xsi:schemaLocation=" http://xmlns.jcp.org/xml/ns/persistence/orm
                 http://xmlns.jcp.org/xml/ns/persistence/orm_2_1.xsd"
                 version="2.1">

   <persistence-unit-metadata>
      <persistence-unit-defaults>
         <entity-listeners>
            <entity-listener class="org.agoncal.book.javaee7.chapter06.
DebugListener"/>
         </entity-listeners>
      </persistence-unit-defaults>
   </persistence-unit-metadata>
</entity-mappings>
```

В этом файле тег <persistence-unit-metadata> определяет все метаданные, у которых нет какого-либо эквивалента в виде аннотаций. Тег <persistence-unit-defaults> задает все настройки по умолчанию для единицы сохраняемости, а тег <entity-listener> определяет слушателя по умолчанию. В persistence.xml должна иметься ссылка на этот файл, развертывание которого необходимо выполнить с использованием приложения. Тогда DebugListener будет автоматически вызываться для каждой сущности.

Если вы объявите список слушателей сущности по умолчанию, то каждый из них будет вызываться в том порядке, в котором они указаны в файле отображения XML. Слушатели сущности по умолчанию всегда вызываются раньше любых слушателей сущности, определенных в аннотации @EntityListeners. Если потребуется, чтобы для сущности не применялись слушатели по умолчанию, то можно использовать аннотацию @ExcludeDefaultListeners, как показано в листинге 6.44.

Листинг 6.44. Сущность Customer, для которой исключ ются слуш тели по умолч нию

```
@ExcludeDefaultListeners
@Entity
public class Customer {
  @Id @GeneratedValue
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private String phoneNumber;
  @Temporal(TemporalType.DATE)
  private Date dateOfBirth;
  @Transient
  private Integer age;
  @Temporal(TemporalType.TIMESTAMP)
  private Date creationDate;
  // Конструкторы, геттеры, сеттеры
}
```

Когда будет инициировано событие, слушатели станут выполняться в следующем порядке.

1. Указанные в @EntityListeners слушатели для определенной сущности или суперкласса в порядке расположения в массиве.
2. Слушатели сущности для суперклассов (в первую очередь станут выполняться те, что располагаются в самом верху списка).
3. Слушатели сущности для сущности.
4. Обратные вызовы суперклассов (в первую очередь будут выполняться те, что располагаются в самом верху списка).
5. Обратные вызовы сущности.

Резюме

Из этой главы вы узнали, как выполнять запросы к сущностям. Менеджер сущностей — это основной инструмент, используемый для взаимодействия с постоянными

сущностями. Он может создавать, обновлять сущности, выполнять поиск по идентификатору, удалять и синхронизировать сущности с базой данных с помощью контекста постоянства, который играет роль кэша первого уровня. JPA также сопутствует очень мощный язык запросов JPQL, который не зависит от поставщика базы данных. Вы можете извлекать сущности благодаря богатому синтаксису, задействуя операторы `WHERE`, `ORDER BY` или `GROUP BY`, а при конкурентном доступе к своим сущностям вы будете знать, как использовать контроль версий и когда следует применять оптимистическую или пессимистическую блокировку.

В этой главе также описан жизненный цикл сущности и то, как менеджер сущностей перехватывает события, чтобы запускать методы обратного вызова. Такие методы могут определяться в одной сущности и снабжаться несколькими аннотациями (`@PrePersist`, `@PostPersist` и т. д.). Метод также может извлекаться в классы-слушатели и использоваться несколькими или всеми сущностями (с применением слушателей сущностей по умолчанию). Благодаря методам обратного вызова вы понимаете, что сущности не просто анемичные объекты (объекты без бизнес-логики, располагающие только атрибутами, геттерами и сеттерами). Сущности способны включать бизнес-логику, которая может вызываться другими объектами в приложении, либо вызываться автоматически с помощью менеджера сущностей в зависимости от жизненного цикла сущности.